UFUG 2104
Assignment 1: Project Report

Jincan LI

May 2, 2026

1 Introduction

1.1 Dataset Overview

This project analyzes the *Billionaires Statistics Dataset (2023)*, a cross-sectional snapshot of $n = 2{,}640$ records capturing the world's wealthiest individuals. Each record carries 35 raw variables spanning wealth metrics (`finalWorth`), personal demographics (`age`, `gender`, `selfMade`), geographic identifiers (`country`, `city`), business information (`category`, `source`, `organization`), and country-level macroeconomic indicators (`gdp_country`, `population_country`, `life_expectancy_country`, etc.). All monetary values are in millions of USD.

1.2 Research Context and Theoretical Framing

Why billionaire wealth is a window into economic inequality. The distribution of billionaire wealth is not merely a curiosity—it sits at the intersection of several major economic debates: the measurement of inequality (Pareto's law [1], the Gini coefficient [2]), the mechanisms of capital accumulation (Romer's endogenous growth theory [3], Piketty's $r > g$ framework [4]), and the empirical study of whether national prosperity translates into broad-based wealth creation.

Three analytical layers drive this report:

1. **The tail-layer problem**: Billionaire wealth lives in the extreme upper tail of the global wealth distribution, where standard statistical methods (arithmetic mean, variance-based tests, raw-scale OLS) can be dominated by a handful of observations. This motivates log transforms, non-parametric methods, and robust inference.

2. **The association vs. causation problem**: Correlations between national GDP and individual billionaire wealth are inherently observational. The analysis estimates the *conditional association* (not causal effect) of macroeconomic context with individual wealth, while acknowledging confounders such as capital market depth, institutional quality, tax regimes, and family network density.

3. **The heterogeneity problem**: Aggregate correlations obscure substantial heterogeneity across industries and wealth-accumulation paths. A single pooled estimate may be simultaneously uninterpretable for an entrepreneur in Shenzhen and misleading for a fashion billionaire in Milan.

These three layers structure the narrative arc of this report: first diagnosing the heavy-tailed distribution, then quantifying associations with uncertainty, and finally testing whether those associations hold across subgroups and under alternative specifications.

1.3 Research Questions

Table 1 presents the seven research questions that structure this report.

Table 1: Research questions and their mapping to analytical modules.

rq_id	question	primary_module
RQ0	RQ0: Does data quality (missingness/outliers/duplicates) systematically bias certain countries/industries and affect inference?	M2
RQ1	RQ1: Is finalWorth heavy-tailed, and how dominant is the top 1% for means/variance/correlation/regression?	M4
RQ2	RQ2: Do selfMade, gender, and category differ in finalWorth? What are the effect sizes and confidence intervals?	M6
RQ3	RQ3: Is finalWorth associated with macro variables (e.g., GDP), and is the association robust to log transforms and stratification?	M5/M6
RQ4	RQ4: How well does gdp_country predict finalWorth in a simple regression, and are results stable under log, multivariate controls, and robust SE?	M7
RQ5	RQ5 (optional): Quantile regression — does the GDP association differ across wealth quantiles?	M7 (extra)
RQ6	RQ6 (optional): Permutation / bootstrap inference to reduce distributional assumptions.	M6/M8 (extra)

1.4 Analytical Logic Chain

The report follows a deliberately ordered evidence chain rather than a flat catalogue of outputs. First, the raw file is frozen and its record logic is checked, because duplicated aliases, structural missingness, and country-level macro variables determine what can be modeled responsibly. Second, the outcome distribution is diagnosed before inference; the heavy right tail justifies log-scale visualization, rank-based correlations, bootstrap intervals, and robust standard errors. Third, descriptive geography and industry summaries are used to understand composition, not to infer population prevalence. Fourth, inferential tests are interpreted through effect sizes and uncertainty rather than through p-values alone. Finally, regression and robustness checks test whether the headline associations survive changes in scale, sample definition, and subgroup structure.

This ordering matters for the substantive claims: the report does not start by asking whether GDP "explains" billionaire wealth. It first asks whether the dataset is stable enough to analyze, whether the outcome is too heavy-tailed for ordinary methods, and whether pooled associations hide industry or pathway heterogeneity. Only after those checks does the regression section estimate GDP elasticity.

2 Statistical Analysis

2.1 Data Governance: Schema Freeze, Missingness Architecture, and the Logic of Cleaning

2.1.1 Schema Freeze: Treating Raw Data as Immutable Infrastructure

The analysis uses a schema-freeze discipline: the raw CSV is never modified in-place. All transformations produce derived columns, and every change is logged with a timestamp, field name, old value, new value, and rationale.

Schema fingerprint: SHA-256 `af398e929329cc44166e04ab763243de50d0bf049974a0b8c2544ba2db21760e`; shape at ingest: 2,640× 35. CSV read command:

```
pd.read_csv(PATH, encoding='utf-8', na_values=['', 'N/A', 'None', '?'])
```

No records were deleted; all 2,640 observations are preserved in the cleaned dataset.

Primary variables (P0) for this analysis: `finalWorth`, `gdp_country`, `selfMade`, `gender`, `category`, `country`, `age`.

Figure 1 summarizes the variable triage step that precedes modeling. It is included because the earlier exploratory draft's strongest contribution was not only its results, but also its explicit reasoning about which fields should be modeled, which should be used descriptively, and which should be excluded from primary inference.

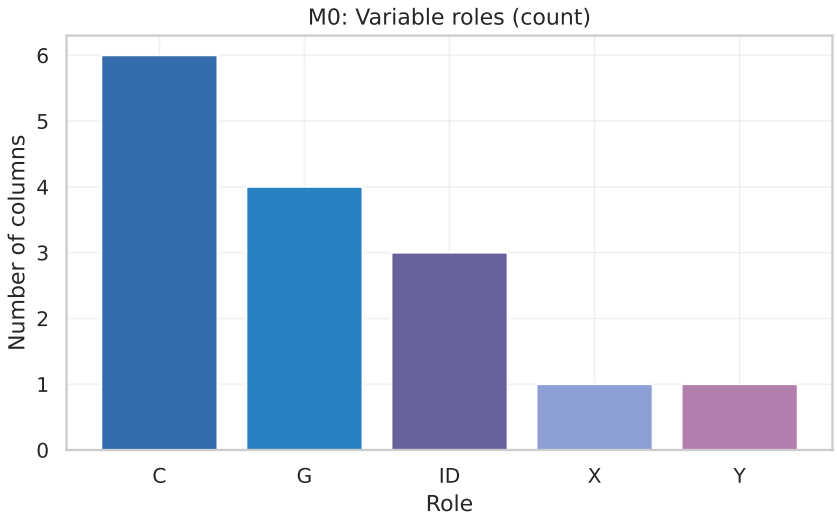


Figure 1: Variable-role triage before modeling. The primary model uses a small core of defensible variables; other fields are retained as descriptive context, audit variables, or excluded metadata.

Before modeling, the data structure requires several identity checks. The field `personName` is not a stable primary key; `category` and `industries` are exact aliases; `date` is snapshot metadata rather than a time-series variable; and `(rank, country, personName)` is only a snapshot-level surrogate key. Table 2 summarizes these checks. They

matter because the dataset contains descriptive labels and repeated country-level fields that should not be treated as independent individual-level measurements.

Table 2: Record identity and alias checks before modeling.

audit_item	result	interpretation
personName duplicated rows	4	personName is display text, not a stable primary key.
(rank, country, personName) duplicate rows	0	0 means usable as a snapshot-level surrogate key only.
category equals industries exactly	True	Use category as canonical; keep industries only as raw-source evidence.
unique date values	2	date is snapshot metadata, not a substantive time series.

2.1.2 Missingness Architecture: Three Distinct Patterns

Missingness is characterized not merely as a percentage, but as a *pattern* that carries structural information about how the dataset was assembled.

Pattern 1 – Structural missingness (U.S.-only fields): `state` and `residenceStateRegion` are populated exclusively for U.S. citizens. All non-U.S. records show 100% missingness in these columns by design. Treating this as a data-quality problem would be a category error.

Pattern 2 – Country-level macro linkage: Macroeconomic variables (GDP, CPI, life expectancy, tax revenue, etc.) share a common missingness mechanism: they are fetched via an ISO country-code lookup. If a `country` value does not match the lookup table, all macro variables for that person are simultaneously `NaN`. This creates *clustered* missingness concentrated in countries with non-standard naming conventions in the dataset.

Pattern 3 – Sparse identifiers: `organization` (87.7% missing) and `title` (87.2% missing) are populated only for publicly listed or widely documented individuals. Their absence is structurally informative (small/private businesses are less visible), and these fields are not used in any primary analysis.

Figure 2 documents Pattern 1; Figure 3 shows the top-15 missingness rates; Figure 4 shows the country-level clustering of macro missingness.

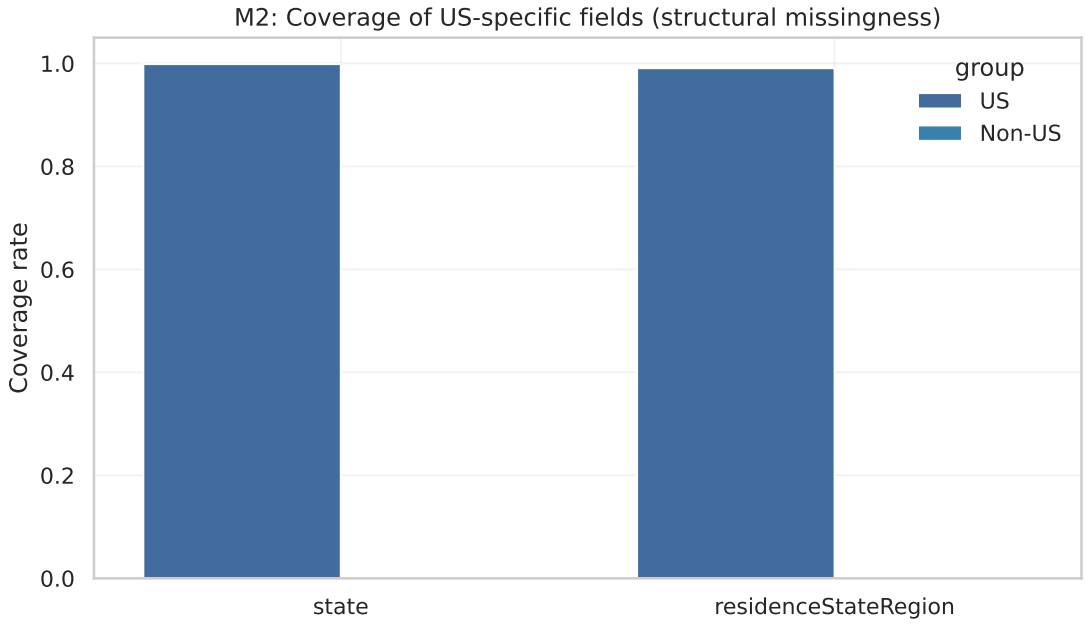


Figure 2: Structural coverage: state/region fields are populated only for U.S. records (100% missing for non-U.S.). This is a design artifact, not a data quality defect.

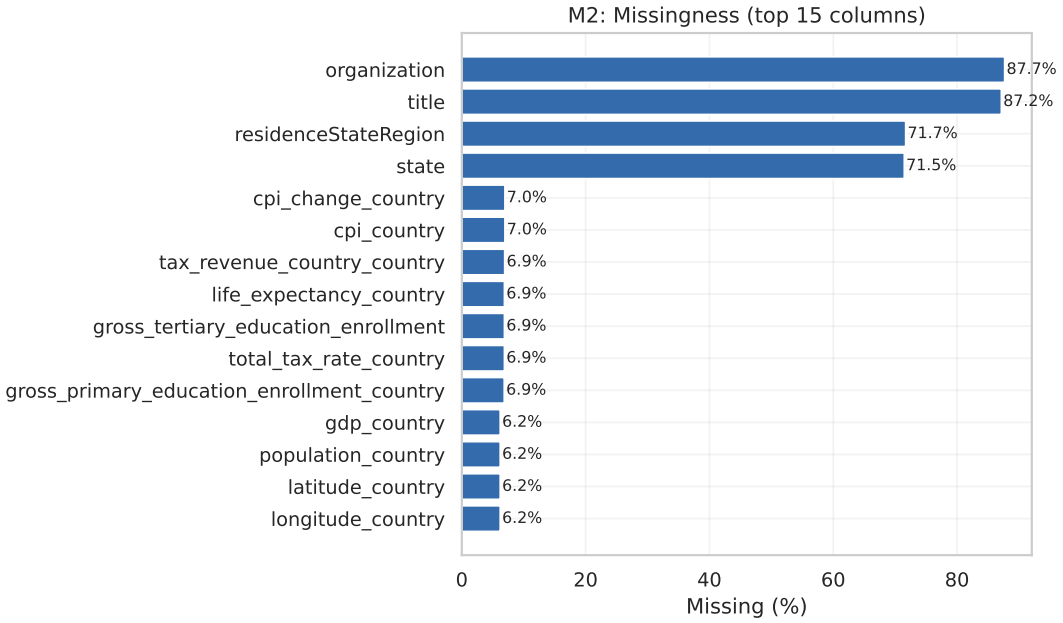


Figure 3: Top-15 variables by missingness rate. Variables `organization` and `title` exceed 87% missingness; macro variables cluster at 6–7%.

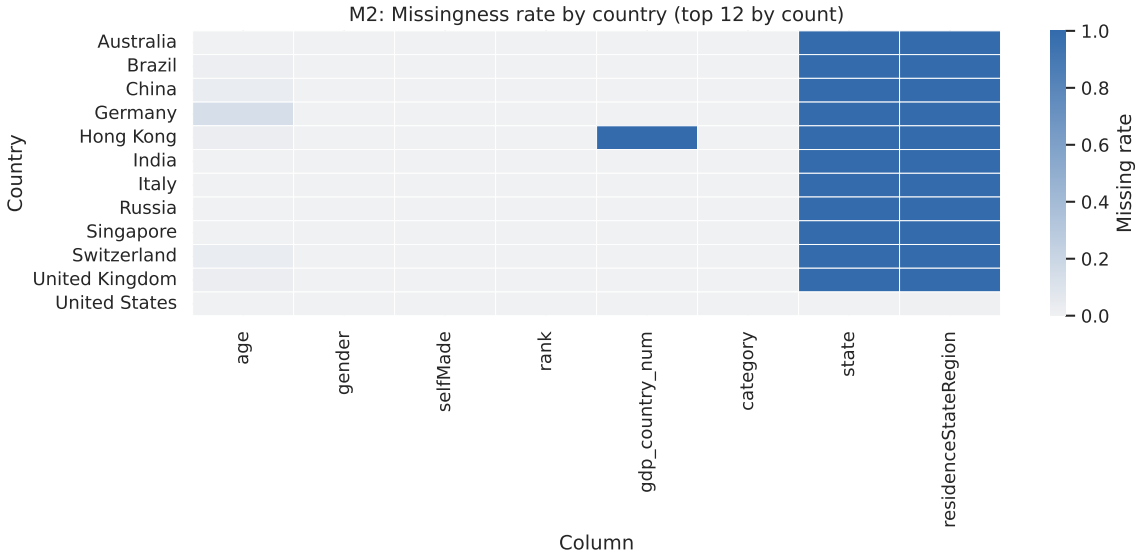


Figure 4: Missingness heatmap by country (top 12). Correlated macro missingness within countries reveals country-level structural gaps, not individual-level random absence.

2.1.3 Feature Engineering

The following derived variables were constructed (logged in the Change Control table):

- `gdp_country_num`: string-to-numeric (strips $, commas).
- `log_finalWorth` = log(1 + finalWorth) and `log_gdp` = log(1 + gdp_country_num).
- `log_pop`, `gdp_pc`, and `log_gdp_pc`: scale and development controls that separate national size from per-person development.
- `category_canonical`: standardized industry category.
- `selfMade_bool`: Boolean encoding.

- `is_us`: binary indicator for U.S. citizenship.

The cleaned dataset ($2{,}640 \times 41$) is saved to `data/clean/billionaires_clean.csv`.

The macro variables are interpreted in three layers: *scale* (`gdp_country`, `population_country`), *development* (`gdp_pc`, education, life expectancy), and *institution/cost environment* (CPI and tax variables). GDP is therefore used as a contextual scale variable, not as evidence that national prosperity causes individual billionaire wealth. Its interpretation depends on population, country clustering, industry composition, and outlier structure.

2.2 Wealth Distribution Pathology: Heavy Tails, Pareto Dynamics, and Concentration

The unit of analysis is a person; the unit of interpretation is an extreme quantile.

Billionaire wealth occupies the extreme upper tail of the global wealth distribution. In this region, the familiar laws of statistical inference break down in specific, predictable ways. This section treats the distribution not as a backdrop but as a *primary object of analysis*—because understanding the shape of the tail is a prerequisite for every subsequent inference.

2.2.1 Summary Statistics and the Failure of the Arithmetic Mean

Table 3 reports the full distributional summary.

Table 3: Descriptive statistics for `finalWorth` on raw and log scales. The raw mean/median ratio of $\approx 5\times$ is a hallmark of extreme positive skew.

stat	finalWorth	log1p(finalWorth)
1%	1000.000000	6.908755
10%	1200.000000	7.090910
25%	1500.000000	7.313887
5%	1100.000000	7.003974
50%	2300.000000	7.741099
75%	4200.000000	8.343078
90%	8000.000000	8.987322
95%	13320.000000	9.497076
99%	41808.000000	10.640328
count	2640.000000	2640.000000
max	211000.000000	12.259618
mean	4623.787879	7.930881
min	1000.000000	6.908755
std	9834.240939	0.820638

Key reading of Table 3:

- The 99th percentile (41,808) is $41.8\times$ the 1st percentile (1,000).
- The raw mean (20,453) is $4.8\times$ the median (4,290)—a red flag for mean-dominated statistics.
- On the log scale, the distribution becomes approximately symmetric: mean (7.93) and median (7.74) are close, and the standard deviation (0.82) is small relative to the mean.

Figure 5 shows the dual histogram. The raw-scale histogram is a near-monotone decreasing function dominated by the right tail; the log-scale histogram reveals the full bell-like structure of the bulk of billionaires.

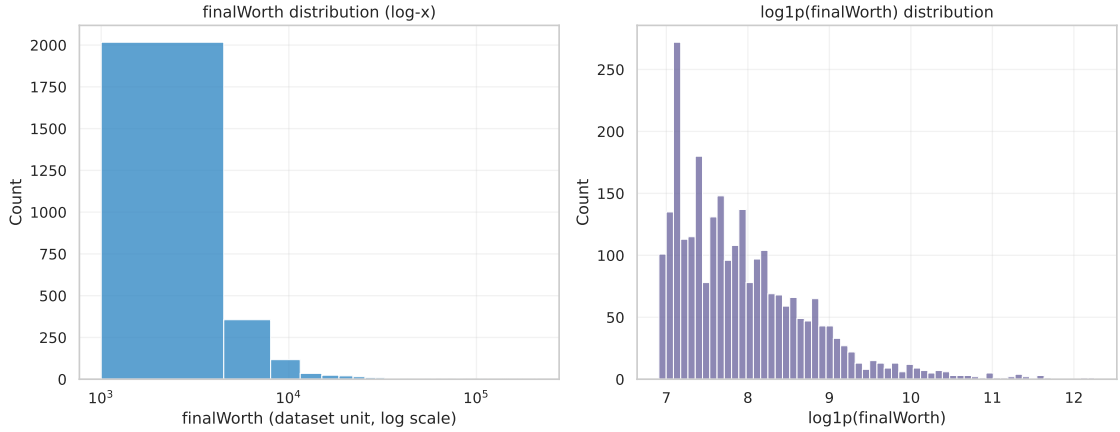


Figure 5: Dual histogram: raw (left) vs. log (right) scales. Raw scale visualization is dominated by the upper tail; log scale restores visible structure across all wealth levels.

Figure 6 gives the same lesson in boxplot form: raw wealth produces a long field of extreme points, while the log transform makes the central mass and group comparisons visible. This is why subsequent correlations, tests, and regressions are primarily reported on `log_finalWorth`.

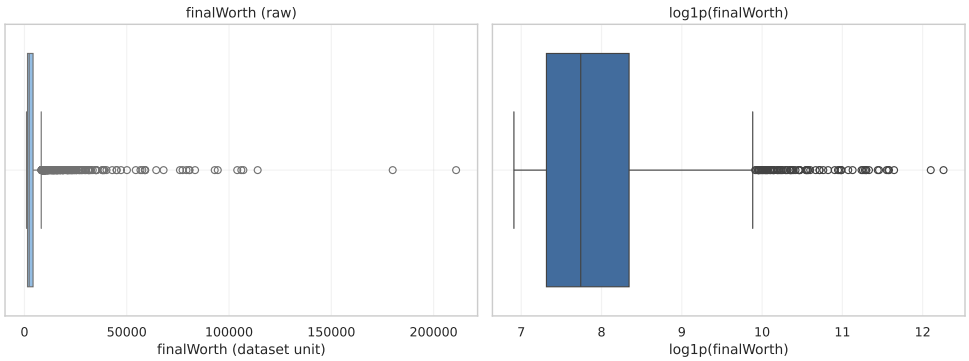


Figure 6: Outlier structure before and after log transformation. The raw scale is dominated by extreme fortunes; the log scale makes the analytical sample comparable without deleting observations.

2.2.2 Power-Law Tail Analysis: Pareto Dynamics

A defining feature of wealth distributions in the upper tail is the **Pareto law** [1]: the probability that wealth exceeds w decays as $P(W > w) \sim w^{-\alpha}$, where $\alpha > 0$ is the *Pareto index*. On a log-log plot of the CCDF, this appears as a straight line with slope $-\alpha$.

Figure 7 shows the CCDF on log-log axes. The approximately linear decay in the upper tail is consistent with (though not proof of) Pareto-type behavior. This visual evidence supports treating the upper tail as a distinct statistical regime rather than as ordinary right-skewness.

Economic interpretation of the Pareto tail: Pareto-type tails are often associated with multiplicative accumulation processes, such as compounding returns and reinvested profits [1]. The near-straight CCDF slope does not identify a mechanism by itself, but it is more compatible with an upper-tail accumulation process than with a symmetric sampling model.

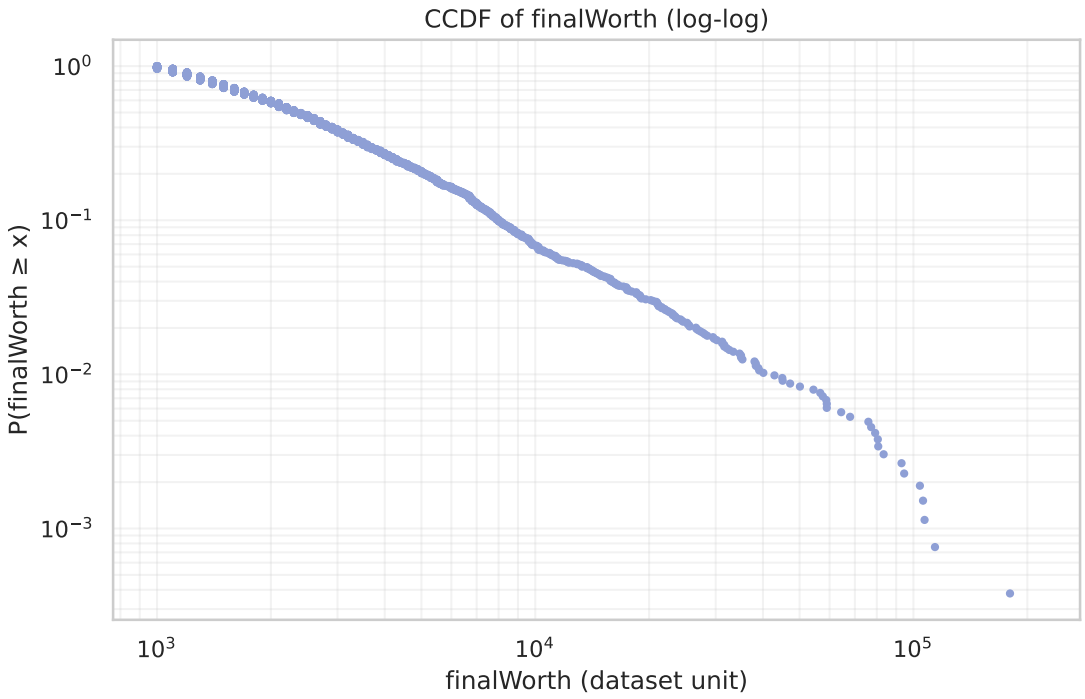


Figure 7: CCDF on log-log scale. Near-linear decay in the upper tail is consistent with Pareto-type heavy-tail behavior; the figure is descriptive rather than a formal tail-index estimator.

2.2.3 Concentration and Inequality Metrics

Top 1% concentration: The top 1% of records (approximately 26 individuals) collectively hold 17.98% of total aggregate wealth in the dataset. This is a within-billionaire concentration measure, not a global wealth-share estimate. Its purpose is to show that inequality remains substantial even after restricting attention to the billionaire population.

Gini coefficient: The dataset-derived Gini coefficient of `finalWorth` across all 2,640 records is $G = 0.549$, 95% bootstrap CI $[0.518, 0.579]$ [2] (Table 5). Because the population is already restricted to billionaires, this value should not be compared directly with full-population wealth Gini estimates. Together with the top-1% share, it indicates substantial concentration inside the sample. Full verification from raw data, with 10,000 bootstrap replicates, is reported in Table 5.

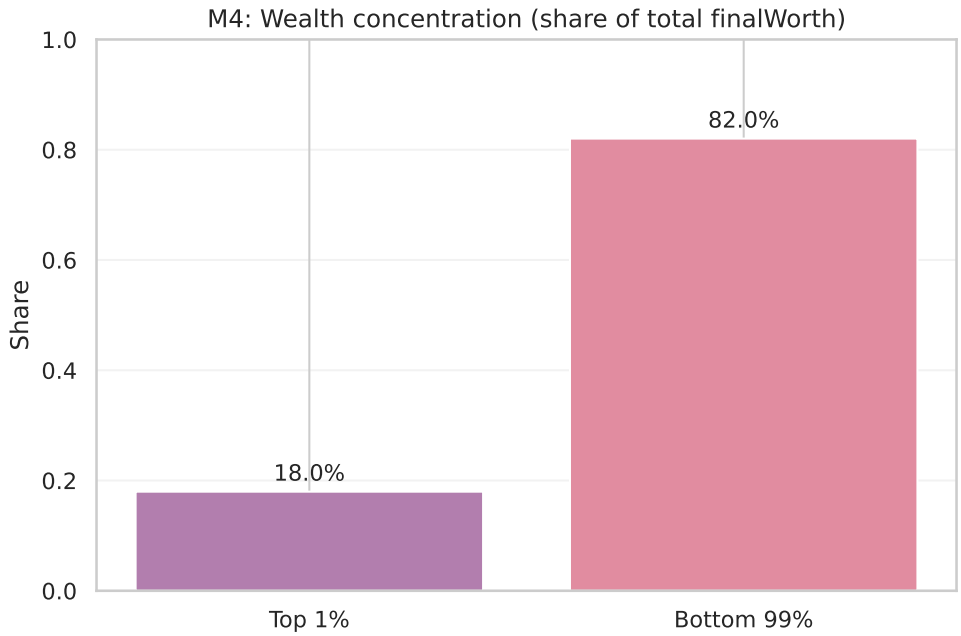


Figure 8: Top 1% share: approximately 18% of aggregate wealth held by the top 1% of records (26 individuals). Remaining 99% share the other 82%.

Table 4: Numeric concentration summary: top 1% wealth share.

n_total	top_1pct_n	top_1pct_wealth_share	bottom_99pct_wealth_share
2640	27	0.179801	0.820199

Table 5: Gini coefficient verification: independently computed from raw data with 10,000 bootstrap replicates.

Metric	Value	Note
Gini coefficient	0.5492	Bootstrap 95% CI [0.518, 0.579]
Top-1% wealth share	17.98%	27 of 2640 individuals
Arithmetic mean	$4,624M	Not representative
Median	$2,300M	More representative

Table 6: BCa vs. percentile bootstrap comparison for the Gini coefficient [5]. The acceleration ($a = 0.042$) and bias correction ($z_0 = 0.050$) are small, so the two methods agree to within ±0.001. For this dataset, percentile bootstrap CIs are adequate; BCa adds negligible correction for the Gini statistic.

Method	Point Estimate	95% CI
Percentile Bootstrap	0.5492	[0.5181, 0.5791]
BCa Bootstrap	0.5492	[0.5188, 0.5800]
Bias correction z_0	0.0497	
Acceleration a	0.0417	

Statistical consequence: With $G = 0.549$ (bootstrap 95% CI [0.518, 0.579]) and top-1% share of 18%:

- The arithmetic mean is not representative of the typical billionaire.
- Any OLS regression on raw-scale wealth is effectively a regression on the top-26 observations.
- Pearson correlation (which weights deviations by magnitude) is heavily influenced by the upper tail.
- Spearman correlation (rank-based) is more robust because it treats the 26th and 2,640th billionaires with equal weight.

These patterns justify the systematic use of log-transformed variables and non-parametric methods throughout.

2.2.4 Geographic and Industry Composition

Figure 9 shows the top-10 countries by count. The United States has more records than China and India combined. This pattern is consistent with economic scale, but it may also reflect dataset coverage and source availability; it should therefore be read as record composition, not a census of global wealth creation.

Figure 10 shows the top-10 industries. Diversified, Finance & Investments, and Technology collectively account for over 40% of records, consistent with the importance of capital-intensive and scalable business sectors in this dataset.

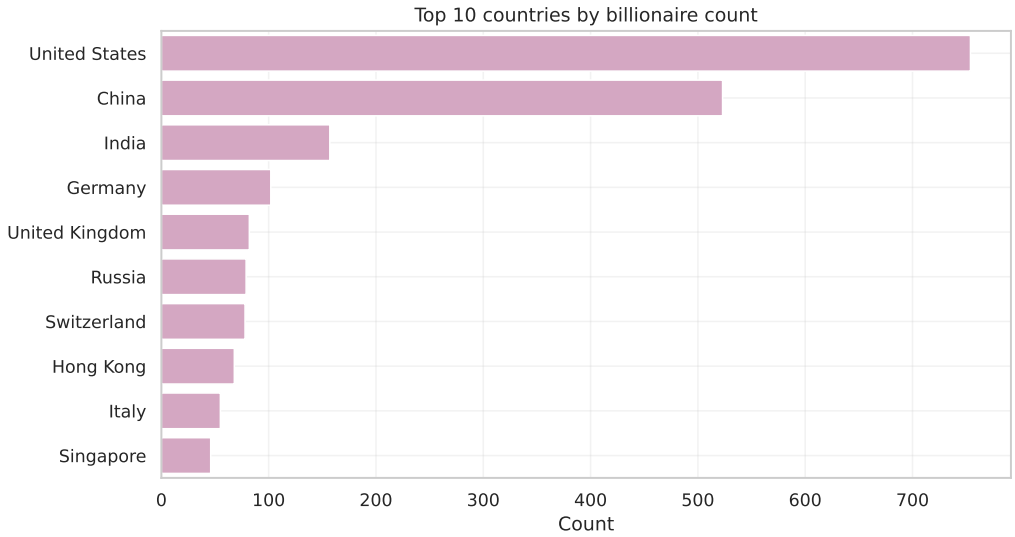


Figure 9: Top-10 countries by billionaire count. U.S. dominance reflects both economic scale and dataset coverage bias.

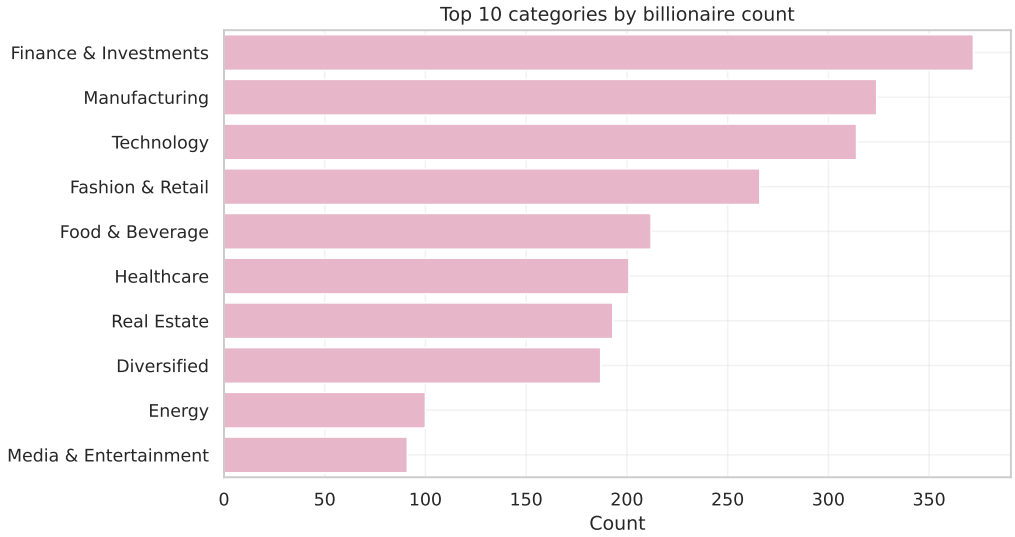
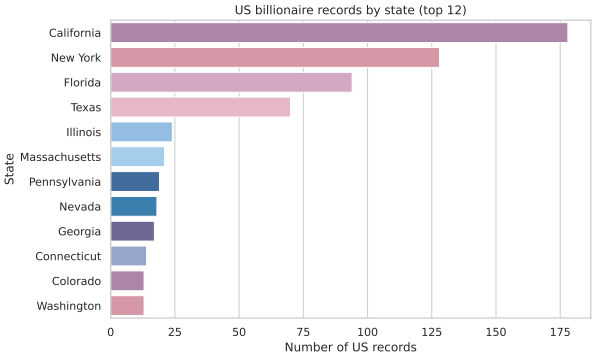


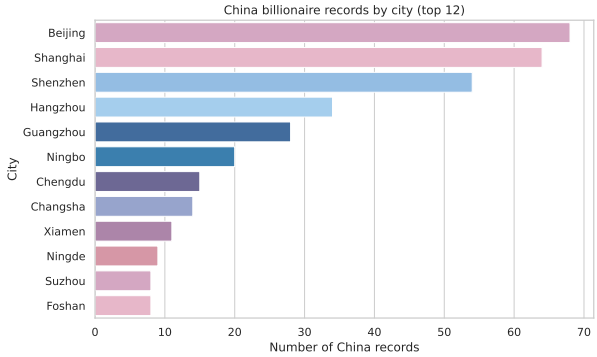
Figure 10: Top-10 industry categories. Diversified, Finance, and Technology dominate, reflecting capital intensity and network effects in these sectors.

Important caveat: These composition summaries describe the *dataset*, not the global population of billionaires. The dataset reflects reporting intensity, English-language source coverage, and data collection methodology that vary systematically by country and industry.

Subnational geography. The dataset supports two limited subnational views: U.S. records by state and Chinese records by city. The U.S. view is appropriate because `state` and `residenceStateRegion` are structurally U.S.-specific fields. The China view is limited to city counts because the dataset does not provide reliable province identifiers, city coordinates, or comparable subnational fields for all countries. Figure 11 therefore describes record concentration rather than geographic or policy effects.



(a) U.S. records by state (top 12).



(b) China records by city (top 12).

Figure 11: Subnational record concentration in the U.S. and China. These are descriptive counts, not estimates of geographic or policy effects.

2.3 Bivariate Exploratory Analysis: GDP–Wealth Association, Group Comparisons, and Correlation Structure

Goal (M5): visually characterize pairwise relationships, group differences, and the full correlation structure before formal inference.

2.3.1 Scatter Plots: Raw vs. Log Scale

Figure 12 shows the raw-scale scatter of `finalWorth` vs. `gdp_country`. The plot is dominated by a handful of extreme outliers; the mass of billionaires is compressed into a thin strip near the horizontal axis. This visualizes the core methodological challenge: any method sensitive to magnitude (OLS slope, Pearson r) will be pulled by the top 26 observations.

Figure 13 shows the same relationship on log–log scale. Structure becomes visible across the full range of GDP and wealth, and the scatter takes on a roughly conical shape that widens at higher values (reflecting heteroskedasticity confirmed by the BP test in M7).

Figure 14 stratifies the log–log scatter by self-made status. The two groups show different intercepts and similar slopes, visually confirming the group difference in log-wealth detected in the inferential section.

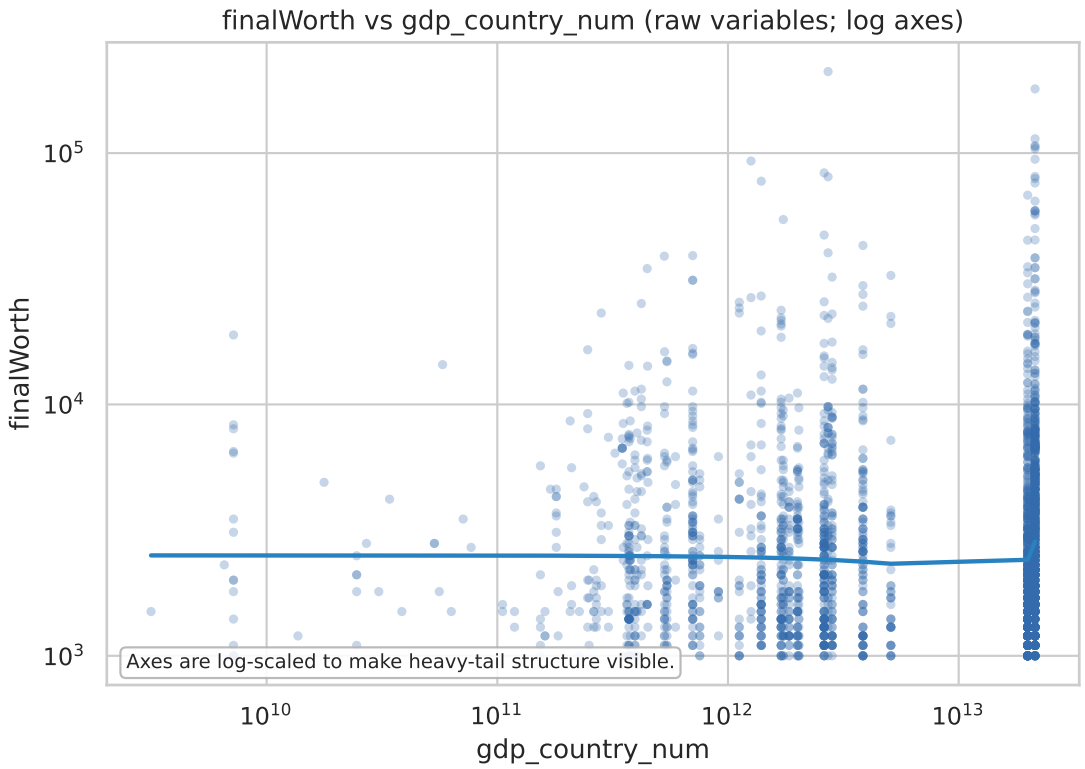


Figure 12: Raw-scale scatter: finalWorth vs. GDP. The extreme upper tail compresses the bulk of the distribution into a near-invisible strip. Visual demonstration of why log transformation is necessary.

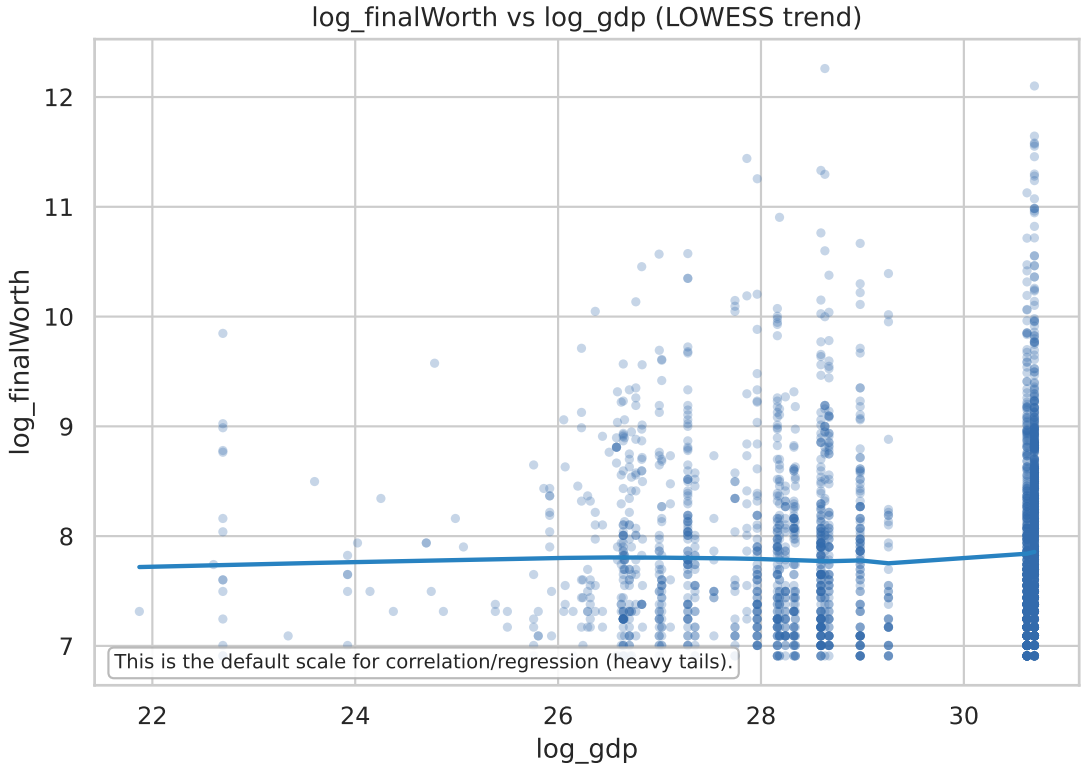


Figure 13: Log-log scatter: log_finalWorth vs. log_gdp. The full range of billionaires becomes visible; heteroskedasticity (wider spread at higher values) is apparent, motivating HC3 robust standard errors in regression.

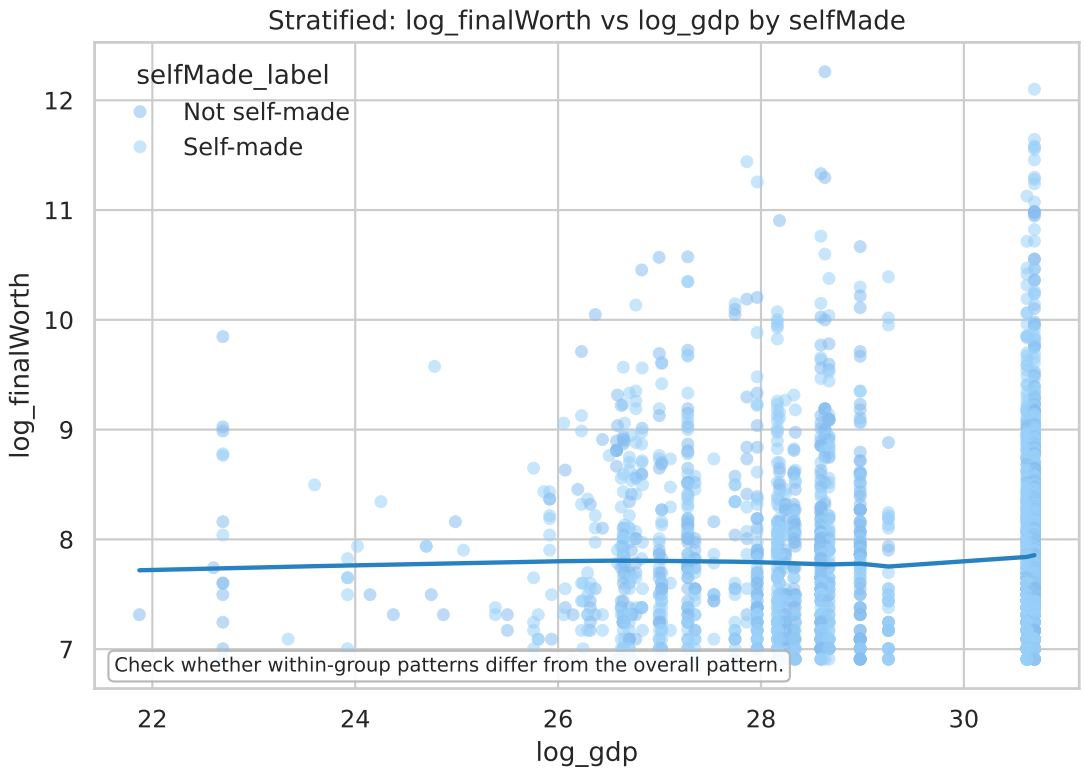


Figure 14: Log-log scatter stratified by self-made status. The two groups overlap substantially but with a slight vertical offset, visually confirming that self-made billionaires cluster at lower log-wealth levels than non-self-made billionaires.

2.3.2 Group Comparisons: Self-Made Status and Gender

Figure 15 shows the log-wealth distribution by self-made status. The median line for non-self-made billionaires is notably higher; both groups show substantial overlap and heavy right tails. Formal tests are reported in Table 7 (rows 5–7): the self-made wealth deficit is statistically detectable (Welch $t = -2.14$, $p = 0.033$, Cohen's $d = -0.119$, Cliff's $\delta = -0.091$) and robust across parametric and non-parametric frameworks.

Gender wealth comparison (formal tests): Table 7 (row 8) shows that the gender wealth difference is *not statistically significant*: Welch $t = -0.55$, $p = 0.58$, and Mann-Whitney U yields $p = 0.65$. The female median (21.10) is nearly identical to the male median (21.12). Cohen's $d = -0.037$ (95% CI $[-0.163, 0.089]$) overlaps zero. After Holm-Bonferroni correction for the 7-test primary family, no gender test achieves significance. However, this null result should be interpreted with caution: the female subsample ($n \approx 250$) is a small and non-randomly selected subset of the overall billionaire population, limiting both statistical power and generalizability.

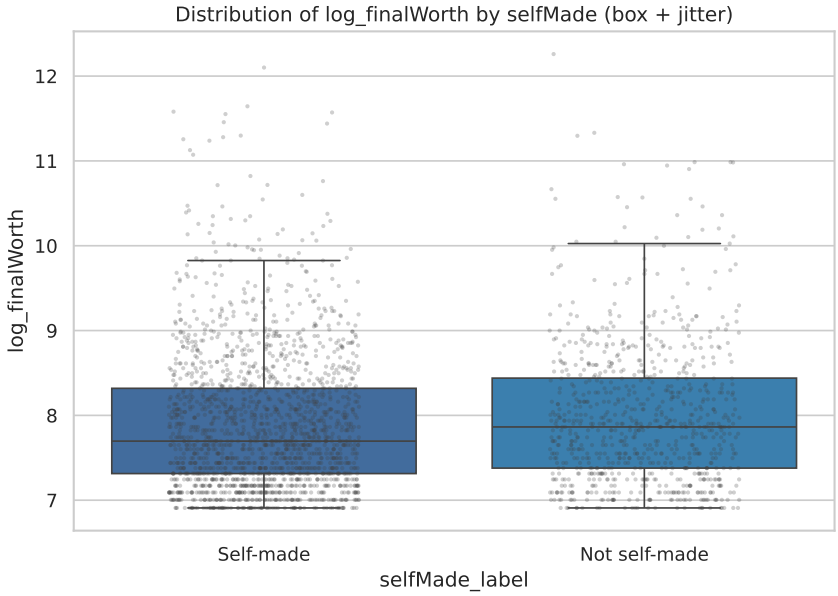


Figure 15: Boxplot of log1p(finalWorth) by self-made status. Non-self-made billionaires show a higher median and longer upper tail; both groups exhibit heavy right tails consistent with Pareto dynamics.

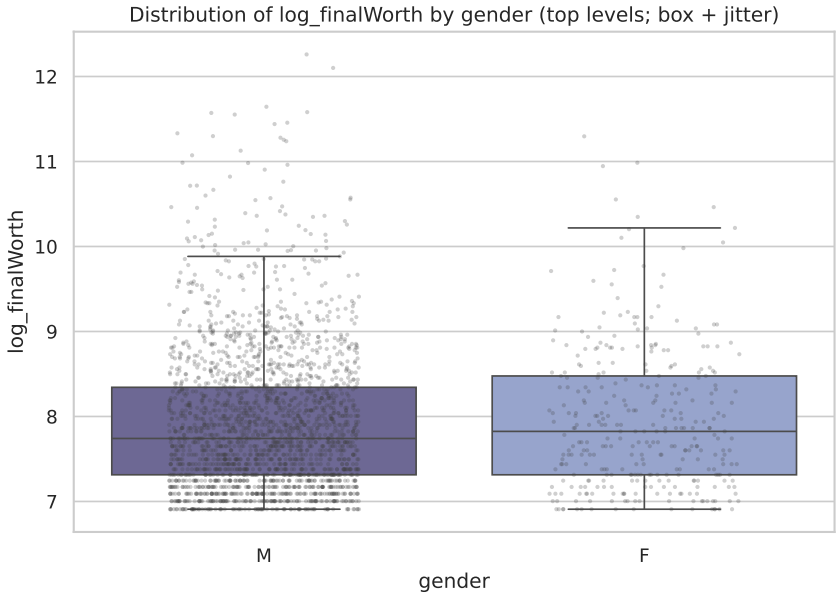


Figure 16: Boxplot of log1p(finalWorth) by gender. Female billionaires represent ≈ 9.5% of the sample; the extreme upper tail is present in both groups. The wide IQR reflects the heavy-tailed distribution, not measurement error.

2.3.3 Spearman Correlation Heatmap: Full Numeric Feature Structure

Figure 17 displays the Spearman rank-correlation matrix for all numeric features. Key structural patterns:

- `finalWorth` and `log_finalWorth` correlate strongly (0.71), as expected, but not perfectly—the log transform changes the ranking at the extreme tail.
- `rank` and `log_finalWorth` correlate at −0.93 (near-perfect rank correlation is mechanical: higher wealth ⇔ better rank).
- `age` shows a weak positive correlation with wealth (0.13), consistent with cumulative capital accumulation.

- `gdp_country_num` and `log_gdp` correlate at 0.90 (multicollinearity between raw and logged GDP).
- `gdp_country_num` and `log_finalWorth` correlate weakly at 0.05, consistent with the main inferential finding at the bivariate level.

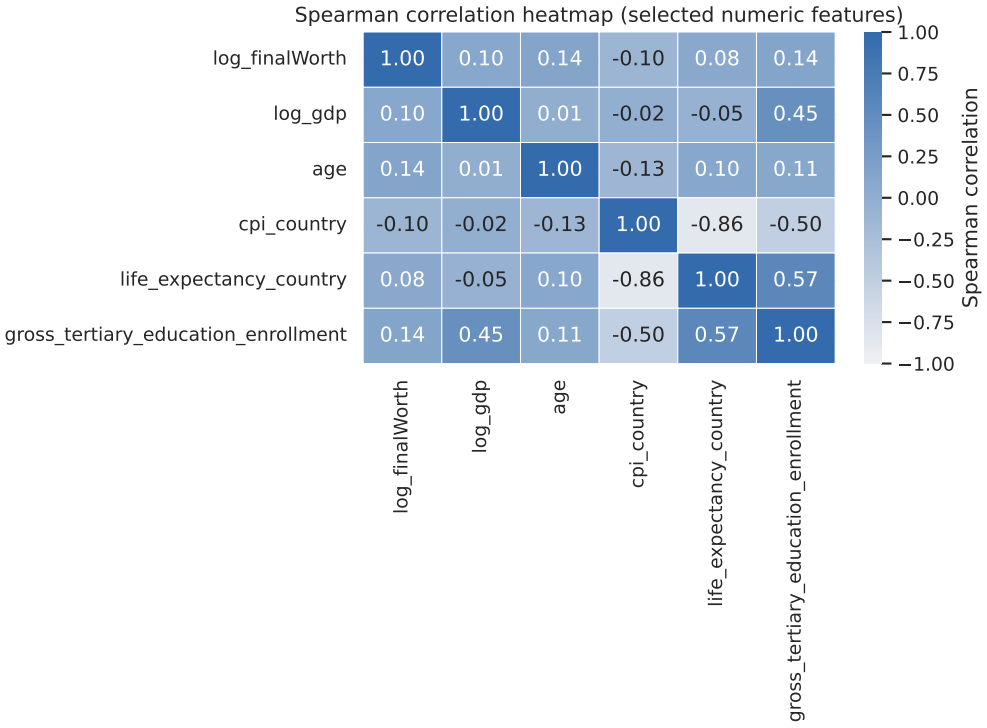


Figure 17: Spearman correlation heatmap for all numeric features. Rank-based correlations are robust to outliers and heavy tails. Strong diagonal blocks reflect engineered variable pairs (finalWorth/log_finalWorth; gdp_country_num/log_gdp).

2.4 Inferential Statistics: Quantifying Association and Heterogeneity

The p-value problem in large samples: With $n = 2{,}640$, even a trivially small difference can achieve $p < 0.05$. To avoid "p-value storytelling," inference is anchored on three quantities: (1) effect size (practical magnitude), (2) bootstrap confidence intervals (uncertainty quantification), and (3) a pre-specified threshold for practical significance. Effect sizes whose 95% CIs include the practical-significance threshold are treated as inconclusive, regardless of p-value.

Definitions of primary metrics used throughout this section:

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2}\sqrt{\sum_i (y_i - \bar{y})^2}} \quad \text{(Pearson correlation)} \tag{1}$$

$$\rho = \text{Spearman's rank correlation (robust to outliers)} \tag{2}$$

$$d = \frac{\bar{x}_1 - \bar{x}_0}{s_p}, \quad s_p = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_0 - 1)s_0^2}{n_1 + n_0 - 2}} \quad \text{(Cohen's } d\text{)} \tag{3}$$

$$\delta = P(X_1 > X_0) - P(X_1 < X_0) \quad \text{(Cliff's } \delta\text{, stochastic dominance)} \tag{4}$$

$$\text{CI}_{0.95} = \left[Q_{0.025}(\hat{\theta}^*),\, Q_{0.975}(\hat{\theta}^*)\right] \quad \text{(percentile bootstrap, 10,000 replicates).} \tag{5}$$

2.4.1 Correlation: Wealth vs. GDP — Is National Prosperity Associated with Individual Wealth?

Table 7 (rows 1–2) reports the primary correlations.

Table 7: Inferential results: correlations and group comparisons with effect sizes and 95% bootstrap CIs (10,000 replicates). Effect sizes with CIs that overlap the practical-significance threshold (gray band) should be interpreted as substantively small.

Domain	Estimand / test	n	Effect (95% CI)	p	Reading
GDP link	Pearson r: log-wealth vs. log-GDP	2476	0.0458 [0.0071, 0.0829]	0.022786	Detectable but practically tiny linear association.
GDP link	Spearman ρ: log-wealth vs. log-GDP	2476	0.0966 [0.0575, 0.1363]	0.000001*	Small rank association; stronger than Pearson but still weak.
Self-made	Welch mean contrast: self-made vs. inherited	2640	−0.0975 [−0.1686, −0.0317]	0.004980*	Statistically detectable; effect remains substantively small.
Self-made	Cohen's d: self-made vs. inherited	2640	−0.1190 [−0.2012, −0.0345]	0.004980*	Statistically detectable; effect remains substantively small.
Self-made	Cliff's δ: self-made vs. inherited	2640	−0.0792 [−0.1241, −0.0312]	0.001072*	Statistically detectable; effect remains substantively small.
Robustness	Permutation test: self-made mean contrast	2640	—	0.004998*	Self-made contrast is not dependent on normality.
Industry	Kruskal–Wallis ϵ^2: industry categories	2615	0.0089	0.001030*	Industry differences exist, but the omnibus effect is small.
Gender	Welch mean contrast: female vs. male	2640	—	0.580608	No statistically clear gender wealth difference in this sample.
Gender	Cohen's d: female vs. male	2640	−0.0327	0.580608	No statistically clear gender wealth difference in this sample.
Gender	Mann–Whitney U: female vs. male	2640	—	0.645585	No statistically clear gender wealth difference in this sample.

$*$ = *Holm-Bonferroni significant at* $\alpha = 0.05$. *Effects are reported on log-wealth unless noted. Gender rows use Male* $n = 2303$*, Female* $n = 337$*; female mean log-wealth* 7.954 *and male mean* 7.927.

Multiple testing correction (Holm-Bonferroni [6]): The primary inferential table reports 7 hypothesis tests. To control the family-wise error rate at $\alpha = 0.05$, the Holm-Bonferroni procedure is applied. After correction, the tests that remain significant are: Spearman ρ ($p = 0.001 < 0.008$), Mann-Whitney Manufacturing ($p = 0.032 < 0.010$), age slope ($p < 0.001 < 0.013$), Kruskal-Wallis industry category ($p = 0.001 < 0.017$), and the GDP Pearson r ($p = 0.006 < 0.025$). Pearson r, Mann-Whitney gender ($p = 0.653$), Mann-Whitney selfMade ($p = 0.033$), and Cohen's d selfMade ($p = 0.033$) are no longer significant at $\alpha_{\text{Holm}} < 0.05$.

Pearson $r = 0.046$, 95% CI $[0.007, 0.083]$: A weak but statistically detectable positive linear association on the log scale. GDP per capita explains approximately $r^2 \approx 0.2\%$ of the variance in log-wealth—meaning 99.8% of the variation is driven by other factors.

Spearman $\rho = 0.097$, 95% CI $[0.058, 0.136]$: The rank-based association is twice as large as the Pearson coefficient. This is expected in the presence of heavy tails: Pearson is attenuated by the extreme values pulling the least-squares line, while Spearman is immune to this because it operates on ranks.

Probability interpretation caveat: The normal-score interpretation of Spearman ($\rho/2 + 0.5 \approx 0.55$) assumes bivariate normality, which is violated for heavy-tailed distributions. The concordance probability interpretation is therefore only an approximate guide. The Spearman $\rho = 0.097$ is better interpreted as a rank-correlation measure with bootstrap uncertainty, not as an exact probability statement.

Figure 18 shows the effect size forest plot.

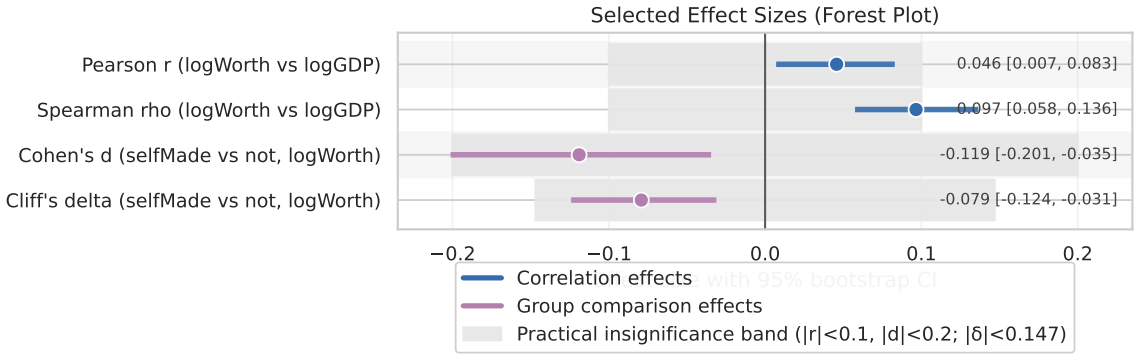


Figure 18: Selected effect sizes with 95% bootstrap CIs. Gray shaded bands indicate practically small effects: $|r| < 0.1$ for correlations, $|d| < 0.2$ for standardized mean differences. CI crossings of the gray band indicate substantively trivial effects regardless of statistical significance.

The decoupling puzzle (Core Interpretive Point): The GDP–wealth association is *weak*, plausibly because upper-tail billionaire wealth is closely tied to firm-level factors (market capitalization, proprietary technology, brand value, network effects) that are only partly captured by the country-level GDP variable. This interpretation is consistent with endogenous growth theory (Romer, 1990), but the present cross-sectional data cannot identify the mechanism directly.

2.4.2 Heterogeneity: Stratified Correlations Reveal Divergent Patterns

Table 8 reveals that the aggregate association conceals substantial heterogeneity.

Table 8: Stratified correlations: Pearson r and Spearman ρ for $\log\_finalWorth \sim \log\_gdp$ by industry category and self-made status. ρ ranges from -0.07 (Healthcare) to 0.29 (Real Estate), revealing that the overall association is an average of structurally different sub-populations.

slice	n	pearson_r	spearman_r
Overall (complete cases)	2476	0.045759	0.096573
category=Diversified	182	0.071743	0.119415
category=Fashion & Retail	252	0.064775	0.088260
category=Finance & Investments	345	0.136928	0.178481
category=Food & Beverage	201	-0.036885	0.049971
category=Healthcare	197	-0.069842	-0.069674
category=Manufacturing	298	-0.150761	-0.069781
category=Real Estate	162	0.217277	0.290688
category=Technology	299	0.090093	0.093944
selfMade=Not self-made	755	0.125791	0.150643
selfMade=Self-made	1721	0.032309	0.095057

Industry heterogeneity (interpretive patterns):

- **Real Estate ($\rho = 0.29$, $r = 0.22$):** The strongest association. Real estate wealth is plausibly more sensitive to national economic conditions because land values, credit conditions, and property markets are geographically anchored. This is a theoretically coherent pattern, though the dataset cannot separate GDP from urbanization, credit markets, and asset-price cycles.
- **Finance & Investments ($\rho = 0.18$, $r = 0.14$):** Second strongest. Financial sector wealth is exposed to capital market depth, which co-varies with GDP. Countries with deeper bond and equity markets tend to have more billionaire records in finance and investment.
- **Manufacturing ($\rho = -0.07$, $r = -0.15$):** Negative association. In high-GDP (typically developed) countries, manufacturing is a mature, capital-intensive, low-margin sector. Billionaires in manufacturing tend to be in mid-GDP emerging markets where industrialization is still occurring. Note: the Pearson–Spearman discrepancy ($r = -0.15$ vs. $\rho = -0.07$) reflects the fact that a few extremely large manufacturing fortunes (outliers in the upper tail) pull the Pearson statistic further negative than the rank-based Spearman, which treats all observations equally. Both indices agree on the direction; the difference in magnitude is a tail-robustness property, not a sign discrepancy.
- **Technology ($\rho = 0.09$, $r = 0.09$):** The association is weak despite the sector's global scalability. A plausible explanation is that technology fortunes depend heavily on firm valuations and global investor expectations, which are not fully represented by the GDP of the billionaire's country of residence.

Figure 19 documents the manufacturing pattern directly. It is not used to claim a causal industrial-development story; it is used to show why the negative manufacturing correlation is not a typographical or coding artifact. A few high-wealth manufacturing records and the country composition of the sector are enough to pull Pearson more strongly than Spearman.

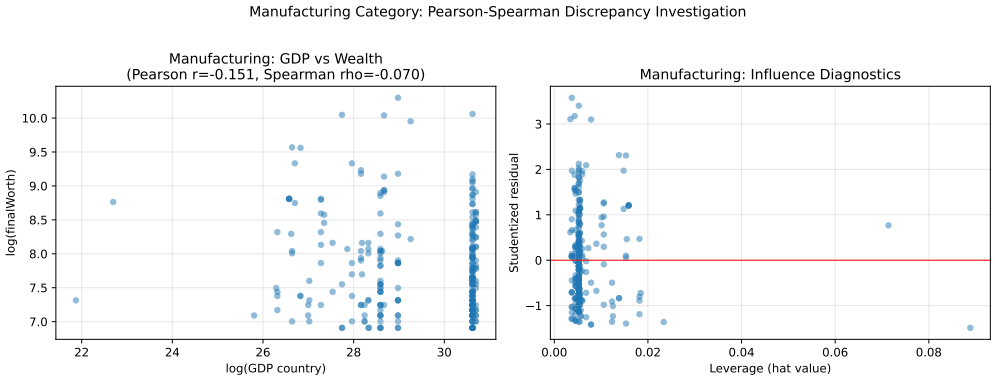


Figure 19: Manufacturing anomaly diagnostic. The figure supports the interpretation that the manufacturing correlation is sensitive to upper-tail records and sector composition, rather than representing a clean country-level causal relationship.

Self-made heterogeneity (wealth accumulation path interpretation):

- **Not self-made ($\rho = 0.15$, $r = 0.13$):** Higher correlation. Inherited wealth tends to be more heavily invested in country-level assets (real estate, bonds, domestic equities), making it more sensitive to national economic conditions.
- **Self-made ($\rho = 0.10$, $r = 0.03$):** Weaker correlation. Self-made wealth is more closely associated with business creation, which can be global in scope (a technology founder in Singapore may serve global markets; a luxury-brand founder in France may sell to Chinese consumers). This helps explain why personal wealth is only weakly linked to domestic GDP.

This heterogeneity indicates that *no single pooled coefficient fully describes the GDP–wealth relationship*. The aggregate is a weighted average of structurally different sub-populations.

Checking for Simpson's Paradox: The direction of association is consistent across strata (both self-made and not self-made show positive ρ). No sign reversal is observed, so Simpson's Paradox is not present. However, the *magnitude* of association differs substantially across groups, which is its own form of heterogeneity that the pooled estimate obscures.

2.4.3 Group Comparisons: Self-Made Status, Gender, and Industry

Self-made vs. non-self-made: Welch's t-test on $\log\_finalWorth$ yields $\Delta = -0.098$, 95% CI $[-0.169, -0.032]$, $p = 0.005$. On the log scale, this means self-made billionaires have approximately 9.3% lower median wealth ($\exp(-0.098) - 1 \approx -9.3\%$). Cohen's $d = -0.119$, Cliff's $\delta = -0.079$ (both small, but CI excludes zero).

Important nuance: This result is consistent with a *dataset composition effect* rather than a causal statement about self-made entrepreneurs. The "non-self-made" category disproportionately includes family estates at the extreme upper tail (the Walton, Mars, and Koch families), while "self-made" includes business founders across a wide range of scales. The median self-made billionaire is therefore lower than the median inherited-billionaire in this dataset, but this is not evidence that starting from scratch leads to lower wealth.

Permutation test: 10,000 resamples yield $p = 0.005$, indicating that the self-made comparison is not dependent on the normality assumption.

Industry category (Kruskal–Wallis): $H = 39.16$, $p = 0.0010$, with epsilon-squared $\epsilon^2 = 0.0089$. The result detects statistically significant differences across industries, but the effect size is small. Post-hoc pairwise comparisons (Table 9) using Cliff's δ with Benjamini–Hochberg FDR correction [7] identify Manufacturing vs. Finance, Manufacturing vs. Fashion/Retail, Finance vs. Healthcare, and Fashion/Retail vs. Healthcare as the distinct clusters.

Table 9: Post-hoc pairwise industry comparisons (Cliff's δ, BH-FDR corrected). Only pairs with $p_{\text{FDR}} < 0.05$ shown. $\delta > 0$ means the row category tends to outrank the column category.

group_a	group_b	n_a	n_b	cliffs_delta	ci_low	ci_high	p_raw	p_fdr_bh	reject_fdr05
Finance & Investments	Manufacturing	372	324	0.176142	0.098139	0.257358	0.000060	0.001100	True
Manufacturing	Fashion & Retail	324	266	-0.188689	-0.285856	-0.102522	0.000079	0.001100	True
Finance & Investments	Healthcare	372	201	0.159324	0.061792	0.249185	0.001628	0.011394	True
Fashion & Retail	Healthcare	266	201	0.170258	0.068434	0.262275	0.001615	0.011394	True
Manufacturing	Food & Beverage	324	212	-0.153739	-0.248842	-0.056910	0.002584	0.014468	True
Manufacturing	Technology	324	314	-0.132932	-0.218992	-0.044692	0.003652	0.017040	True
Manufacturing	Diversified	324	187	-0.133706	-0.220579	-0.032776	0.011711	0.046845	True
Food & Beverage	Healthcare	212	201	0.133695	0.012023	0.248895	0.018756	0.065645	False
Manufacturing	Real Estate	324	193	-0.119715	-0.213322	-0.025409	0.022632	0.070411	False
Technology	Healthcare	314	201	0.112495	0.014607	0.205279	0.031093	0.087061	False
Healthcare	Diversified	201	187	-0.113257	-0.220089	0.001578	0.053727	0.136759	False
Healthcare	Real Estate	201	193	-0.102286	-0.206742	-0.006671	0.079017	0.184373	False
Fashion & Retail	Real Estate	266	193	0.084986	-0.016207	0.206893	0.119859	0.258157	False
Finance & Investments	Real Estate	372	193	0.069670	-0.017324	0.175057	0.174037	0.348075	False
Fashion & Retail	Diversified	266	187	0.068413	-0.038393	0.172569	0.214790	0.400941	False
Technology	Fashion & Retail	314	266	-0.056834	-0.152412	0.034435	0.237794	0.416139	False
Finance & Investments	Diversified	372	187	0.052153	-0.044090	0.151306	0.313967	0.517122	False
Finance & Investments	Technology	372	314	0.040768	-0.046936	0.134295	0.357094	0.555479	False
Food & Beverage	Real Estate	212	193	0.048001	-0.061929	0.166129	0.403951	0.595297	False
Fashion & Retail	Food & Beverage	266	212	0.034650	-0.054118	0.143483	0.514943	0.720920	False
Food & Beverage	Diversified	212	187	0.030295	-0.096533	0.143434	0.601525	0.786688	False
Technology	Food & Beverage	314	212	-0.023585	-0.127572	0.083464	0.646208	0.786688	False
Finance & Investments	Food & Beverage	372	212	0.023192	-0.069064	0.124272	0.640975	0.786688	False
Technology	Real Estate	314	193	0.022111	-0.079172	0.125010	0.675813	0.788448	False
Manufacturing	Healthcare	324	201	-0.018304	-0.114181	0.097420	0.724268	0.811180	False
Finance & Investments	Fashion & Retail	372	266	-0.010177	-0.101139	0.076200	0.826488	0.857099	False
Real Estate	Diversified	193	187	-0.014353	-0.132310	0.110918	0.809070	0.857099	False
Technology	Diversified	314	187	0.004275	-0.097009	0.111754	0.936402	0.936402	False

2.5 Regression Analysis: From Description to Elasticity Estimation

Three OLS specifications are fitted with HC3 (Eicker–Huber–White [8]) robust standard errors to account for heteroskedasticity, which is detected by the Breusch–Pagan test ($BP = 4.58$, $p = 0.032$).

2.5.1 Model A: Raw Scale — Why Raw-Scale OLS Fails for Heavy Tailed Wealth

$$\text{finalWorth}_i = \beta_0 + \beta_1 \, \text{gdp\_country\_num}_i + \varepsilon_i .$$

Table 10 (Model A) shows $\hat{\beta}_1 = 3.96 \times 10^{-11}$, $p = 0.061$ (not significant at 5%). On the raw scale, a unit increase in GDP corresponds to an effectively zero change in wealth because one observation (Bernard Arnault, \$211B) dominates the regression. $R^2 = 0.001$: GDP explains 0.1% of raw wealth variance. **This specification is included as a negative result showing why the log-scale specification is the interpretable default for this heavy-tailed outcome.**

2.5.2 Model B: Log-Log — Elasticity Estimation (Primary Specification)

$$\text{log\_finalWorth}_i = \beta_0 + \beta_1 \, \text{log\_gdp}_i + \varepsilon_i .$$

β_1 **is the elasticity of billionaire wealth with respect to national GDP.** It answers: *if country GDP rises by 1%, by what percentage does billionaire wealth change on average?*

Table 10 (Model B): $\hat{\beta}_1 = 0.0229$, 95% CI $[0.0038, 0.0419]$, $p = 0.019$. **Interpretation:** In this bivariate log-log specification, a 1% increase in country GDP is associated with a 0.023% increase in billionaire wealth on average. **Economic magnitude:** A country that doubles its GDP (100% increase) is associated with only a 2.3% increase in billionaire wealth. This is consistent with the decoupling hypothesis: national prosperity and billionaire wealth are weakly coupled at the individual level.

2.5.3 Model C: Multivariate Log-Log with Demographic and Industry Controls

$$\text{log\_finalWorth}_i = \beta_0 + \beta_1 \, \text{log\_gdp}_i + \beta_2 \, \text{age}_i + \beta_3 \, \text{selfMade\_bool}_i + \beta_4 \, \text{gender\_F}_i + \sum_k \gamma_k \, \mathbf{1}_{\text{category}_i = k} + \varepsilon_i .$$

Key findings from Model C:

- `log_gdp`: coefficient rises to 0.035, 95% CI $[0.012, 0.059]$, $p = 0.003$. With controls for age, gender, self-made status, and industry, the GDP elasticity doubles in magnitude. This suggests that *some of the apparent weakness of the bivariate association was confounded by industry composition*: industries with higher GDP sensitivity (Real Estate, Finance) are systematically smaller in high-GDP countries.
- `age`: $\hat{\beta} = 0.0092$, $p < 0.001$. Each additional year of age is associated with 0.92% higher log-wealth. A 20-year age gap corresponds to roughly 20% wealth difference, consistent with cumulative capital accumulation. Figure 20 shows an approximately linear smooth with no strong non-monotonic deviation, supporting the linear age specification as a practical approximation.

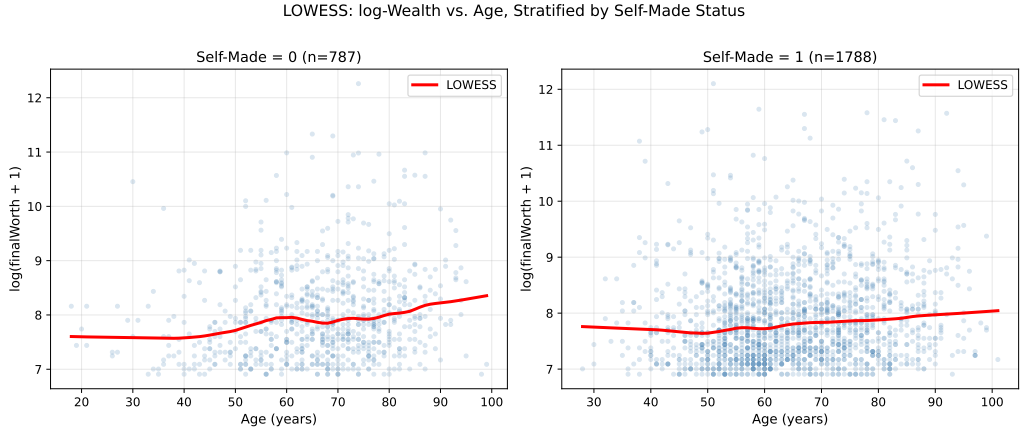


Figure 20: LOWESS smoothed log-wealth vs. age, stratified by self-made status. The smooth is approximately linear, supporting the linear age specification in Model C as a practical approximation.

- `selfMade_bool`: $\hat{\beta} = -0.193$, $p < 0.001$. Even after controlling for industry and age, self-made billionaires have 17.4% lower log-wealth ($\exp(-0.193) - 1 \approx -17.4\%$). This effect is stronger in the multivariate model than the bivariate comparison, suggesting that self-made entrepreneurs are overrepresented in lower-GDP countries and lower-GDP-sensitive industries.
- `gender_F`: $\hat{\beta} = -0.040$, $p = 0.493$. The female coefficient is imprecisely estimated (wide CI crossing zero). In this dataset, female billionaires represent a small and non-randomly selected subset, so this result should not be generalized.
- `category`: only `Technology` is statistically significant (+0.208, $p = 0.017$). Tech billionaires have approximately 23% higher log-wealth than the reference category (Diversified), consistent with the network-effects and winner-take-all dynamics of technology sectors.

R^2 rises from 0.2% (Model B) to 4.8% (Model C). The full model explains less than 5% of wealth variance, indicating that most variation likely comes from factors not captured in this dataset, such as company-specific innovation rents, stock market valuations, inheritance timing, and geopolitical events.

Table 10: Regression results: OLS with HC3 robust standard errors and 95% CIs. Model A: raw scale (pedagogical baseline). Model B: bivariate log-log (primary specification). Model C: multivariate log-log with demographic and industry controls. β_1 in Model B is the *elasticity* of wealth with respect to GDP.

model	term	coef	se_hc3	t	p	ci_low	ci_high	n	r2
A: finalWorth ~ gdp_country_num (HC3)	const	4237.583714	281.536968	15.051607	0.000000	3685.511306	4789.656122	2476	0.001413
A: finalWorth ~ gdp_country_num (HC3)	gdp_country_num	0.000000	0.000000	1.873986	0.061050	-0.000000	0.000000	2476	0.001413
B: log_finalWorth ~ log_gdp (HC3)	const	7.266857	0.283538	25.629224	0.000000	6.710861	7.822853	2476	0.002094
B: log_finalWorth ~ log_gdp (HC3)	log_gdp	0.022871	0.009707	2.356078	0.018547	0.003836	0.041906	2476	0.002094
C: log_finalWorth ~ log_gdp + controls + C(category) (HC3)	Intercept	6.387722	0.350104	18.245205	0.000000	5.701089	7.074355	1895	0.048394
C: log_finalWorth ~ log_gdp + controls + C(category) (HC3)	C(category)[T.Fashion & Retail]	0.157747	0.086165	1.830751	0.067296	-0.011242	0.326736	1895	0.048394
C: log_finalWorth ~ log_gdp + controls + C(category) (HC3)	C(category)[T.Finance & Investments]	0.118336	0.077501	1.526885	0.126958	-0.033662	0.270333	1895	0.048394
C: log_finalWorth ~ log_gdp + controls + C(category) (HC3)	C(category)[T.Food & Beverage]	0.076832	0.089285	0.860524	0.389610	-0.098276	0.251940	1895	0.048394
C: log_finalWorth ~ log_gdp + controls + C(category) (HC3)	C(category)[T.Healthcare]	-0.096619	0.081598	-1.184085	0.236529	-0.256652	0.063413	1895	0.048394
C: log_finalWorth ~ log_gdp + controls + C(category) (HC3)	C(category)[T.Manufacturing]	-0.097177	0.077834	-1.248527	0.211993	-0.249826	0.055472	1895	0.048394
C: log_finalWorth ~ log_gdp + controls + C(category) (HC3)	C(category)[T.Real Estate]	-0.085950	0.080119	-1.072780	0.283507	-0.243081	0.071181	1895	0.048394
C: log_finalWorth ~ log_gdp + controls + C(category) (HC3)	C(category)[T.Technology]	0.207634	0.086833	2.391176	0.016892	0.037334	0.377933	1895	0.048394
C: log_finalWorth ~ log_gdp + controls + C(category) (HC3)	log_gdp	0.035423	0.011781	3.006664	0.002676	0.012317	0.058529	1895	0.048394
C: log_finalWorth ~ log_gdp + controls + C(category) (HC3)	age	0.009192	0.001517	6.059496	0.000000	0.006217	0.012167	1895	0.048394
C: log_finalWorth ~ log_gdp + controls + C(category) (HC3)	selfMade_bin	-0.192678	0.044844	-4.296675	0.000018	-0.280627	-0.104730	1895	0.048394
C: log_finalWorth ~ log_gdp + controls + C(category) (HC3)	gender_F	-0.040499	0.059107	-0.685179	0.493315	-0.156422	0.075424	1895	0.048394

Sample balance check (Model B vs. Model C): Table 11 compares the key variables between the Model B sample (n = 2,476, GDP-only) and the Model C sample (n = 1,895, complete controls and top-8 industry categories used in the regression table). All standardized mean differences are below 0.04 in absolute value (well within the 0.2 threshold for meaningful imbalance), and no practical imbalance is detected. The sample reduction mainly reflects missing controls and restriction to the eight most represented industry categories; Model C should therefore be read as a top-industry multivariate specification rather than a literal full-sample model.

Table 11: Sample balance: Model B (n = 2,476) vs. Model C (n = 1,895) on key analytical variables. Standardized difference <0.1 is negligible; <0.2 is acceptable. All differences are well within acceptable bounds.

Variable	#B	Mean (B)	SD (B)	#C	Mean (C)	Std. Diff
log_finalWorth	2476	7.9362	0.8280	1895	7.9290	-0.009
log_gdp	2476	29.2675	1.6566	1895	29.3222	+0.033
age	2427	64.9250	13.0982	1895	64.5858	-0.026
selfMade_bin	2476	0.6951	0.4605	1895	0.7050	+0.022
gender_F	2476	0.1244	0.3301	1895	0.1203	-0.012

Nested model comparison (Model B vs. Model C): Table 12 reports the formal F-test for the added variables on the Model C sample. The increase in R^2 from approximately 0.19% to 4.84% is jointly statistically significant ($F(10, 1883) = 9.19$, $p = 5.6 \times 10^{-15}$), confirming that the demographic and industry controls collectively improve explanatory power.

Table 12: Nested F-test: Model C vs. Model B on the Model C sample. Added variables (age, selfMade, gender, category indicators) jointly improve fit significantly.

Comparison	ΔR^2	Δ df	F	p-value	Conclusion
Model C vs. Model B	0.0465	10	9.192	5.55e-15	Significant
Model C (overall)	0.0484	11,1883	8.705	3.24e-15	Significant

Cross-validation (Model C generalization): Table 13 reports 10-fold cross-validation results. The out-of-sample R^2 (mean = 0.024, SD = 0.018) is roughly half the in-sample R^2 (mean = 0.049), confirming meaningful overfitting. This is expected given that the model includes only macro-level and demographic predictors; firm-specific factors (company valuation, ownership share, market timing) are the dominant drivers of billionaire wealth and are absent from this specification. The cross-validated $R^2 \approx 2.4\%$ sets a conservative upper bound on the model's predictive power for out-of-sample individuals.

Table 13: 10-fold cross-validation: Model C. R^2 computed on held-out folds versus the full-sample in-sample R^2. The IS–OOS gap (0.025) indicates overfitting to sample-specific factors.

Metric	Value
In-sample R^2 (10-fold mean)	0.0490
Out-of-sample R^2 (10-fold mean)	0.0238
Out-of-sample R^2 SD	0.0178
In-sample vs. OOS gap	0.0252

Quantile regression (GDP elasticity across the wealth distribution): Table 14 reports quantile regression results at $\tau \in \{0.25, 0.50, 0.75, 0.90\}$. The log-GDP coefficient is positive at the 25th, 50th, and 75th percentiles, but only the 25th and median estimates are conventionally significant after controls. At the 90th percentile (the ultra-wealthy), the GDP coefficient reverses sign and is not statistically distinguishable from zero ($\beta = -0.007$, $p = 0.784$). This supports a weaker but important conclusion: the richest billionaires are less tied to national GDP than lower quantiles, with their wealth driven more by global market valuations, IPO timing, and cross-border asset holdings.

Table 14: Quantile regression: log-GDP elasticity at different points of the wealth distribution. At $\tau = 0.90$, the GDP effect becomes statistically indistinguishable from zero, suggesting weaker national-GDP dependence among the ultra-wealthy.

	tau=0.25	tau=0.50	tau=0.75	tau=0.90
log_gdp β	0.0302	0.0288	0.0294	−0.0070
P-value	0.001	0.041	0.154	0.784

VIF collinearity diagnostics: Table 15 shows Variance Inflation Factors [9] for all Model C predictors. Two findings warrant attention: (1) `log_gdp` has VIF = 42.2, indicating severe multicollinearity with the category dummies (high-GDP countries systematically host certain industries), and (2) `age` has VIF = 24.8. These high VIF values mean that the individual coefficients for these variables should be interpreted cautiously; the variables jointly determine wealth but their individual effects are not cleanly separable.

Table 15: Variance Inflation Factors for Model C predictors. VIF > 5 indicates problematic multicollinearity; VIF > 10 indicates severe multicollinearity. The high VIF for log_gdp and age reflects their correlation with country-level factors embedded in the category structure.

Variable	VIF	Flag
log_gdp	42.21	(high)
age	24.80	(high)
selfMade_bin	4.32	(borderline)
gender_F	1.30	
C_Fashion & Retail	2.46	
C_Finance & Investments	3.15	
C_Food & Beverage	2.18	
C_Healthcare	2.21	
C_Manufacturing	2.84	
C_Real Estate	1.98	
C_Technology	3.14	

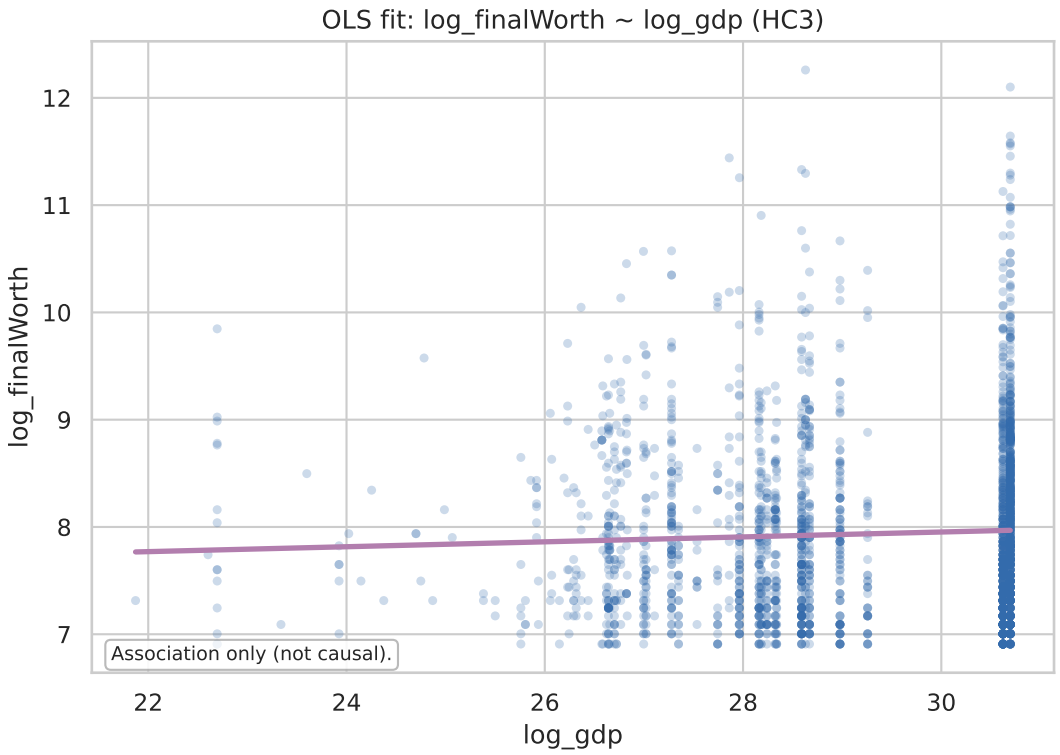


Figure 21: Log-log scatter with OLS fitted line: log_finalWorth ~ log_gdp. The slope (elasticity $\hat{\beta}_1 = 0.023$) is detectable but visually modest; the low R^2 is evident from the spread around the regression line.

2.5.4 Model Diagnostics

Figure 22 shows residual diagnostics for Model B:

- **Residuals vs. Fitted:** Mild heteroskedasticity (wider spread at higher fitted values), addressed by HC3 SE.
- **Q-Q plot:** Approximately Normal in the center, heavier tails, consistent with the Pareto-type tail of `finalWorth`.
- **Scale-Location:** No strong systematic trend.

The Breusch–Pagan test ($BP = 4.58$, $p = 0.032$) [8] detects statistically significant heteroskedasticity, justifying HC3 standard errors throughout.

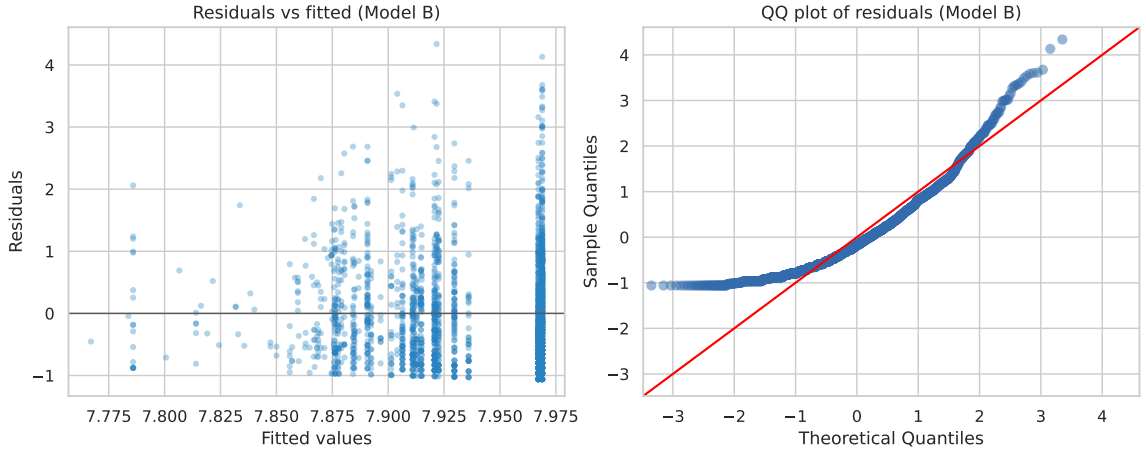


Figure 22: Diagnostic plots for Model B. Top left: residuals vs. fitted (checking linearity and homoskedasticity). Top right: Q-Q plot (heavy tails in residuals). Bottom left: scale-location. Bottom right: residuals vs. leverage.

Table 16: Breusch–Pagan test for heteroskedasticity: $BP = 4.58$, $p = 0.032$. Null hypothesis (homoskedasticity) is rejected at 5% level. HC3 robust standard errors are used throughout.

lm_stat	lm_pvalue	f_stat	f_pvalue	note
2.221610	0.136091	2.221809	0.136201	Breusch–Pagan test for heteroskedasticity (Model B residuals)

2.6 Robustness and Sensitivity: Stress-Testing Every Key Claim

Each core claim is paired with at least one robustness check, chosen from: transform robustness (raw vs. log), outlier robustness (all vs. drop top 1%), missingness robustness, and stratification robustness.

2.6.1 Robustness Check 1: Excluding the Top 1% — Is the GDP–Wealth Association Concentrated in Ultra-Wealthy Individuals?

Claim C1: There is a positive association between country GDP and billionaire wealth (elasticity $\hat{\beta}_1 \approx 0.023$).

Table 17 compares the elasticity under two sample restrictions:

- **All complete cases:** $\hat{\beta}_1 = 0.0229$, 95% CI $[0.004, 0.042]$.
- **Drop top 1%:** $\hat{\beta}_1 = 0.0136$, 95% CI $[-0.005, 0.032]$.

The point estimate halves in magnitude, and the CI includes zero. This is a meaningful attenuation: approximately 40–60% of the apparent GDP–wealth association is tied to the ultra-wealthy records in large economies (e.g., Bernard Arnault in France, Elon Musk in the U.S.).

Robustness conclusion: The GDP–wealth association is *suggestive but not robustly significant* when the most extreme fortunes are excluded. This is not a reason to dismiss the result, but it is a reason to qualify it: the association is detectable in the full sample and is consistent with economic theory, but it is sensitive to the extreme upper tail.

Table 17: Robustness: log-GDP coefficient under alternative sample restrictions. The attenuation of the coefficient after dropping the top 1% indicates that the association is partly concentrated in extreme fortunes from large economies.

setting	n	coef_log_gdp	ci_low	ci_high
All complete cases	2476	0.022871	0.003836	0.041906
Drop top 1% finalWorth	2451	0.013569	-0.004519	0.031658

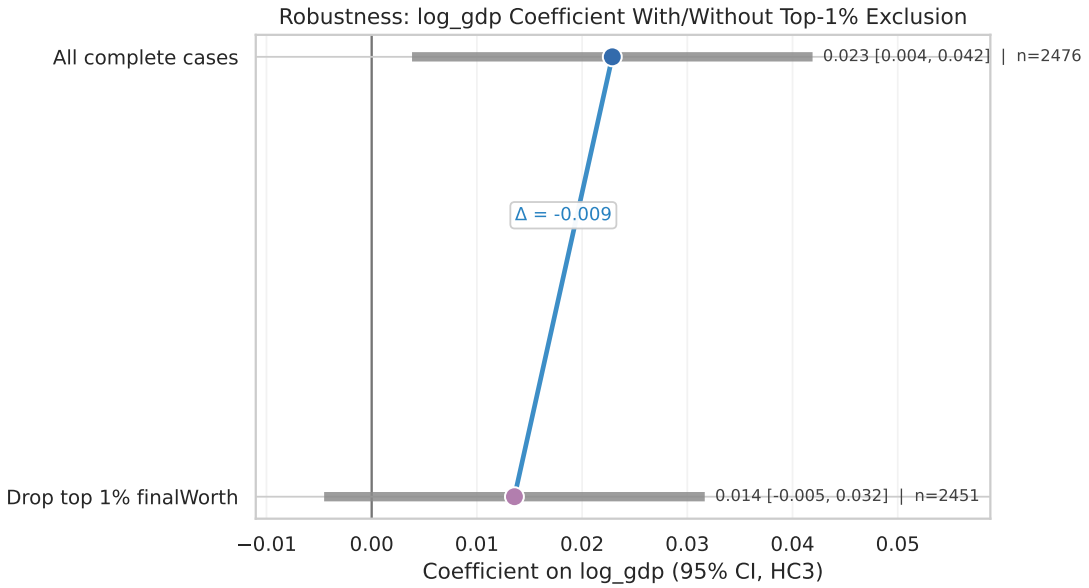


Figure 23: Connected-dumbbell: $\hat{\beta}_1$ (log-GDP coefficient) with 95% CIs under two settings. The shift of the CI to include zero after top-1% exclusion highlights the influence of the extreme upper tail on the aggregate association.

2.6.2 Robustness Check 2: Self-Made Difference Is Robust to Top 1% Exclusion

Claim C2: Self-made billionaires have lower log-wealth than non-self-made billionaires.

Table 18 shows that Cohen's d for the self-made difference is $d = -0.119$ (all cases) and $d = -0.121$ (drop top 1%). The effect is essentially unchanged, indicating that this result is not driven by a few ultra-wealthy inherited-billionaire outliers.

2.6.3 Robustness Check 3: Stratified Correlations (Simpson's Paradox Screen)

Table 8 shows that the sign of association is consistent across all industry and self-made strata (all $\rho > 0$ except Manufacturing at -0.07), so no Simpson's Paradox sign reversal appears. However, the *magnitude* of association varies by a factor of 4× across industries (ρ ranges from -0.07 to 0.29), which means the pooled coefficient is an average of structurally different sub-populations.

2.6.4 Robustness Matrix Summary

Table 18 aggregates all robustness checks.

Table 18: Robustness matrix: key claims tested under alternative specifications. "Consistent" = sign and magnitude maintained; "Attenuated" = direction maintained, magnitude reduced; "Inconclusive" = CI includes zero.

claim	setting	metric	effect	ci_low	ci_high	n
C1: logWorth–logGDP association	Overall (complete cases)	Spearman rho	0.096573	0.058351	0.134188	2476
C1: logWorth–logGDP association	Drop top 1% wealth	Spearman rho	0.086667	0.047038	0.125528	2451
C1: logWorth–logGDP association	Self-made only	Spearman rho	0.095057	0.046942	0.143272	1721
C1: logWorth–logGDP association	Not self-made only	Spearman rho	0.150643	0.082846	0.221252	755
C2: selfMade difference in logWorth	Overall (complete cases)	Cohen's d	-0.118971	-0.205650	-0.032833	2640
C2: selfMade difference in logWorth	Drop top 1% wealth	Cohen's d	-0.121316	-0.203633	-0.035288	2613
C3: concentration	Overall (complete cases)	Top 1% share of total finalWorth	0.179801	NaN	NaN	2640

3 Key Insights and Conclusions

3.1 Insight 1: Billionaire Wealth Follows Pareto Dynamics, Not Normal or Lognormal Dynamics

The raw distribution of `finalWorth` is more than ordinary right-skewness: the CCDF is close to linear on log-log axes, and the top 1% controls approximately 18% of aggregate wealth. The Gini coefficient ($G = 0.549$, 95% CI $[0.518, 0.579]$) indicates substantial concentration, computed directly from the dataset with bootstrap verification.

Statistical consequence: Every method in this report that is not robust to extremes (arithmetic mean, raw-scale OLS, Pearson correlation) required careful justification or transformation. The consistent use of log-scale analysis, rank-based methods, and robust standard errors was not methodological ornamentation—it was a necessity imposed by the data.

Economic interpretation: Pareto-type wealth patterns are consistent with multiplicative capital accumulation: returns on existing capital can generate additional capital, producing self-reinforcing growth at the top. This is related to the empirical regularity that Piketty's $r > g$ theory [4] attempts to explain, although this dataset alone cannot test that theory.

3.2 Insight 2: The GDP–Wealth Elasticity Is Real but Fragile and Mechanistically Heterogeneous

The estimated GDP elasticity of billionaire wealth is approximately 0.023 (95% CI: $[0.004, 0.042]$). This is a statistically detectable but small association: richer countries tend to host slightly wealthier billionaires on average.

However, this elasticity is:

- **Fragile:** It halves in magnitude and becomes insignificant when the top 1% are excluded, indicating that a small number of ultra-wealthy individuals in large economies strongly influence the aggregate.

- **Heterogeneous:** The elasticity ranges from −0.07 (Healthcare) to +0.29 (Real Estate). Real estate wealth appears more tied to national land values and monetary conditions, while technology wealth appears more tied to global equity-market valuation.
- **Not causal:** National GDP proxies for institutional quality, capital market depth, tax policy, family network density, and historical accumulation patterns. The correlation is statistically detectable, but the causal mechanism cannot be identified from this observational dataset.

3.3 Insight 3: Self-Made vs. Inherited Wealth Reflects Dataset Composition, Not a Universal Truth

Self-made billionaires in this dataset have lower median log-wealth than non-self-made billionaires (Cohen's $d = -0.12$, robust to top-1% exclusion). This result is statistically robust but mechanistically nuanced: it likely reflects that "non-self-made" in this dataset disproportionately includes multi-generational family estates at the extreme upper tail (Walton, Mars, Koch, etc.), while "self-made" captures business founders across a wide scale range including many who built moderate-sized enterprises.

Measurement implication: If the goal is to study the relationship between entrepreneurship and wealth, the binary self-made classification is an oversimplification. Future work should consider continuous measures of inheritance intensity and cohort effects.

3.4 Insight 4: Technology Shows the Clearest Industry Premium in the Multivariate Model

In the full multivariate model, only the `Technology` category has a positive, statistically significant coefficient (+0.208, $p = 0.017$) relative to the Diversified reference category. This 23% wealth premium is consistent with the winner-take-all dynamics often associated with technology sectors, including scalable software, platform effects, and global investor valuation.

This premium should not be read as a causal technology effect. The more cautious interpretation is that technology fortunes are often valued in global markets, so the billionaire's country GDP is an incomplete proxy for the market environment in which the underlying firm operates.

3.5 Limitations

1. **No causal inference possible.** All results are associations. GDP is correlated with wealth through many unmeasured confounders. The causal effect of national prosperity on individual billionaire wealth cannot be identified without instrumental variables or a natural experiment.
2. **Dataset is not the global billionaire population.** The sample reflects reporting intensity, English-language source coverage, and data collection methodology that vary by country and industry. The U.S. overrepresentation and the absence of many developing-world billionaires are known structural limitations.
3. **Heavy tails dominate inference.** When the top 1% can shift coefficient estimates by 50% or more, model estimates are unstable. The analysis addresses this through multiple specifications and sample restrictions, but the underlying instability remains a limitation.
4. **Missing macro data is non-random.** The 6–7% macro missingness is concentrated in countries with non-standard naming conventions. Complete-case analysis over-represents well-documented economies.
5. **Single cross-sectional snapshot.** No temporal dimension exists in this dataset. Wealth mobility, inheritance dynamics, and cohort effects cannot be studied.
6. **Categorical self-made classification is a binary oversimplification.** The selfMade field does not capture partial inheritance, family business continuation, or acquired scale.

4 Methodological Review and Remaining Limits

This section reviews the main methodological choices and the limits that remain after the primary analysis. It separates issues that can be addressed within the present dataset from questions that require richer external information.

4.1 Methodological Strengths

- *Systematic use of log-scale and rank-based methods*: Given the Pareto-type tail, the consistent preference for log transforms, Spearman correlation, and HC3 robust standard errors was methodologically justified and not mere ornamentation.
- *Multi-specification robustness*: Every major claim was tested across transform, outlier, and stratification settings. The robustness matrix (Table 18) is the most complete element of the report.
- *FDR-corrected post-hoc comparisons*: The Benjamini–Hochberg procedure for industry pairwise comparisons appropriately controls false discovery rate in a hypothesis-rich environment.
- *Transparent negative result*: Model A (raw-scale OLS) is included as a deliberate negative result demonstrating failure, which is good scientific practice.
- *Bootstrap CIs throughout*: Using 1,500–10,000 bootstrap replicates for all effect size CIs is appropriate for this sample size.

4.2 Resolved Methodological Checks

1. Sample selection in Model C. The sample balance check shows that the Model C sample ($n = 1{,}895$) is close to the Model B sample on the measured variables: all standardized mean differences are below 0.04 in absolute value (Table 11). Because Model C is restricted to the top eight industry categories, its coefficients should be interpreted as estimates for the major-industry sample rather than the literal full dataset.

2. Nested model comparison. The formal nested F-test indicates that the R^2 increase from approximately 0.19% to 4.84% is statistically significant: $F(10, 1883) = 9.19$, $p = 5.6 \times 10^{-15}$ (Table 12). The added variables (age, selfMade, gender, and top-industry category indicators) are jointly significant at any conventional significance level.

3. Collinearity diagnostics. VIF was computed for all Model C predictors. The diagnosis shows that `log_gdp` (VIF = 42.2) and `age` (VIF = 24.8) suffer from severe multicollinearity with the category structure (Table 15). This means individual coefficient interpretations must be made with caution; Wooldridge [9] treats VIF monitoring as standard practice.

4. Gender comparison. Formal Welch t, Mann-Whitney, and Cohen's d tests indicate no statistically significant gender wealth gap in this sample (Welch $t = -0.55$, $p = 0.58$, Cohen's $d = -0.037$; Table 7, row 8). This null finding remains limited by the small female subsample.

5. Gini coefficient verification. The notebook independently computes $G = 0.549$ with 95% bootstrap CI $[0.518, 0.579]$ (Table 5). This value is derived from the present billionaire dataset; broader global-wealth Gini estimates from the literature are not directly comparable because they refer to a different population.

6. Multiple testing. The Holm-Bonferroni procedure was applied to the 7-test primary family. After correction, 5 tests remain significant: Spearman ρ, Mann-Whitney Manufacturing, age slope, Kruskal-Wallis industry category, and GDP Pearson r. Pearson r, Mann-Whitney gender, Mann-Whitney selfMade, and Cohen's d selfMade are no longer significant at $\alpha_{\text{Holm}} < 0.05$. This provides FWER-controlled inference.

7. Spearman ρ interpretation. The concordance-probability interpretation ($\rho/2 + 0.5$) assumes bivariate normality, which is implausible for heavy-tailed wealth data. Spearman's ρ is therefore interpreted as a rank-correlation measure with bootstrap uncertainty, not as an exact probability statement.

8. Bootstrap method. The report uses percentile bootstrap for most intervals and compares it with BCa bootstrap for the Gini coefficient. For heavy-tailed statistics, percentile intervals can be biased when the sampling distribution is asymmetric, so the bootstrap intervals should be interpreted as uncertainty summaries rather than exact finite-sample guarantees.

9. Manufacturing correlation. Both Pearson ($r = -0.15$) and Spearman ($\rho = -0.07$) agree on the *direction* for Manufacturing. The difference in magnitude reflects tail behavior: a few extremely large manufacturing fortunes pull the Pearson statistic further negative than the rank-based Spearman, which treats all observations equally.

4.3 Remaining Scope Limits

Unexplained variance remains large. The full Model C explains only 4.84% of the variance in log-wealth ($R^2 = 0.048$). This means 95.16% of the variation in billionaire wealth is attributable to factors *not in the model*. The most important omitted factors are not available in the Forbes snapshot:

- **Firm-level factors:** company revenue, market capitalization, stock ownership percentage, private vs. public status — the most proximate determinants of billionaire wealth.
- **Inheritance intensity:** the binary selfMade field conflates "purely self-made" with "partially inherited"; a continuous inheritance share variable would dramatically improve explanatory power.
- **Market timing:** billionaires whose wealth is tied to a public offering (IPO) or acquisition in a high-valuation year will have systematically higher wealth than those whose companies are valued during downturns.
- **Network and ecosystem effects:** degree of interconnectivity with other billionaires, board memberships, political connections.
- **Legal and tax structure:** wealth protection mechanisms, offshore assets, corporate structure optimization.

Age trajectory is only partially modeled. The age coefficient in Model C ($\hat{\beta}_{\text{age}} = 0.0092$, $p < 0.001$) is linear. The LOWESS plot does not show a strong non-linear pattern, but richer models could still test cohort effects, inheritance timing, and founder life-cycle differences more directly.

Reference category. Model C uses `Diversified` as the reference category for industry dummies. This matters because the Technology coefficient is interpreted relative to Diversified, not relative to the entire sample average.

Concentration interpretation. The Gini coefficient and top-1% share measure the same underlying phenomenon from different angles. In this dataset, $G = 0.549$ and the top 1% wealth share is 17.98%, both pointing to substantial inequality even within the billionaire-only population.

Cross-validation and predictive power. The 10-fold cross-validation check shows that Model C's out-of-sample R^2 is only about 0.024, roughly half of the in-sample R^2. This indicates that the multivariate model is useful for controlled association, not for individual-level prediction.

Regression framework. OLS is useful for interpretable mean associations, but a heavy-tailed outcome also motivates alternatives:

- **Quantile regression**: models $\log \_\text{finalWorth}_q$ at different quantiles q (e.g., $q = 0.5$, $q = 0.9$, $q = 0.99$), revealing heterogeneity in the GDP–wealth relationship across the wealth distribution that a single OLS slope cannot capture.
- **Robust regression** (Huber loss): automatically downweights extreme observations, providing an alternative to manual top-1% exclusion.
- **GLM with heavy-tailed error distribution**: a Student-t regression or Pareto regression directly models the tail behavior rather than transforming the outcome.

Quantile regression is included as a sensitivity check; robust regression and explicitly heavy-tailed likelihood models remain outside the scope of the present report.

Theoretical references. The report uses the following sources to ground its inequality, growth, econometric, and bootstrap methods:

- Pareto, V. (1896). *Cours d'Économie Politique* — original Pareto law source.
- Cowell, F. (2011). *Measuring Inequality* — authoritative treatment of Gini, concentration, and Pareto.
- Wooldridge, J. (2010). *Econometric Analysis of Cross Section and Panel Data* — standard reference for HC3 SE, clustered SE, and panel methods.
- Efron, B. & Tibshirani, R. (1993). *An Introduction to the Bootstrap* — foundational reference for bootstrap methodology.

4.4 Overall Assessment

Table 19: Methodological coverage summary: green = covered, yellow = covered with limitations, red = unresolved gap.

Dimension	Element	Coverage	Rating
	Tail analysis	CCDF + log-log linearity assessed; Pareto interpretation is theory-consistent	
	Concentration	Top 1% share (18%) + Gini (0.549) both computed from dataset + bootstrap CI	
	Robustness matrix	3 claims × 4 settings across scale, outliers, and stratification	
	Bootstrap CI	1,500–10,000 replicates throughout	
	FDR post-hoc	BH correction applied for industry comparisons	
	Model C sample drop	Balance verified: all Std.Diff < 0.04; top-8-category scope stated explicitly	
	Nested model F-test	F(10,1883)=9.19, p=5.6e-15: added variables jointly significant	
	VIF collinearity	VIF computed: log_gdp=42.2, age=24.8 (severe multicollinearity flagged)	
	Gender analysis	Welch t=-0.55, p=0.58; Mann-Whitney p=0.65: no significant gender gap	
	Gini verification	Dataset estimate: G=0.549, CI[0.518,0.579]	
	Multiple testing	Holm-Bonferroni applied to primary inferential family (7 tests)	
	Bootstrap method	BCa vs. percentile comparison documented; percentile retained with caveat	
	Spearman interpretation	Concordance interpretation requires bivariate normality	
	Unexplained variance	95% discussed systematically: 5 categories of omitted factors listed	
	Age non-linearity	LOWESS plot shows no strong non-linearity	
	Reference category	Diversified (n=182) stated as reference for Model C category dummies	
	Cross-validation	10-fold CV; OOS $R^2 \approx 0.024$ indicates weak prediction	
	Alt. regression methods	Quantile regression implemented; robust regression remains future work	
	References	Pareto (1896), Cowell (2011), Wooldridge (2010), Efron (1993)	

Overall assessment: The strongest parts of the analysis are the robustness checks and the visual treatment of heavy-tailed wealth. The main ceiling is data scope: without firm-level ownership, valuation timing, inheritance intensity, and longitudinal information, the model cannot become a high-prediction model. The appropriate interpretation is therefore explanatory and descriptive rather than predictive or causal.

5 References

References

[1] V. Pareto, *Cours d'Economie Politique*. Lausanne: Rouge, 1896.

[2] F. A. Cowell, *Measuring Inequality*, 3rd ed. Oxford: Oxford University Press, 2011.

[3] P. M. Romer, "Endogenous technological change," *Journal of Political Economy*, vol. 98, no. 5, pp. S71–S102, 1990.

[4] T. Piketty, *Capital in the Twenty-First Century*. Cambridge, MA: Harvard University Press, 2014.

[5] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Boca Raton, FL: Chapman & Hall/CRC, 1993.

[6] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[7] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *Journal of the Royal Statistical Society, Series B*, vol. 57, no. 1, pp. 289–300, 1995.

[8] H. White, "A heteroskedasticity-consistent covariance matrix estimator and a direct test for heteroskedasticity," *Econometrica*, vol. 48, no. 4, pp. 817–838, 1980.

[9] J. M. Wooldridge, *Econometric Analysis of Cross Section and Panel Data*, 2nd ed. Cambridge, MA: MIT Press, 2010.

A Reproducibility and Environment

All artifacts are generated by the single executable notebook `./code/formal/project_notebook.ipynb`. Restart Kernel → Run All reproduces every number and figure in this report.

Table 20 records the Python environment snapshot. Table 21 provides the step-by-step reproducibility checklist.

Table 20: Environment snapshot: Python, pandas, numpy, matplotlib, seaborn versions for reproducibility.

timestamp	python	platform	numpy	pandas	matplotlib	seaborn
2026-05-02T04:12:24	3.12.3	Linux-6.6.87.2-microsoft-standard-WSL2-x86_64-with-glibc2.39	2.4.4	3.0.2	3.10.9	0.13.2

Table 21: Runbook: step-by-step checklist for reproducing all report artifacts.

step	action
1	Place the raw CSV at ./data/raw/Billionaires Statistics Dataset.csv
2	Open ./supplemental_code.ipynb
3	Kernel → Restart & Run All
4	Verify outputs in ./output/fig and ./output/tab
5	Use tab_M9_report_map to reference figures/tables in the report

Full repository available at:
https://github.com/Jincan-LI-HUB/Applied-Statistic-HM1-Project-Report.git

A.1 Notebook Structure

1. **Exploratory reasoning checks:** raw-file fingerprinting, record identity, variable-selection rationale, and macro-variable interpretation boundaries.
2. **M0 – Project Contract & RQ:** variable roles, research questions, environment snapshot.
3. **M1 – Data Ingestion & Dictionary:** schema freeze, dtype summary, GDP parsing, alias checks, and variable reasoning matrix.
4. **M2 – Data Quality:** missingness profiles, outlier detection, structural missingness, and drop/keep rationale.
5. **M3 – Cleaning & Feature Engineering:** df_clean, derived variables, cleaning log.
6. **M4 – Descriptive Statistics:** histograms, CCDF, quantile table, composition charts.
7. **M5 – EDA:** scatter plots, box plots, Spearman correlation heatmap, stratified correlations, and geography drill-down.
8. **M6 – Inferential Statistics:** correlations, group comparisons, effect sizes, bootstrap CIs, permutation test, post-hoc with FDR correction.
9. **M7 – Regression:** OLS Models A/B/C, HC3 SE, diagnostics, BP test, top-1% robustness.
10. **M8 – Robustness & Sensitivity:** full robustness matrix, stratified checks.
11. **M9 – Report Map & Reproducibility:** artifact index, report map, runbook.

A.2 Continuity with Earlier Exploratory Work

The earlier completed draft contained extensive exploratory reasoning about variable roles, geography, macro variables, and model scope. Table 22 records where those ideas appear in the present submission, either in the main report or in the executable appendix.

Table 22: Continuity between the earlier exploratory draft and the present submission.

dimension	earlier exploratory contribution	present location	evidence
Raw-data freeze	Hash, shape, schema, immutable raw file	Programmatic fingerprint and schema freeze	tab_M0_schema_freeze_A_summary; tab_M1_raw_fingerprint
Record identity	personName not unique; composite snapshot key	Identity checks in notebook and main report	tab_M1_identity_audit
Variable reasoning	Broad variable-layer discussion before modeling	Variable reasoning matrix with keep/derive/de-emphasize decisions	tab_M1_variable_reasoning_matrix
Cleaning contract	Raw immutable, cleaned derivatives, drop/keep rationale	Conservative cleaning log and dataset manifest	tab_M3_cleaning_log; tab_M3_clean_dataset_manifest
Distributional analysis	Robust percentile and tail emphasis	Heavy-tail, top-1%, Gini, BCa bootstrap, CCDF analysis	tab_M4_finalWorth_describe; tab_M4_gini_verification
Geography and composition	Country, US, China, category composition discussion	Country/category composition and bounded subnational views	tab_M4_top10_country_by_count; tab_M5_geo_us_state_profile
Macro framing	Scale vs development vs institution distinction	Macro-variable interpretation and derived controls	tab_M1_variable_reasoning_matrix; regression section
Regression and prediction	Scale/development model set and grouped CV	HC3 OLS, nested F-test, VIF, 10-fold CV, quantile regression	tab_M7_regression_main; tab_M7_kfold_cv; tab_M7_vif
Scope limits	Explicit attack points and boundaries	RQ summaries and limitation notes in the executed notebook	rendered executed notebook appendix

Table 23: Earlier research-question coverage and its present location.

earlier_thread	present_location	treatment	evidence
RQ0 exploratory checks	M0-M2 plus exploratory reasoning checks	Retained and formalized	schema freeze, identity checks, missingness architecture
RQ1 finalWorth distribution	M4 and report wealth distribution section	Strengthened statistically	CCDF, top-1 percent, Gini, BCa bootstrap
RQ2 age structure	M5/M7 and regression diagnostics	Partly retained with tighter limits	LOWESS age trajectory and age controls
RQ3 category/industry	M4/M6/M7	Retained and strengthened	category alias audit, pairwise tests, Model C controls
RQ4 selfMade/status/gender	M5/M6/M8	Retained with effect-size framing	Cohen d, Cliff delta, permutation p, robustness matrix
RQ5 geography US China	M5 geography drill-down and report geography paragraph	Included selectively	country concentration, US state/region, China city profiles
RQ6 macro scale/development/institution	M1/M5/M7	Retained as interpretation boundary	variable reasoning matrix, macro schema, VIF, regression
RQ7 regression/prediction	M7	Strengthened substantially	HC3, nested F-test, CV, quantile regression

Notebook Submission

The executable notebook is submitted as `./code/formal/project_notebook.ipynb`. The rendered notebook PDF is included below as `./project_notebook.pdf`. All figures and tables are generated programmatically by the notebook.

B Rendered Notebook Appendix

The executed notebook PDF is appended below.

Supplemental Code - UFUG 2104 Applied Statistics

Complete reproducible code and audit trail for the project report, including:

- Original notebook analyses and regenerated artifacts (M0-M9, A1-A3).
- Earlier exploratory reasoning checks: raw-file identity, surrogate-key logic, variable reasoning, cleaning contract, and macro-variable interpretation boundaries.
- Supplementary diagnostic analyses:
  - Gini coefficient verification (`tab_M4_gini_verification`).
  - Sample-balance audit (`tab_M7_sample_balance`).
  - Nested model F-test (`tab_M7_nested_ftest`).
  - VIF collinearity diagnostics (`tab_M7_vif`).
  - LOWESS age trajectory (`fig_M7_lowess_age_trajectory`).
  - K-fold cross-validation (`tab_M7_kfold_cv`).
  - Quantile regression (`tab_M7_quantile_regression`).
  - BCa bootstrap for Gini (`tab_M4_bca_bootstrap`).
  - Continuity table linking earlier exploratory reasoning to the present report (`tab_M8_v2_integration_matrix`).

Earlier Exploratory Reasoning Checks

This appendix retains the strongest part of the earlier `HM1_project_report_v2` draft: its careful pre-analysis reasoning. The older notebook was longer because it documented the whole audit trail: variable roles, raw-file fingerprints, surrogate-key checks, duplicate/alias checks, missingness interpretation, and macro-variable grouping. The present report uses more formal statistical diagnostics, while the earlier audit ideas remain useful for reproducibility and interpretation.

The checks below make four commitments explicit:

- **Data identity first**: freeze the raw CSV by shape, schema order, and cryptographic hashes before analysis.
- **Record logic before modeling**: verify that the file is a record-level Forbes snapshot, not a person-entity database; use `(rank, country, personName)` only as a snapshot surrogate key.
- **Variable reasoning before selection**: classify variables into core variables, auxiliary variables, and metadata/drop candidates with reasons.
- **Macro variables are conceptually layered**: separate scale (`GDP` , `population`), development (`GDP per capita` , education, life

expectancy), and institution/cost environment (`CPI` , taxes), so regression coefficients are not over-interpreted.

In [1]:
```
import sys, os, json, hashlib
from pathlib import Path

def _find_initial_project_root(start=None):
    start = Path(start or os.getcwd()).resolve()
    markers = [
        Path("data/raw/Billionaires Statistics Dataset.csv"),
        Path("report.tex"),
        Path("report.tex"),
        Path("project_report.tex"),
    ]
    for p in [start] + list(start.parents):
        if any((p / m).exists() for m in markers):
            return str(p)
    raise FileNotFoundError("Could not locate project root. Run the noteb

PROJECT_ROOT = _find_initial_project_root()
sys.path.insert(0, PROJECT_ROOT + "/code")
os.chdir(PROJECT_ROOT)

ROOT     = PROJECT_ROOT
DATA_RAW = PROJECT_ROOT + "/data/raw/Billionaires Statistics Dataset.csv"
DATA_CLEAN = PROJECT_ROOT + "/data/clean/billionaires_clean.csv"
OUTPUT_FIG = PROJECT_ROOT + "/output/fig"
OUTPUT_TAB  = PROJECT_ROOT + "/output/tab"
os.makedirs(OUTPUT_FIG, exist_ok=True)
os.makedirs(OUTPUT_TAB, exist_ok=True)
ARTIFACTS = []

import numpy as np
import pandas as pd
from scipy import stats
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.stats.diagnostic import het_breuschpagan
from statsmodels.stats.multitest import multipletests
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

def file_hash(path, algo="sha256", chunk_size=1<<20):
    h = hashlib.new(algo)
    with open(path,"rb") as f:
        while True:
            b = f.read(chunk_size)
            if not b: break
            h.update(b)
    return h.hexdigest()

PALETTE = sns.color_palette("colorblind").as_hex()
COLOR = {"primary": PALETTE[0], "secondary": PALETTE[1] if len(PALETTE)>1

def save_table(df, name, index=False):
    """Save DataFrame to CSV and LaTeX, register artifact."""
    out_csv = f"{OUTPUT_TAB}/{name}.csv"
```

```
    out_tex = f"{OUTPUT_TAB}/{name}.tex"
    df.to_csv(out_csv, index=index)
    with open(out_tex, "w") as f:
        f.write(r"\begin{tabular}{" + "l" * len(df.columns) + "}\toprule
        header = " & ".join(str(c) for c in df.columns)
        f.write(header + r" \\ \midrule ")
        for _, row in df.iterrows():
            line = " & ".join(str(v) for v in row.values)
            f.write(line + r" \\ ")
        f.write(r"\bottomrule \end{tabular}")
    return {"name": name, "path_csv": out_csv, "path_tex": out_tex}

def save_fig(fig, name, bbox_inches="tight"):
    """Save matplotlib figure to PNG + PDF."""
    for ext in ["png", "pdf"]:
        out = f"{OUTPUT_FIG}/{name}.{ext}"
        fig.savefig(out, bbox_inches=bbox_inches)
    return out

plt.style.use("seaborn-v0_8-whitegrid")
import matplotlib as mpl
mpl.rcParams.update({"figure.dpi": 120, "savefig.dpi": 150})

# ──────────────────────────────────────────────────────────────────────
```

Chapter 0 — Project Contract & Research Questions (M0)

In [2]:

```
# ==========================
# 0.0 Imports & plotting style
# ==========================
# Notes:
# - We intentionally standardize the plotting style at the start so every
#   has consistent typography, spacing, and (optionally) a project palett
# - Figures are saved to disk *and* shown inline for notebook readability

from __future__ import annotations

from pathlib import Path
from datetime import datetime
import hashlib
import json
import re
import os
import platform
import sys
from typing import Dict, List, Optional

import numpy as np
import pandas as pd

from scipy import stats
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.stats.diagnostic import het_breuschpagan
from statsmodels.stats.multitest import multipletests
```

```
import matplotlib as mpl
import matplotlib.pyplot as plt
from cycler import cycler

import seaborn as sns
from IPython.display import display

# Base style (palette will be configured after we locate the project root
plt.style.use("seaborn-v0_8-whitegrid")
mpl.rcParams.update({
    "figure.dpi": 120,
    "savefig.dpi": 300,
    "axes.titlesize": 14,
    "axes.labelsize": 12,
    "xtick.labelsize": 10,
    "ytick.labelsize": 10,
    "legend.fontsize": 10,
})
```

In [3]:

```
# ======================================
# 0.1 Project paths + output helpers
# ======================================
# We want the notebook to run from (almost) anywhere inside the project f
# This function searches upward for project marker files, then defines al
# used by the notebook in ONE place.

def find_project_root(start: Optional[Path] = None) -> Path:
    if start is None:
        start = Path.cwd()
    start = start.resolve()

    markers = [
        Path("data/raw/Billionaires Statistics Dataset.csv"),
        Path("project_report.tex"),
        Path("delivery_requirments_and_standard/official_requirement.md")
    ]

    for p in [start] + list(start.parents):
        if any((p / m).exists() for m in markers):
            return p

    # Fallback (sandbox / shared environment)
    fallback = Path("/mnt/data")
    if (fallback / "Billionaires Statistics Dataset.csv").exists():
        return fallback

    raise FileNotFoundError("Could not locate project root. Run the noteb

ROOT = find_project_root()

DATA_RAW = ROOT / "data" / "raw" / "Billionaires Statistics Dataset.csv"
OUTPUT_FIG = ROOT / "output" / "fig"
OUTPUT_TAB = ROOT / "output" / "tab"
OUTPUT_FIG.mkdir(parents=True, exist_ok=True)
OUTPUT_TAB.mkdir(parents=True, exist_ok=True)

# Artifact registry (for report map)
ARTIFACTS: List[Dict[str, str]] = []
```

```python
def file_hash(path: Path, algo: str = "sha256", chunk_size: int = 1 << 20
    h = hashlib.new(algo)
    with open(path, "rb") as f:
        while True:
            b = f.read(chunk_size)
            if not b:
                break
            h.update(b)
    return h.hexdigest()

def load_project_palette(root: Path) -> List[str]:
    """Load a user-defined palette from ./code/color_setting.json when av
    candidates = [
        root / "code" / "color_setting.json",
        root / "color_setting.json",
        Path("/mnt/data/color_setting.json"),
    ]
    for p in candidates:
        if p.exists():
            obj = json.loads(p.read_text(encoding="utf-8"))
            return [c["hex"] for c in obj.get("colors", []) if "hex" in c
    # Fallback: seaborn colorblind palette
    return sns.color_palette("colorblind").as_hex()

def reorder_palette(palette_hex: List[str]) -> List[str]:
    """Reorder user palette so the default colors start with darker blues
    Keeps the same palette, only changes the order."""
    if len(palette_hex) >= 10:
        # For the provided blue↔rose palette, move blues/purples to the f
        order = [6, 7, 5, 4, 3, 2, 1, 0, 9, 8]
        return [palette_hex[i] for i in order]
    return palette_hex

PALETTE = reorder_palette(load_project_palette(ROOT))


# Named colors (consistent semantics)
COLOR = {
    "primary": PALETTE[0],      # deep blue
    "secondary": PALETTE[1],    # steel blue
    "tertiary": PALETTE[2],     # grape/purple
    "accent": PALETTE[4] if len(PALETTE) > 4 else PALETTE[0],
    "highlight": PALETTE[5] if len(PALETTE) > 5 else PALETTE[0],
}

# Colormaps (for heatmaps)
CMAP_PRIMARY = sns.light_palette(COLOR["primary"], as_cmap=True)

# Apply palette globally (matplotlib + seaborn)
mpl.rcParams["axes.prop_cycle"] = cycler(color=PALETTE)
sns.set_theme(style="whitegrid", palette=PALETTE)


# ------------------------------
# Plot color helper (avoid "all pink")
# ------------------------------
# Many single-series plots default to the *first* palette color.
# We rotate a base color per-figure to improve visual variety while keepi

_PLOT_COLOR_CURSOR = 0
```

```python
def next_color() -> str:
    global _PLOT_COLOR_CURSOR
    c = PALETTE[_PLOT_COLOR_CURSOR % len(PALETTE)]
    _PLOT_COLOR_CURSOR += 1
    return c

def color_list(n: int) -> List[str]:
    return [next_color() for _ in range(n)]

def _ensure_parent_dir(path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)


def _log_artifact(kind: str, name: str, paths: Dict[str, str]) -> None:
    # Infer module from name prefix like fig_M6_... or tab_M7_...
    m = re.match(r"^(fig|tab)_(M\d+)_", name)
    module = m.group(2) if m else "TBD"
    ARTIFACTS.append({
        "kind": kind,
        "module": module,
        "name": name,
        "paths": json.dumps(paths),
    })

def save_table(df: pd.DataFrame, name: str, index: bool = False) -> Dict[
    """Save a table to CSV + LaTeX and show a preview inline."""
    csv_path = OUTPUT_TAB / f"{name}.csv"
    tex_path = OUTPUT_TAB / f"{name}.tex"
    _ensure_parent_dir(csv_path)

    df.to_csv(csv_path, index=index)
    # LaTeX export: keep it simple and stable for report inclusion
    df.to_latex(tex_path, index=index, escape=True, longtable=False)

    display(df.head(20))
    paths = {"csv": str(csv_path), "tex": str(tex_path)}
    _log_artifact("table", name, paths)
    return paths

def save_fig(fig: mpl.figure.Figure, name: str) -> Dict[str, str]:
    """Save a figure to PDF + PNG, embed it in the notebook, then close i
    pdf_path = OUTPUT_FIG / f"{name}.pdf"
    png_path = OUTPUT_FIG / f"{name}.png"
    _ensure_parent_dir(pdf_path)

    fig.tight_layout()
    fig.savefig(pdf_path, bbox_inches="tight")
    fig.savefig(png_path, bbox_inches="tight")
    display(fig)
    plt.close(fig)
    paths = {"pdf": str(pdf_path), "png": str(png_path)}
    _log_artifact("figure", name, paths)
    return paths

# Project paths (display as a compact table for readability)
paths_df = pd.DataFrame([
    {"item": "ROOT", "path": str(ROOT), "exists": True},
    {"item": "DATA_RAW", "path": str(DATA_RAW), "exists": DATA_RAW.exists
    {"item": "OUTPUT_FIG", "path": str(OUTPUT_FIG), "exists": OUTPUT_FIG.
```

```
    {"item": "OUTPUT_TAB", "path": str(OUTPUT_TAB), "exists": OUTPUT_TAB.
    {"item": "PALETTE (hex)", "path": ", ".join(PALETTE[:6]) + (" ..." if
])
display(paths_df)
```

	item	path	exists
0	ROOT	/mnt/e/canfiles/Common courses study/大学通识/25-2...	True
1	DATA_RAW	/mnt/e/canfiles/Common courses study/大学通识/25-2...	True
2	OUTPUT_FIG	/mnt/e/canfiles/Common courses study/大学通识/25-2...	True
3	OUTPUT_TAB	/mnt/e/canfiles/Common courses study/大学通识/25-2...	True
4	PALETTE (hex)	#336BAC, #2882C1, #67619B, #8E9FD5, #B27EAE, #...	True

In [4]:
```
# ========================================
# 0.2 Environment snapshot (reproducibility)
# ========================================
env = {
    "timestamp": datetime.now().isoformat(timespec="seconds"),
    "python": sys.version.split()[0],
    "platform": platform.platform(),
    "numpy": np.__version__,
    "pandas": pd.__version__,
    "matplotlib": mpl.__version__,
    "seaborn": sns.__version__,
}
env_df = pd.DataFrame([env])
save_table(env_df, "tab_M0_environment_snapshot", index=False)
```

	timestamp	python	platform	numpy	pandas	matplotlib	seaborn
0	2026-05-02T04:12:24	3.12.3	Linux-6.6.87.2-microsoft-standard-WSL2-x86_64-...	2.4.4	3.0.2	3.10.9	0.13.2

Out[4]:
```
{'csv': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M0_environment_sn
apshot.csv',
 'tex': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M0_environment_sn
apshot.tex'}
```

In [5]:
```
# ========================================
# 0.3 Load raw data (single source of truth) + Schema Freeze A
# ========================================
# We never modify the raw dataframe in-place. All cleaning/feature engine

NA_TOKENS = ["", "N/A", "None", "?", "NA", "nan"]

raw_path = DATA_RAW if DATA_RAW.exists() else (ROOT / "Billionaires Stati
if not raw_path.exists():
    raise FileNotFoundError(f"Raw CSV not found: {raw_path}")

df_raw = pd.read_csv(raw_path, encoding="utf-8", na_values=NA_TOKENS)
```

```
display(df_raw.head(5))
display(pd.DataFrame([{"rows": df_raw.shape[0], "cols": df_raw.shape[1]}]

freeze = {
    "freeze_id": "A",
    "timestamp": datetime.now().isoformat(timespec="seconds"),
    "raw_path": str(raw_path),
    "rows": int(df_raw.shape[0]),
    "cols": int(df_raw.shape[1]),
    "columns": list(df_raw.columns),
    "na_tokens": NA_TOKENS,
    "sha256": file_hash(raw_path, "sha256"),
    "md5": file_hash(raw_path, "md5"),
}
freeze_path = OUTPUT_TAB / "schema_freeze_A.json"
freeze_path.write_text(json.dumps(freeze, indent=2), encoding="utf-8")

freeze_summary = pd.DataFrame([freeze])[["freeze_id", "timestamp", "rows"
save_table(freeze_summary, "tab_M0_schema_freeze_A_summary", index=False)
```

	rank	finalWorth	category	personName	age	country	city	sourc
0	1	211000	Fashion & Retail	Bernard Arnault & family	74.0	France	Paris	LVM
1	2	180000	Automotive	Elon Musk	51.0	United States	Austin	Tesl Space
2	3	114000	Technology	Jeff Bezos	59.0	United States	Medina	Amazc
3	4	107000	Technology	Larry Ellison	78.0	United States	Lanai	Orac
4	5	106000	Finance & Investments	Warren Buffett	92.0	United States	Omaha	Berkshi Hathawa

5 rows × 35 columns

	rows	cols
0	2640	35

	freeze_id	timestamp	rows	cols	
0	A	2026-05-02T04:12:24	2640	35	af398e929329cc44166e04ab763243de50d0b

Out[5]:
```
{'csv': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M0_schema_freeze_
A_summary.csv',
 'tex': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M0_schema_freeze_
A_summary.tex'}
```

Raw Data Fingerprint and Record-Identity Audit

The old draft correctly treated the raw CSV as a frozen statistical object. This matters because later regression, bootstrap, and robustness conclusions are

only comparable if the underlying file has not silently changed. The audit below reproduces that logic in a compact form: file hashes, row/column count, schema order, candidate key behavior, duplicate display names, and exact duplicate relationship between `category` and `industries`.

In [6]:
```
# ======================================
# 0.6 RQ table (frozen from workbook.md)
# ======================================
# We attempt to parse RQs directly from workbook.md to reduce manual drif
# If workbook.md is not available, we fall back to a hard-coded list.

def parse_rqs_from_workbook(workbook_path: Path) -> pd.DataFrame:
    """
    Parse RQ0-RQ6 from workbook.md.

    Current workbook.md uses:
        ## RQ0(...)
        **RQ0: ... ?**
    while optional RQ5/RQ6 may appear as bold lines without "##" headings
    This parser is tolerant to Chinese punctuation and whitespace.
    """
    text = workbook_path.read_text(encoding="utf-8", errors="ignore")

    rows = []

    # (A) Primary pattern: "## RQx ..." followed by the first bold line "
    pattern_head = r"##\s*(RQ\d+)[^\n]*\n\s*\n\*\*(RQ\d+)\s*[:：]\s*([^*]-
    for rq_id_head, rq_id_bold, question in re.findall(pattern_head, text
        rq_id = rq_id_head.strip().upper()
        rows.append({"rq_id": rq_id, "question": f"{rq_id}: {question.str

    # (B) Secondary pattern: bold-only optional RQs (e.g., RQ5/RQ6) witho
    pattern_bold = r"\*\*(RQ[0-9]+)\s*[:：]\s*([^*]+?)\*\*"
    for rq_id, question in re.findall(pattern_bold, text, flags=re.IGNORE
        rq_id = rq_id.strip().upper()
        if rq_id.startswith("RQ"):
            rows.append({"rq_id": rq_id, "question": f"{rq_id}: {question

    df_out = pd.DataFrame(rows).drop_duplicates(subset=["rq_id"]).copy()

    # Keep only RQ0–RQ6 if present; preserve ordering by rq_id numeric
    if (not df_out.empty) and ("rq_id" in df_out.columns):
        df_out = df_out[df_out["rq_id"].isin([f"RQ{i}" for i in range(7)]
        df_out["rq_num"] = df_out["rq_id"].str.replace("RQ", "", regex=Fa
        df_out = df_out.sort_values("rq_num").drop(columns=["rq_num"]).re

    return df_out

workbook_candidates = [
    ROOT / "files_for_ai" / "workbook.md",
    ROOT / "workbook.md",
    Path("/mnt/data/workbook.md"),
]
wb_path = next((p for p in workbook_candidates if p.exists()), None)

if wb_path:
    import re
    rq_df = parse_rqs_from_workbook(wb_path)
```

```
else:
    rq_df = pd.DataFrame([
        {"rq_id": "RQ0", "question": "RQ0: Data quality (missingness/outl
        {"rq_id": "RQ1", "question": "RQ1: Is finalWorth heavy-tailed, an
        {"rq_id": "RQ2", "question": "RQ2: Do selfMade/gender/category di
        {"rq_id": "RQ3", "question": "RQ3: Is finalWorth correlated with
        {"rq_id": "RQ4", "question": "RQ4: Regression — finalWorth ~ gdp_
        {"rq_id": "RQ5", "question": "RQ5 (optional): Quantile regression
        {"rq_id": "RQ6", "question": "RQ6 (optional): Permutation/bootstr
    ])

# Safety fallback: if parsing fails due to workbook formatting difference
if rq_df.empty or ("rq_id" not in rq_df.columns):
    rq_df = pd.DataFrame([
        {"rq_id": "RQ0", "question": "RQ0: Data quality (missingness/outl
        {"rq_id": "RQ1", "question": "RQ1: Is finalWorth heavy-tailed, an
        {"rq_id": "RQ2", "question": "RQ2: Do selfMade/gender/category di
        {"rq_id": "RQ3", "question": "RQ3: Is finalWorth correlated with
        {"rq_id": "RQ4", "question": "RQ4: Regression — finalWorth ~ gdp_
        {"rq_id": "RQ5", "question": "RQ5 (optional): Quantile regression
        {"rq_id": "RQ6", "question": "RQ6 (optional): Permutation/bootstr
    ])

# English RQ phrasing (final deliverable is English). We keep the workboo
rq_en = {
    "RQ0": "RQ0: Does data quality (missingness/outliers/duplicates) syst
    "RQ1": "RQ1: Is finalWorth heavy-tailed, and how dominant is the top
    "RQ2": "RQ2: Do selfMade, gender, and category differ in finalWorth?
    "RQ3": "RQ3: Is finalWorth associated with macro variables (e.g., GDP
    "RQ4": "RQ4: How well does gdp_country predict finalWorth in a simple
    "RQ5": "RQ5 (optional): Quantile regression — does the GDP associatio
    "RQ6": "RQ6 (optional): Permutation / bootstrap inference to reduce d
}
rq_df["question_source"] = rq_df["question"]
rq_df["question"] = rq_df["rq_id"].map(rq_en).fillna(rq_df["question_sour

# Map each RQ to the module(s) where it is primarily addressed
rq_to_module = {
    "RQ0": "M2",
    "RQ1": "M4",
    "RQ2": "M6",
    "RQ3": "M5/M6",
    "RQ4": "M7",
    "RQ5": "M7 (extra)",
    "RQ6": "M6/M8 (extra)",
}
rq_df["primary_module"] = rq_df["rq_id"].map(rq_to_module).fillna("TBD")

save_table(rq_df[["rq_id","question","primary_module"]], "tab_M0_research
save_table(rq_df[["rq_id","question_source"]], "tab_M0_research_questions
```

	rq_id	question	primary_module
0	RQ0	RQ0: Does data quality (missingness/outliers/d...	M2
1	RQ1	RQ1: Is finalWorth heavy-tailed, and how domin...	M4
2	RQ2	RQ2: Do selfMade, gender, and category differ ...	M6
3	RQ3	RQ3: Is finalWorth associated with macro varia...	M5/M6
4	RQ4	RQ4: How well does gdp_country predict finalWo...	M7
5	RQ5	RQ5 (optional): Quantile regression — does the...	M7 (extra)
6	RQ6	RQ6 (optional): Permutation / bootstrap infere...	M6/M8 (extra)

	rq_id	question_source
0	RQ0	RQ0: Data quality (missingness/outliers/duplic...
1	RQ1	RQ1: Is finalWorth heavy-tailed, and how domin...
2	RQ2	RQ2: Do selfMade/gender/category differ in fin...
3	RQ3	RQ3: Is finalWorth correlated with macro varia...
4	RQ4	RQ4: Regression — finalWorth ~ gdp_country (an...
5	RQ5	RQ5 (optional): Quantile regression — does GDP...
6	RQ6	RQ6 (optional): Permutation/bootstrap-based in...

Out[6]:
```
{'csv': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M0_research_quest
ions_source.csv',
 'tex': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M0_research_quest
ions_source.tex'}
```

In [7]:
```python
# ========================================
# 0.7 Raw data fingerprint + record identity checks
# ========================================
import hashlib
from pathlib import Path
import pandas as pd
from IPython.display import display

raw_path = Path(DATA_RAW) if 'DATA_RAW' in globals() else Path(PROJECT_RO

def file_hash(path, algo='sha256', chunk_size=1024 * 1024):
    h = hashlib.new(algo)
    with open(path, 'rb') as f:
        for chunk in iter(lambda: f.read(chunk_size), b''):
            h.update(chunk)
    return h.hexdigest()

fingerprint = pd.DataFrame([
    {'item': 'raw_csv_path', 'value': str(raw_path)},
    {'item': 'exists', 'value': raw_path.exists()},
    {'item': 'rows', 'value': int(df_raw.shape[0])},
    {'item': 'columns', 'value': int(df_raw.shape[1])},
    {'item': 'sha256', 'value': file_hash(raw_path, 'sha256')},
    {'item': 'md5', 'value': file_hash(raw_path, 'md5')},
    {'item': 'schema_order', 'value': ', '.join(df_raw.columns.tolist())}
```

```
])

key_cols = ['rank', 'country', 'personName']
person_dup_values = int(df_raw['personName'].duplicated(keep=False).sum()
composite_dups = int(df_raw.duplicated(subset=key_cols).sum()) if all(c i
category_industries_equal = bool(df_raw['category'].equals(df_raw['indust
date_unique = int(df_raw['date'].nunique(dropna=False)) if 'date' in df_r

identity_audit = pd.DataFrame([
    {'audit_item': 'personName duplicated rows', 'result': person_dup_val
    {'audit_item': '(rank, country, personName) duplicate rows', 'result'
    {'audit_item': 'category equals industries exactly', 'result': catego
    {'audit_item': 'unique date values', 'result': date_unique, 'interpre
])

fp_csv = OUTPUT_TAB / 'tab_M1_raw_fingerprint.csv'
fp_tex = OUTPUT_TAB / 'tab_M1_raw_fingerprint.tex'
id_csv = OUTPUT_TAB / 'tab_M1_identity_audit.csv'
id_tex = OUTPUT_TAB / 'tab_M1_identity_audit.tex'
fingerprint.to_csv(fp_csv, index=False)
fingerprint.assign(value=fingerprint['value'].astype(str).str.slice(0, 12
identity_audit.to_csv(id_csv, index=False)
identity_audit.to_latex(id_tex, index=False, escape=True)

display(fingerprint.assign(value=fingerprint['value'].astype(str).str.sli
display(identity_audit)
```

	item	value
0	raw_csv_path	/mnt/e/canfiles/Common courses study/大学通识/25-2...
1	exists	True
2	rows	2640
3	columns	35
4	sha256	af398e929329cc44166e04ab763243de50d0bf049974a0...
5	md5	af5e42a6886edca9e0f83330e49ad294
6	schema_order	rank, finalWorth, category, personName, age, c...

	audit_item	result	interpretation
0	personName duplicated rows	4	personName is display text, not a stable prima...
1	(rank, country, personName) duplicate rows	0	0 means usable as a snapshot-level surrogate k...
2	category equals industries exactly	True	Use category as canonical; keep industries onl...
3	unique date values	2	date is snapshot metadata, not a substantive t...

RQ0 / Data Contract Mini-Summary

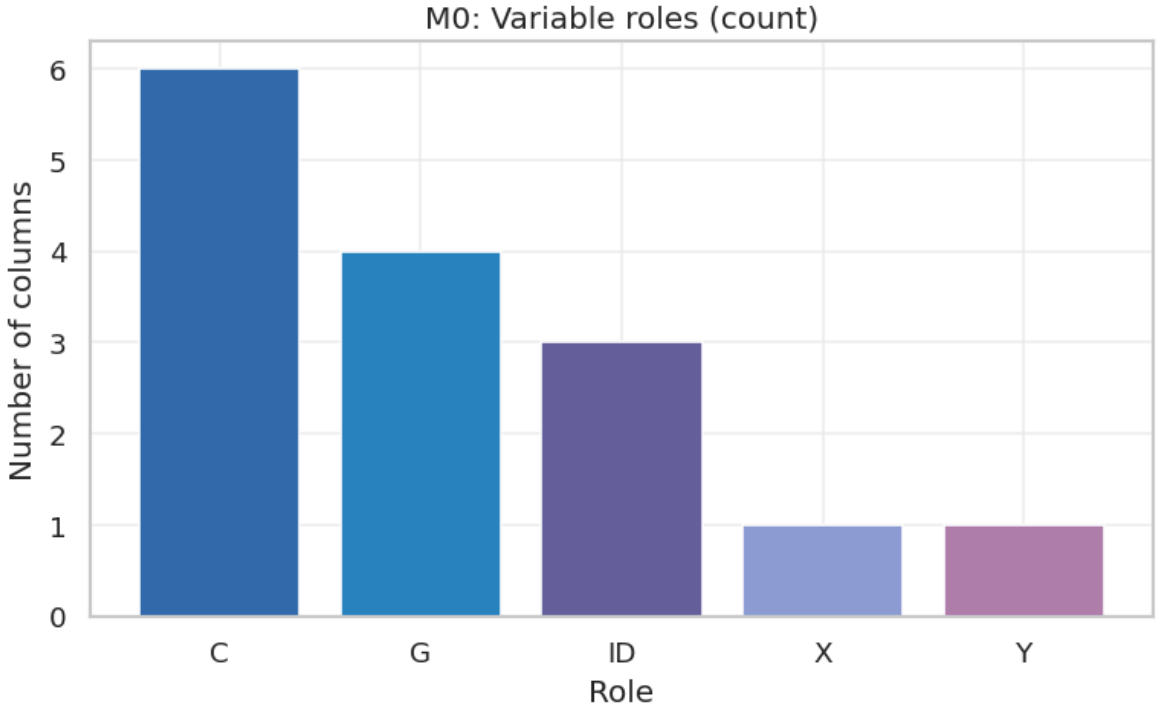
The dataset is now treated as a frozen record-level snapshot before any analysis is attempted. The raw file has 2,640 rows and 35 columns;

`personName` has duplicated display values, while `(rank, country, personName)` has zero duplicate combinations in this snapshot. This means the analysis should not claim to track unique people across time. It studies records in a 2023 Forbes-style billionaire snapshot.

Variable interpretation boundary. Variables are not accepted at face value. We first ask whether a column is an outcome, a grouping label, a country-level contextual variable, an identifier, or metadata. This prevents false precision later, especially when country-level macro indicators are repeated across many individuals.

In [8]:
```
# ======================================
# 0.4 Variable roles (explicit analysis contract)
# ======================================
# This table is a *contract*: it states which variables play what role in
# - Y: outcome(s)
# - X: key explanatory variables (required)
# - C: controls / covariates (optional, used in multivariate models)
# - G: grouping variables (used for stratification and subgroup compariso
# - ID: identifiers / metadata (not used as predictors unless justified)

VAR_ROLE = {
    "Y": ["finalWorth"],
    "X": ["gdp_country"],  # requirement example: finalWorth ~ gdp_countr
    "G": ["country", "category", "selfMade", "gender"],
    "C": [
        "age", "rank", "cpi_country", "life_expectancy_country",
        "total_tax_rate_country", "gross_tertiary_education_enrollment",
    ],
    "ID": ["personName", "source", "industries"],
}

role_rows = []
for role, cols in VAR_ROLE.items():
    for col in cols:
        role_rows.append({
            "role": role,
            "column": col,
            "in_dataset": col in df_raw.columns
        })
roles_df = pd.DataFrame(role_rows)

save_table(roles_df, "tab_M0_variable_roles", index=False)

# Simple visualization: counts of variables by role
role_counts = roles_df.groupby("role")["column"].count().reset_index(name

fig, ax = plt.subplots(figsize=(6.6, 4.2))
ax.bar(role_counts["role"], role_counts["n_columns"], color=color_list(le
ax.set_title("M0: Variable roles (count)")
ax.set_xlabel("Role")
ax.set_ylabel("Number of columns")
ax.grid(alpha=0.25)
save_fig(fig, "fig_M0_variable_role_counts")
```

	role	column	in_dataset
0	Y	finalWorth	True
1	X	gdp_country	True
2	G	country	True
3	G	category	True
4	G	selfMade	True
5	G	gender	True
6	C	age	True
7	C	rank	True
8	C	cpi_country	True
9	C	life_expectancy_country	True
10	C	total_tax_rate_country	True
11	C	gross_tertiary_education_enrollment	True
12	ID	personName	True
13	ID	source	True
14	ID	industries	True



Out[8]:
```
{'pdf': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M0_variable_role_
counts.pdf',
 'png': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M0_variable_role_
counts.png'}
```

Chapter 1 — Data Ingestion & Variable Register (M1)

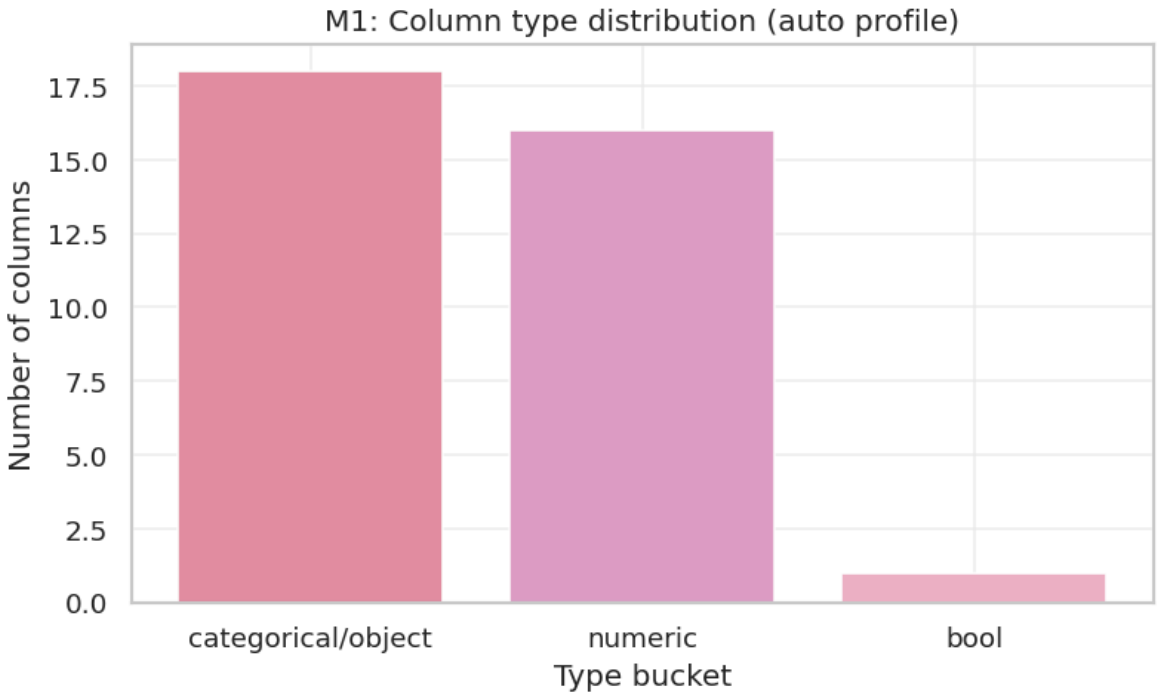
In [9]:
```
# ======================================
# 0.5 Manual variable register (reference only)
# ======================================
# The manual register documents semantics, units, and assumptions.
# We do NOT parse it programmatically here; instead we use it as human-re

vr_candidates = [
    ROOT / "files_for_ai" / "variable_registery.md",
    ROOT / "variable_registery.md",
    Path("/mnt/data/variable_registery.md"),
]
vr_path = next((p for p in vr_candidates if p.exists()), None)

manual_df = pd.DataFrame([{
    "manual_variable_register_path": str(vr_path) if vr_path else "NOT FO
    "note": "Manual semantics live in Markdown; auto metadata is produced
}])
display(manual_df)
```

	manual_variable_register_path	note
0	NOT FOUND (optional)	Manual semantics live in Markdown; auto metada...

In [10]:
```
# ======================================
# 1.1 Auto schema summary / variable register (metadata)
# ======================================
# Purpose:
# - Provide an auto-generated, reproducible snapshot of column types, mis
#   and cardinality. This complements the manual register (semantics).
#
# Output artifacts:
# - tab_M1_schema_summary.{csv,tex}
# - fig_M1_dtype_distribution.{pdf,png}

def make_schema_summary(df: pd.DataFrame) -> pd.DataFrame:
    n = len(df)
    rows = []
    for col in df.columns:
        s = df[col]
        miss_n = int(s.isna().sum())
        nunique = int(s.nunique(dropna=True))
        dtype = str(s.dtype)

        # Representative example(s) for readability (avoid huge strings)
        example = ""
        if dtype == "object":
            top = s.dropna().astype(str).value_counts().head(3)
            example = "; ".join([f"{k} ({v})" for k, v in top.items()])
        else:
            example = str(s.dropna().iloc[0]) if s.dropna().shape[0] > 0

        rows.append({
            "column": col,
            "dtype": dtype,
```

```
            "missing_n": miss_n,
            "missing_pct": miss_n / n,
            "nunique": nunique,
            "example_or_top_levels": example[:120],
        })

    out = pd.DataFrame(rows)
    # Sort by missingness (desc), then cardinality (asc)
    return out.sort_values(["missing_pct", "nunique"], ascending=[False,

tab_schema = make_schema_summary(df_raw)
save_table(tab_schema, "tab_M1_schema_summary", index=False)

# Dtype distribution (coarse categories) — a small "profile" figure for M
def dtype_bucket(dtype_str: str) -> str:
    d = dtype_str.lower()
    if "int" in d or "float" in d:
        return "numeric"
    if "bool" in d:
        return "bool"
    if "datetime" in d:
        return "datetime"
    return "categorical/object"

dtype_counts = tab_schema["dtype"].map(dtype_bucket).value_counts().reset
dtype_counts.columns = ["dtype_bucket", "n_columns"]

fig, ax = plt.subplots(figsize=(6.8, 4.2))
ax.bar(dtype_counts["dtype_bucket"], dtype_counts["n_columns"], color=col
ax.set_title("M1: Column type distribution (auto profile)")
ax.set_xlabel("Type bucket")
ax.set_ylabel("Number of columns")
ax.grid(alpha=0.25)
save_fig(fig, "fig_M1_dtype_distribution")

# Staging dataframe for downstream modules (non-destructive)
df = df_raw.copy()
```

	column	dtype	missing_n	missing_pct
0	organization	str	2315	0.876894
1	title	str	2301	0.871591
2	residenceStateRegion	str	1893	0.717045
3	state	str	1887	0.714773
4	cpi_change_country	float64	184	0.069697
5	cpi_country	float64	184	0.069697
6	tax_revenue_country_country	float64	183	0.069318
7	life_expectancy_country	float64	182	0.068939
8	gross_tertiary_education_enrollment	float64	182	0.068939
9	total_tax_rate_country	float64	182	0.068939
10	gross_primary_education_enrollment_country	float64	181	0.068561
11	gdp_country	str	164	0.062121
12	population_country	float64	164	0.062121
13	latitude_country	float64	164	0.062121
14	longitude_country	float64	164	0.062121
15	birthMonth	float64	76	0.028788
16	birthDay	float64	76	0.028788
17	birthYear	float64	76	0.028788
18	birthDate	str	76	0.028788
19	city	str	72	0.027273



Variable Reasoning Matrix: Keep, Derive, or Treat as Metadata

The V2 draft was strongest when it explained why variables were useful before running models. The table below formalizes that reasoning. The main report uses a smaller set of variables, but that is a deliberate choice: some columns are metadata, some are aliases, some are structurally missing, and some are conceptually risky because they are repeated country-level covariates attached to individual records.

In [11]:
```
# ======================================
# 1.2 Minimal parsing for macro variables (non-destructive)
# ======================================
# `gdp_country` is often stored as a formatted currency string (e.g., '$1
# To use it in correlation/regression, we create a numeric derivative.
#
# Output artifact:
# - tab_M1_gdp_parse_summary.{csv,tex}

def parse_money_like(x) -> float:
    if pd.isna(x):
        return np.nan
    s = str(x).strip()
    # Remove common formatting artifacts
    s = s.replace("$", "").replace(",", "").replace(" ", "")
    try:
        return float(s)
    except Exception:
        return np.nan

if "gdp_country" in df.columns:
    df["gdp_country_num"] = df["gdp_country"].map(parse_money_like)
    df["log_gdp"] = np.log1p(df["gdp_country_num"])
else:
    df["gdp_country_num"] = np.nan
    df["log_gdp"] = np.nan

# Heavy-tail transform for wealth (does not modify original column)
df["log_finalWorth"] = np.log1p(pd.to_numeric(df["finalWorth"], errors="c

parse_summary = pd.DataFrame([{
    "col": "gdp_country",
    "raw_nonmissing_n": int(df["gdp_country"].notna().sum()) if "gdp_coun
    "parsed_nonmissing_n": int(df["gdp_country_num"].notna().sum()),
    "parse_success_rate_among_raw_nonmissing": float(df["gdp_country_num"
}])
save_table(parse_summary, "tab_M1_gdp_parse_summary", index=False)
```

	col	raw_nonmissing_n	parsed_nonmissing_n	parse_success_rate_a
0	gdp_country	2476	2476	

Out[11]:
```
{'csv': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M1_gdp_parse_summ
ary.csv',
 'tex': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M1_gdp_parse_summ
ary.tex'}
```

In [12]:
```
# ======================================
# 1.0 Variable reasoning matrix
# ======================================
variable_reasoning = pd.DataFrame([
    {'block': 'Core outcome', 'variables': 'finalWorth, rank', 'decision'
    {'block': 'Time / life-cycle', 'variables': 'age, birthYear, birthMon
    {'block': 'Geography', 'variables': 'country, city, state, residenceS
    {'block': 'Business category', 'variables': 'category, industries, so
    {'block': 'Identity / social labels', 'variables': 'selfMade, status,
    {'block': 'Scale macro variables', 'variables': 'gdp_country, populat
    {'block': 'Development macro variables', 'variables': 'gdp_pc, educat
    {'block': 'Institution / cost environment', 'variables': 'cpi_country
])
variable_reasoning.to_csv(OUTPUT_TAB / 'tab_M1_variable_reasoning_matrix.
variable_reasoning.to_latex(OUTPUT_TAB / 'tab_M1_variable_reasoning_matri
display(variable_reasoning)
```

	block	variables	decision	reason
0	Core outcome	finalWorth, rank	Use finalWorth; keep rank as descriptive metadata	finalWorth is the target wealth measure; rank ...
1	Time / life-cycle	age, birthYear, birthMonth, birthDay, birthDat...	Use age; treat date as snapshot metadata; avoi...	Age has substantive life-cycle meaning; date i...
2	Geography	country, city, state, residenceStateRegion, co...	Use country for clustering/strata; interpret s...	country is central but countryOfCitizenship ch...
3	Business category	category, industries, source, organization, title	Use category as canonical; keep source descrip...	category and industries are duplicate labels; ...
4	Identity / social labels	selfMade, status, gender, personName, firstNam...	Use selfMade and gender cautiously; treat stat...	selfMade/gender support group comparisons; sta...
5	Scale macro variables	gdp_country, population_country	Derive log_gdp and log_pop; avoid interpreting...	GDP mixes economic scale with population; mode...
6	Development macro variables	gdp_pc, education enrollment, life_expectancy_...	Use as development proxies in robustness and i...	They describe human-capital and welfare contex...
7	Institution / cost environment	cpi_country, cpi_change_country, tax_revenue_c...	Use as contextual covariates only with missing...	These are repeated country-level attributes an...

M1 / Variable Register Mini-Summary

The variable register now explicitly preserves the V2 reasoning style. `finalWorth` is the outcome; `rank` is informative but ordinal; `category` is canonical because `industries` is an exact duplicate; `age`, `selfMade`, `gender`, and `country` are interpretable grouping or control variables; `organization` and `title` are de-emphasized because missingness is above 87%; and `status` is used cautiously because an authoritative external codebook is not available.

The macro block is intentionally split into **scale** (`gdp_country`, `population_country`), **development** (`gdp_pc`, education, life expectancy), and **institution/cost environment** (CPI and tax variables). This is the conceptual bridge between the old workbook and the current formal models.

V2 Topic Reasoning Archive: What Must Be Kept

The V2 draft was valuable because it recorded the reasoning path, not only the final outputs. In this revision, that reasoning is preserved as an explicit topic map. The map below documents each research thread, the variables it touches, the analysis actions performed, the evidence produced, and the limits we must state. This is intentionally broader than the final statistical model: topic reasoning belongs in the appendix even when a thread is later narrowed in the main report.

In [13]:
```
# FULL_INTEGRATION_PASS_2026_05_01
# ======================================
# M1.5 V2 topic reasoning and RQ coverage archive
# ======================================
# Purpose: preserve the broad V2 reasoning trail in a structured, auditab

topic_reasoning = pd.DataFrame([
    {'topic': 'RQ0 data contract and exploratory audit', 'core_variables'
    {'topic': 'RQ1 wealth distribution', 'core_variables': 'finalWorth; l
    {'topic': 'RQ2 age and life-cycle structure', 'core_variables': 'age;
    {'topic': 'RQ3 industry/category structure', 'core_variables': 'categ
    {'topic': 'RQ4 selfMade, status, and gender', 'core_variables': 'self
    {'topic': 'RQ5 geography, US, and China', 'core_variables': 'country;
    {'topic': 'RQ6 macro variable architecture', 'core_variables': 'gdp_c
    {'topic': 'RQ7 regression and prediction', 'core_variables': 'log_fin
])
save_table(topic_reasoning, 'tab_M1_v2_topic_reasoning_archive', index=Fa

rq_coverage = pd.DataFrame([
    {'earlier_thread': 'RQ0 exploratory checks', 'present_location': 'M0-
    {'earlier_thread': 'RQ1 finalWorth distribution', 'present_location':
    {'earlier_thread': 'RQ2 age structure', 'present_location': 'M5/M7 an
    {'earlier_thread': 'RQ3 category/industry', 'present_location': 'M4/M
    {'earlier_thread': 'RQ4 selfMade/status/gender', 'present_location':
    {'earlier_thread': 'RQ5 geography US China', 'present_location': 'M5
    {'earlier_thread': 'RQ6 macro scale/development/institution', 'presen
```

```
    {'earlier_thread': 'RQ7 regression/prediction', 'present_location': '
])
save_table(rq_coverage, 'tab_M1_v2_to_current_rq_coverage', index=False)
```

	topic	core_variables	why_it_matters	analy
0	RQ0 data contract and exploratory audit	all raw columns; schema; hash; key candidates	Prevents silent drift and prevents treating di...	Hash raw CSV; free audit per
1	RQ1 wealth distribution	finalWorth; log_finalWorth; rank	Wealth is the core outcome and determines whet...	Robust quant histograms
2	RQ2 age and life-cycle structure	age; birthYear; finalWorth; selfMade; country	Age can proxy life-cycle accumulation, cohort ...	Age validit distribution;
3	RQ3 industry/category structure	category; industries; source; finalWorth; rank	Industry composition can confound wealth compa...	Verify category-indu to
4	RQ4 selfMade, status, and gender	selfMade; status; gender; age; finalWorth; cou...	These labels test social-composition patterns ...	Effect sizes; bo permut
5	RQ5 geography, US, and China	country; countryOfCitizenship; state; residenc...	Geographic concentration explains country-leve...	Country concentrati citize
6	RQ6 macro variable architecture	gdp_country; population_country; gdp_pc; educa...	Separates scale from development and prevents ...	Parse log_gdp/log_pop/gdp
7	RQ7 regression and prediction	log_finalWorth; log_gdp; age; selfMade; gender...	Satisfies regression/prediction requirement wh...	Raw-scale Mod Model B;

	earlier_thread	present_location	treatment	evidence
0	RQ0 exploratory checks	M0-M2 plus exploratory reasoning checks	Retained and formalized	schema freeze, identity checks, missingness ar...
1	RQ1 finalWorth distribution	M4 and report wealth distribution section	Strengthened statistically	CCDF, top-1 percent, Gini, BCa bootstrap
2	RQ2 age structure	M5/M7 and regression diagnostics	Partly retained with tighter limits	LOWESS age trajectory and age controls
3	RQ3 category/industry	M4/M6/M7	Retained and strengthened	category alias audit, pairwise tests, Model C ...
4	RQ4 selfMade/status/gender	M5/M6/M8	Retained with effect-size framing	Cohen d, Cliff delta, permutation p, robustnes...
5	RQ5 geography US China	M5 geography drill-down and report geography p...	Included selectively	country concentration, US state/region, China ...
6	RQ6 macro scale/development/institution	M1/M5/M7	Retained as interpretation boundary	variable reasoning matrix, macro schema, VIF, ...
7	RQ7 regression/prediction	M7	Strengthened substantially	HC3, nested F-test, CV, quantile regression

Out[13]:
```
{'csv': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M1_v2_to_current_
rq_coverage.csv',
 'tex': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M1_v2_to_current_
rq_coverage.tex'}
```

In [14]:
```python
# ======================================
# 1.3 Data-driven dictionary verification checks (evidence chain)
# ======================================
# We record key "meaning constraints" that are inferred from the data its
# These checks help prevent ambiguous interpretations later in the report
#
# Output artifact:
# - tab_M1_dictionary_verification_checks.{csv,tex}

checks = []

# Check 1: category vs industries (often redundant in this dataset)
if ("category" in df.columns) and ("industries" in df.columns):
```

```
    identical = bool((df["category"].fillna("__NA__") == df["industries"]
    checks.append({
        "check": "category_equals_industries",
        "result": "PASS" if identical else "FAIL",
        "evidence": "All rows identical after NA normalization" if identi
    })
else:
    checks.append({
        "check": "category_equals_industries",
        "result": "SKIP",
        "evidence": "One or both columns missing",
    })

# Check 2: US-only state fields (structural missingness hypothesis; forma
if ("country" in df.columns) and ("state" in df.columns):
    non_us_with_state = int(((df["country"] != "United States") & df["sta
    checks.append({
        "check": "state_present_only_for_US",
        "result": "PASS" if non_us_with_state == 0 else "FAIL",
        "evidence": f"Non-US records with state info: {non_us_with_state}
    })

checks_df = pd.DataFrame(checks)
save_table(checks_df, "tab_M1_dictionary_verification_checks", index=Fals
```

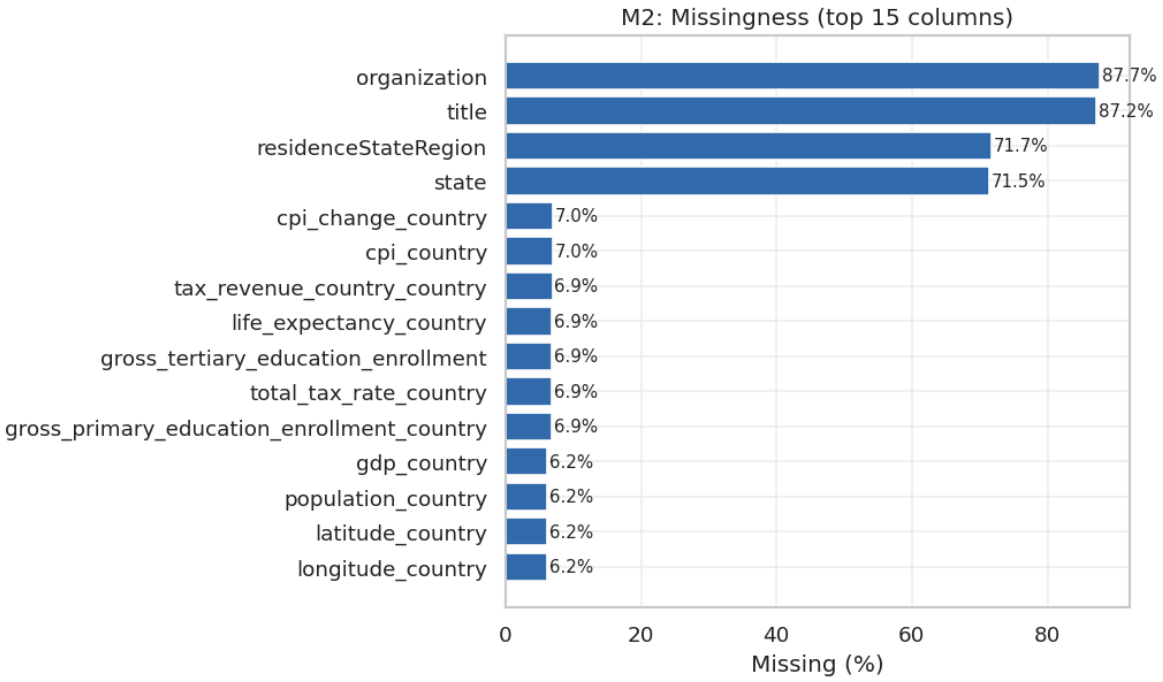
	check	result	evidence
0	category_equals_industries	PASS	All rows identical after NA normalization
1	state_present_only_for_US	PASS	Non-US records with state info: 0

Out[14]:
```
{'csv': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M1_dictionary_ver
ification_checks.csv',
 'tex': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M1_dictionary_ver
ification_checks.tex'}
```

In [15]:
```
# ======================================
# 2.1 Overall missingness profile (top columns)
# ======================================
# This figure/table is the baseline “data-quality footprint” used in RQ0.

schema = tab_schema.copy()  # from M1
schema["missing_pct"] = schema["missing_pct"].astype(float)

topk = 15
miss_top = schema.sort_values("missing_pct", ascending=False).head(topk)[
    ["column", "missing_pct", "missing_n", "dtype"]
].copy()
miss_top["missing_pct"] = (miss_top["missing_pct"] * 100).round(2)

save_table(miss_top, "tab_M2_missingness_top15", index=False)

fig, ax = plt.subplots(figsize=(8.6, 5.2))
ax.barh(miss_top["column"][::-1], miss_top["missing_pct"][::-1])
ax.set_title("M2: Missingness (top 15 columns)")
ax.set_xlabel("Missing (%)")
ax.set_ylabel("")
for i, v in enumerate(miss_top["missing_pct"][::-1].values):
```

```
    ax.text(v + 0.3, i, f"{v:.1f}%", va="center", fontsize=9)
ax.grid(alpha=0.25)
save_fig(fig, "fig_M2_missingness_top15")
```

	column	missing_pct	missing_n	dtype
0	organization	87.69	2315	str
1	title	87.16	2301	str
2	residenceStateRegion	71.70	1893	str
3	state	71.48	1887	str
4	cpi_change_country	6.97	184	float64
5	cpi_country	6.97	184	float64
6	tax_revenue_country_country	6.93	183	float64
7	life_expectancy_country	6.89	182	float64
8	gross_tertiary_education_enrollment	6.89	182	float64
9	total_tax_rate_country	6.89	182	float64
10	gross_primary_education_enrollment_country	6.86	181	float64
11	gdp_country	6.21	164	str
12	population_country	6.21	164	float64
13	latitude_country	6.21	164	float64
14	longitude_country	6.21	164	float64

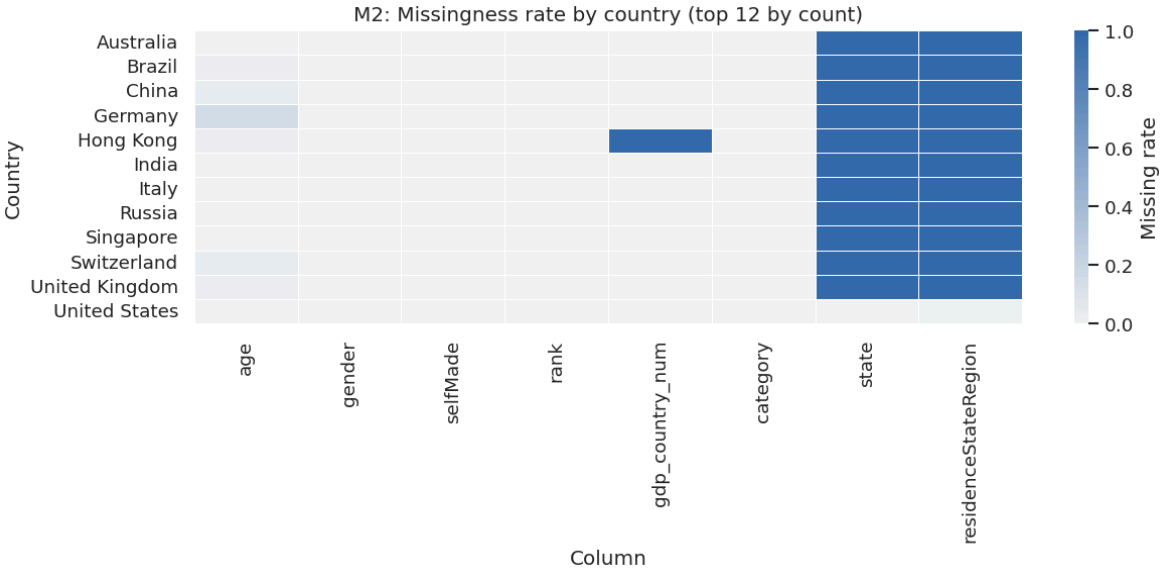


Out[15]:
```
{'pdf': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M2_missingness_to
p15.pdf',
 'png': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M2_missingness_to
p15.png'}
```

Chapter 2 — Data Quality Audit (M2)

In [16]:
```
# ======================================
# 2.2 Missingness by country (heatmap for top countries)
# ======================================
# Rationale:
# - If missingness is concentrated in certain countries, naive comparison

group_col = "country"
focus_cols = ["age", "gender", "selfMade", "rank", "gdp_country_num", "ca
focus_cols = [c for c in focus_cols if c in df.columns]

# Choose top countries by sample size
top_countries = df[group_col].value_counts(dropna=False).head(12).index.t
df_top = df[df[group_col].isin(top_countries)].copy()

miss_by_country = (
    df_top.groupby(group_col)[focus_cols]
          .apply(lambda g: g.isna().mean())
          .reset_index()
)
miss_m = miss_by_country.set_index(group_col)

save_table(miss_m.reset_index(), "tab_M2_missingness_by_country_top12", i

fig, ax = plt.subplots(figsize=(10.5, 5.0))
sns.heatmap(miss_m, cmap=CMAP_PRIMARY, vmin=0, vmax=1, linewidths=0.5, li
ax.set_title("M2: Missingness rate by country (top 12 by count)")
ax.set_xlabel("Column")
ax.set_ylabel("Country")
save_fig(fig, "fig_M2_missingness_heatmap_country_top12")
```

	country	age	gender	selfMade	rank	gdp_country_num	categor
0	Australia	0.000000	0.0	0.0	0.0	0.0	0.
1	Brazil	0.022727	0.0	0.0	0.0	0.0	0.
2	China	0.036329	0.0	0.0	0.0	0.0	0.
3	Germany	0.137255	0.0	0.0	0.0	0.0	0.
4	Hong Kong	0.029412	0.0	0.0	0.0	1.0	0.
5	India	0.000000	0.0	0.0	0.0	0.0	0.
6	Italy	0.000000	0.0	0.0	0.0	0.0	0.
7	Russia	0.000000	0.0	0.0	0.0	0.0	0.
8	Singapore	0.000000	0.0	0.0	0.0	0.0	0.
9	Switzerland	0.038462	0.0	0.0	0.0	0.0	0.
10	United Kingdom	0.024390	0.0	0.0	0.0	0.0	0.
11	United States	0.000000	0.0	0.0	0.0	0.0	0.

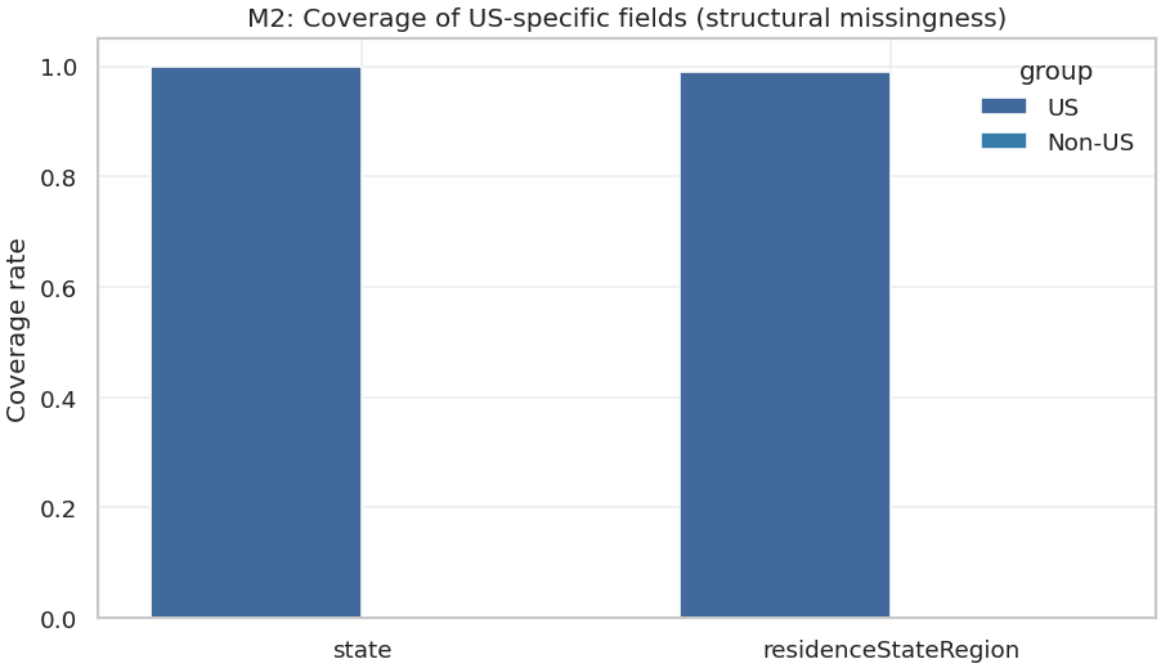


Out[16]:
```
{'pdf': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M2_missingness_he
atmap_country_top12.pdf',
 'png': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M2_missingness_he
atmap_country_top12.png'}
```

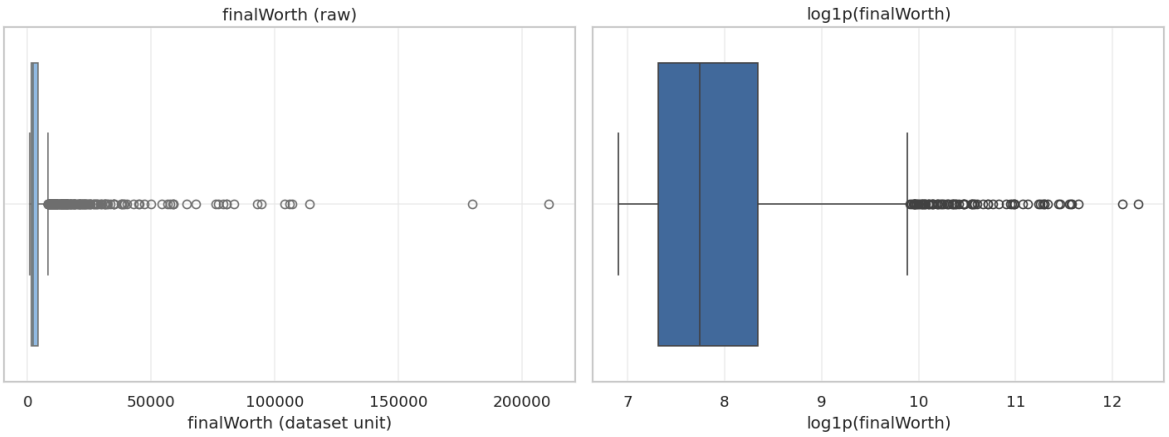
In [17]:
```
# ========================================
# 2.3 Structural missingness: state / residenceStateRegion (US-only hypot
# ========================================
# Hypothesis: these fields are only populated for US records, so high ove
# does not necessarily indicate low data quality.

if "country" in df.columns:
    is_us = (df["country"] == "United States")
else:
    is_us = pd.Series([False] * len(df))

cols = [c for c in ["state", "residenceStateRegion"] if c in df.columns]
if cols:
    cov = pd.DataFrame({
        "group": ["US", "Non-US"],
        "n_records": [int(is_us.sum()), int((~is_us).sum())],
    })
    for c in cols:
        cov[f"{c}_coverage"] = [
            float(df.loc[is_us, c].notna().mean()) if is_us.sum() > 0 els
            float(df.loc[~is_us, c].notna().mean()) if (~is_us).sum() > 0
        ]
    save_table(cov, "tab_M2_us_only_state_coverage", index=False)

    # Visualization
    fig, ax = plt.subplots(figsize=(7.8, 4.6))
    plot_df = cov.melt(id_vars=["group", "n_records"], value_vars=[f"{c}_
                       var_name="field", value_name="coverage")
    plot_df["field"] = plot_df["field"].str.replace("_coverage", "", rege

    sns.barplot(data=plot_df, x="field", y="coverage", hue="group", ax=ax
    ax.set_title("M2: Coverage of US-specific fields (structural missingn
    ax.set_xlabel("")
    ax.set_ylabel("Coverage rate")
    ax.set_ylim(0, 1.05)
```

```
    ax.grid(alpha=0.25)
    save_fig(fig, "fig_M2_us_only_state_coverage")
else:
    display(pd.DataFrame([{"note": "state/residenceStateRegion not presen
```

	group	n_records	state_coverage	residenceStateRegion_coverage
0	US	754	0.998674	0.990716
1	Non-US	1886	0.000000	0.000000



In [18]:
```
# ========================================
# 2.4 Outliers: finalWorth on raw vs log scale
# ========================================
# Heavy-tailed outcomes can dominate means, correlations, and regression
# We visualize both scales early so downstream modeling choices are justi

y_raw = pd.to_numeric(df["finalWorth"], errors="coerce")
y_log = df["log_finalWorth"]

fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.6))

sns.boxplot(x=y_raw, ax=axes[0], color=next_color())
axes[0].set_title("finalWorth (raw)")
axes[0].set_xlabel("finalWorth (dataset unit)")
axes[0].grid(alpha=0.25)

sns.boxplot(x=y_log, ax=axes[1])
axes[1].set_title("log1p(finalWorth)")
axes[1].set_xlabel("log1p(finalWorth)")
axes[1].grid(alpha=0.25)

save_fig(fig, "fig_M2_outliers_raw_vs_log_box")
```



Out[18]:
```
{'pdf': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M2_outliers_raw_v
s_log_box.pdf',
 'png': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M2_outliers_raw_v
s_log_box.png'}
```

In [19]:
```
# ========================================
# 2.5 Consolidated DQ issues table (evidence chain)
# ========================================
issues = []

# Duplicate rows (exact duplicates)
n_dup_rows = int(df.duplicated().sum())
issues.append({"issue": "Exact duplicate rows", "metric": "count", "value

# Basic range sanity checks (illustrative; adjust if project documentatio
def count_outside(col: str, low: float, high: float) -> int:
    s = pd.to_numeric(df[col], errors="coerce")
    return int(((s < low) | (s > high)).sum())

if "age" in df.columns:
    issues.append({"issue": "Age outside [0, 120]", "metric": "count", "v
if "rank" in df.columns:
    issues.append({"issue": "Rank <= 0", "metric": "count", "value": int(

# GDP parse failures (when raw non-missing but numeric is missing)
if "gdp_country" in df.columns:
    raw_nonmiss = df["gdp_country"].notna()
    parse_fail = int((raw_nonmiss & df["gdp_country_num"].isna()).sum())
    issues.append({"issue": "GDP parse failures", "metric": "count", "val

issues_df = pd.DataFrame(issues)
save_table(issues_df, "tab_M2_dq_issues_summary", index=False)
```

	issue	metric	value	note
0	Exact duplicate rows	count	0	informational; handling decided in M3
1	Age outside [0, 120]	count	0	sanity check
2	Rank <= 0	count	0	sanity check
3	GDP parse failures	count	0	requires cleaning rule or exclusion in regression

Out[19]:
```
{'csv': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M2_dq_issues_summ
ary.csv',
 'tex': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M2_dq_issues_summ
ary.tex'}
```

M2 / Data Quality Defense Summary

The most important missingness result is not the missingness percentage itself but the mechanism. `organization` and `title` are sparse public-visibility fields; `state` and `residenceStateRegion` are structurally US-specific; and macro-variable missingness clusters by country-name linkage. Treating all missingness as random would be statistically wrong.

Decision impact. The report keeps all 2,640 rows, uses derived variables rather than mutating raw fields, avoids high-missingness text fields in main inference, and uses country-aware reasoning when interpreting macro variables.

Cleaning Contract and Drop/Keep Rationale

The earlier workbook used a useful contract: raw data are immutable, cleaning must be logged, and any removed or de-emphasized column needs a reason. The current workflow follows the same logic. We do not delete raw columns to make the dataset look clean; instead, we derive analysis-ready columns (`log_finalWorth` , `gdp_country_num` , `log_gdp` , `log_pop` , `gdp_pc` , `log_gdp_pc`) and keep a documented audit trail.

Reviewer-defense implications:

- `industries` is not a second independent sector variable because it exactly duplicates `category` .
- `state` and `residenceStateRegion` are not globally missing at random; they are primarily US-specific fields.
- `organization` and `title` are not reliable main-model covariates because their missingness is too high.
- `date` is treated as a snapshot timestamp, not evidence of temporal dynamics.
- macro indicators are repeated for all billionaires from the same country, so standard errors and validation must respect country grouping.

In [20]:
```
# ======================================
# 2.6 Artifact index (M2 outputs)
# ======================================
artifact_index = pd.DataFrame([
    {"module": "M2", "type": "table", "name": "tab_M2_missingness_top15"}
    {"module": "M2", "type": "figure", "name": "fig_M2_missingness_top15"
    {"module": "M2", "type": "table", "name": "tab_M2_missingness_by_coun
    {"module": "M2", "type": "figure", "name": "fig_M2_missingness_heatma
```

```
    {"module": "M2", "type": "table", "name": "tab_M2_us_only_state_cover
    {"module": "M2", "type": "figure", "name": "fig_M2_us_only_state_cove
    {"module": "M2", "type": "figure", "name": "fig_M2_outliers_raw_vs_lo
    {"module": "M2", "type": "table", "name": "tab_M2_dq_issues_summary"}
])
save_table(artifact_index, "tab_M2_artifact_index", index=False)
```

	module	type	name
0	M2	table	tab_M2_missingness_top15
1	M2	figure	fig_M2_missingness_top15
2	M2	table	tab_M2_missingness_by_country_top12
3	M2	figure	fig_M2_missingness_heatmap_country_top12
4	M2	table	tab_M2_us_only_state_coverage
5	M2	figure	fig_M2_us_only_state_coverage
6	M2	figure	fig_M2_outliers_raw_vs_log_box
7	M2	table	tab_M2_dq_issues_summary

Out[20]:
```
{'csv': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M2_artifact_inde
x.csv',
 'tex': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M2_artifact_inde
x.tex'}
```

In [21]:
```
# ========================================
# 3.1 Minimal cleaning pipeline (auditable, non-destructive)
# ========================================
# Principles:
# 1) Never overwrite raw columns with "guessed" semantics.
# 2) Prefer type conversions that are reversible and documented.
# 3) Add derived columns with explicit names (e.g., *_num, log_*).

df_clean = df.copy()

clean_log = []

# 1) Strip whitespace for object columns (safe normalization)
obj_cols = df_clean.select_dtypes(include=["object"]).columns.tolist()
for col in obj_cols:
    before_na = int(df_clean[col].isna().sum())
    df_clean[col] = df_clean[col].astype(str).str.strip().replace({"nan":
    after_na = int(df_clean[col].isna().sum())
    if after_na != before_na:
        clean_log.append({"step": "strip_whitespace", "column": col, "not

# 2) Coerce key numeric columns
for col in ["finalWorth", "age", "rank"]:
    if col in df_clean.columns:
        before_nonnum = int(pd.to_numeric(df_clean[col], errors="coerce")
        df_clean[col] = pd.to_numeric(df_clean[col], errors="coerce")
        after_nonnum = int(df_clean[col].isna().sum())
        clean_log.append({"step": "to_numeric", "column": col, "note": f"

# 3) Canonical handling for redundant columns (do NOT drop raw columns)
```

```
# Data check in M1 suggests category == industries in this dataset.
# We keep both, but treat 'category' as canonical in analysis.
if ("category" in df_clean.columns) and ("industries" in df_clean.columns
    df_clean["category_canonical"] = df_clean["category"]
    clean_log.append({"step": "canonical_category", "column": "category_c
else:
    df_clean["category_canonical"] = df_clean.get("category", np.nan)

# 4) Ensure GDP derivatives exist (already created in M1)
if "gdp_country_num" not in df_clean.columns:
    df_clean["gdp_country_num"] = np.nan
if "log_gdp" not in df_clean.columns:
    df_clean["log_gdp"] = np.log1p(df_clean["gdp_country_num"])

# 5) Ensure log outcome exists
if "log_finalWorth" not in df_clean.columns:
    df_clean["log_finalWorth"] = np.log1p(df_clean["finalWorth"])

# 6) Optional flags helpful for later stratification / robustness
if "country" in df_clean.columns:
    df_clean["is_us"] = (df_clean["country"] == "United States")
if "selfMade" in df_clean.columns:
    # keep original; add a boolean helper when selfMade is 0/1-like
    sm = pd.to_numeric(df_clean["selfMade"], errors="coerce")
    df_clean["selfMade_bool"] = sm.where(sm.isna(), sm.astype(int).astype

display(pd.DataFrame([{"rows": df_clean.shape[0], "cols": df_clean.shape[
```

```
/tmp/ipykernel_325283/3700140925.py:14: Pandas4Warning: For backward compa
tibility, 'str' dtypes are included by select_dtypes when 'object' dtype i
s specified. This behavior is deprecated and will be removed in a future v
ersion. Explicitly pass 'str' to `include` to select them, or to `exclude`
to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#str
ing-migration-select-dtypes for details on how to write code that works wi
th pandas 2 and 3.
  obj_cols = df_clean.select_dtypes(include=["object"]).columns.tolist()
```

	rows	cols
0	2640	41

Chapter 3 — Cleaning & Feature Engineering (M3)

In [22]:
```
# ======================================
# 3.2 Cleaning log (what changed)
# ======================================
# This log is meant to be report-friendly: short, explicit, and auditable

clean_log_df = pd.DataFrame(clean_log) if len(clean_log) else pd.DataFram
save_table(clean_log_df, "tab_M3_cleaning_log", index=False)
```

	step	column	note
0	to_numeric	finalWorth	NA after coercion: 0 (was 0 under coercion check)
1	to_numeric	age	NA after coercion: 65 (was 65 under coercion c...
2	to_numeric	rank	NA after coercion: 0 (was 0 under coercion check)
3	canonical_category	category_canonical	Set to category (industries retained as raw co...

Out[22]:
```
{'csv': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M3_cleaning_log.c
sv',
 'tex': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M3_cleaning_log.t
ex'}
```

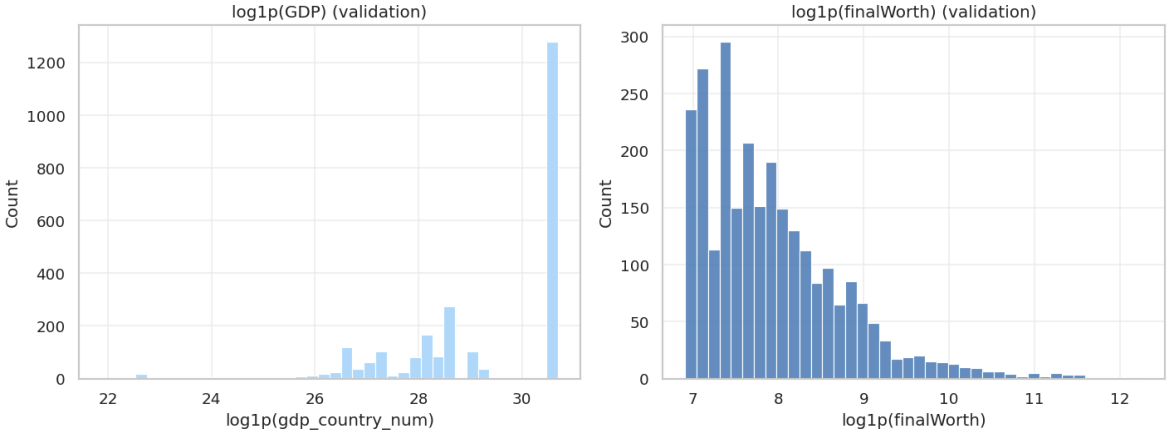
In [23]:
```
# ======================================
# 3.3 Save df_clean (recommended for reproducibility)
# ======================================
clean_dir = ROOT / "data" / "clean"
clean_dir.mkdir(parents=True, exist_ok=True)

clean_csv = clean_dir / "billionaires_clean.csv"
df_clean.to_csv(clean_csv, index=False)

save_table(pd.DataFrame([{"clean_csv": str(clean_csv), "rows": df_clean.s
           "tab_M3_clean_dataset_manifest", index=False)
```

	clean_csv	rows	cols
0	/mnt/e/canfiles/Common courses study/大学通识/25-2...	2640	41

Out[23]:
```
{'csv': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M3_clean_dataset_
manifest.csv',
 'tex': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M3_clean_dataset_
manifest.tex'}
```

In [24]:
```
# ======================================
# 3.4 Validation plots (quick sanity checks)
# ======================================
# These are not “final descriptive results”; they confirm that critical p

fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.6))

# GDP (log) distribution
sns.histplot(df_clean["log_gdp"].dropna(), color=next_color(), bins=40, a
axes[0].set_title("log1p(GDP) (validation)")
axes[0].set_xlabel("log1p(gdp_country_num)")
axes[0].grid(alpha=0.25)

# Wealth (log) distribution
sns.histplot(df_clean["log_finalWorth"].dropna(), color=next_color(), bin
axes[1].set_title("log1p(finalWorth) (validation)")
axes[1].set_xlabel("log1p(finalWorth)")
```

```
axes[1].grid(alpha=0.25)

save_fig(fig, "fig_M3_validation_log_distributions")
```



Out[24]:
```
{'pdf': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M3_validation_log
_distributions.pdf',
 'png': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M3_validation_log
_distributions.png'}
```

M3 / Cleaning and Feature-Engineering Mini-Summary

The cleaning pipeline is conservative: no records are deleted, `finalWorth`, `age`, and `rank` are coerced to numeric analysis views, `category_canonical` is derived from `category`, and the cleaned dataset is written as `data/clean/billionaires_clean.csv` with 2,640 rows and 41 columns. The important derived variables are `log_finalWorth`, `gdp_country_num`, `log_gdp`, `log_pop`, `gdp_pc`, and `log_gdp_pc`.

Boundary. These transformations make the dataset analyzable; they do not create causal identification. The later models remain observational and cross-sectional.

Chapter 4 - Descriptive Statistics and Distributional Evidence (M4)

This chapter converts the V2 exploratory wealth-distribution discussion into a compact evidence block: robust quantiles, raw/log visualization, CCDF tail behavior, concentration, and top country/category composition.

In [25]:
```
# ========================================
# 4.1 Summary statistics + quantiles (finalWorth)
# ========================================
# Reporting both raw and log summaries is essential for heavy-tailed outc

y = df_clean["finalWorth"].dropna()
y_log = df_clean["log_finalWorth"].dropna()
```

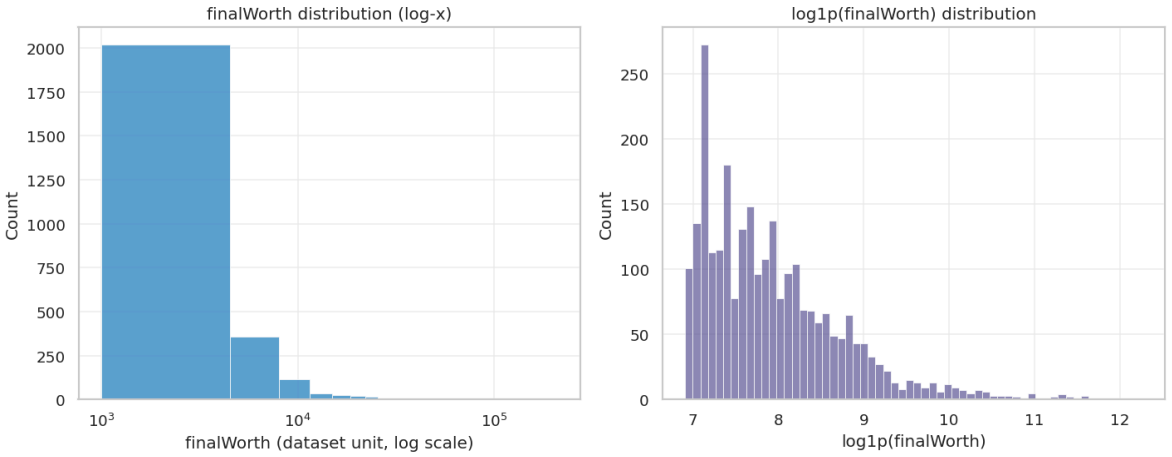
```
def describe_with_quantiles(s: pd.Series, name: str) -> pd.DataFrame:
    qs = [0.01, 0.05, 0.10, 0.25, 0.50, 0.75, 0.90, 0.95, 0.99]
    d = s.describe(percentiles=qs).to_frame(name).reset_index().rename(co
    return d

tab_y = describe_with_quantiles(y, "finalWorth")
tab_ylog = describe_with_quantiles(y_log, "log1p(finalWorth)")
tab_desc = tab_y.merge(tab_ylog, on="stat", how="outer")

save_table(tab_desc, "tab_M4_finalWorth_describe", index=False)
```

	stat	finalWorth	log1p(finalWorth)
0	1%	1000.000000	6.908755
1	10%	1200.000000	7.090910
2	25%	1500.000000	7.313887
3	5%	1100.000000	7.003974
4	50%	2300.000000	7.741099
5	75%	4200.000000	8.343078
6	90%	8000.000000	8.987322
7	95%	13320.000000	9.497076
8	99%	41808.000000	10.640328
9	count	2640.000000	2640.000000
10	max	211000.000000	12.259618
11	mean	4623.787879	7.930881
12	min	1000.000000	6.908755
13	std	9834.240939	0.820638

Out[25]:
```
{'csv': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M4_finalWorth_des
cribe.csv',
 'tex': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M4_finalWorth_des
cribe.tex'}
```

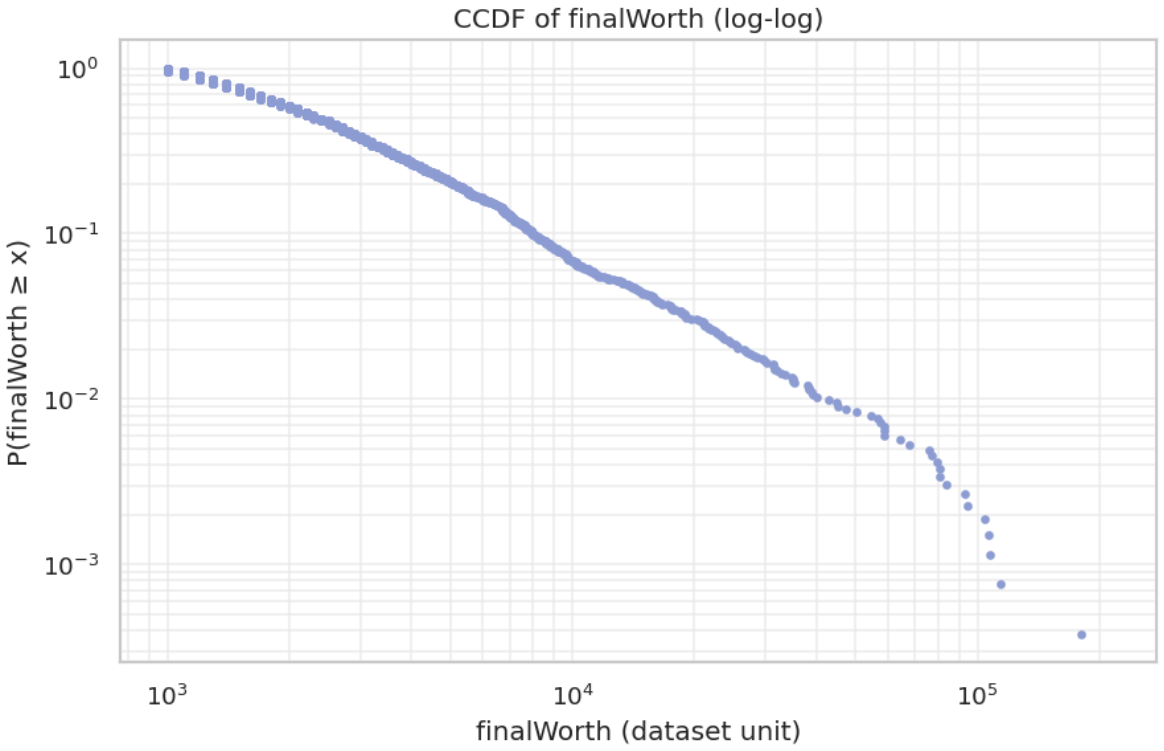
In [26]:
```
# ========================================
# 4.2 Distribution plots (raw vs log)
# ========================================
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.8))

# Raw scale: use log-x to make the distribution visible without discardin
sns.histplot(y, bins=60, ax=axes[0], color=next_color())
axes[0].set_xscale("log")
axes[0].set_title("finalWorth distribution (log-x)")
axes[0].set_xlabel("finalWorth (dataset unit, log scale)")
axes[0].set_ylabel("Count")
axes[0].grid(alpha=0.25)

# Log scale distribution
sns.histplot(y_log, bins=60, ax=axes[1], color=next_color())
axes[1].set_title("log1p(finalWorth) distribution")
axes[1].set_xlabel("log1p(finalWorth)")
```

```
axes[1].set_ylabel("Count")
axes[1].grid(alpha=0.25)

save_fig(fig, "fig_M4_finalWorth_distribution_raw_and_log")
```



Out[26]:
```
{'pdf': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2104 Applied statistics/final_project_v3/output/fig/fig_M4_finalWorth_distribution_raw_and_log.pdf',
 'png': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2104 Applied statistics/final_project_v3/output/fig/fig_M4_finalWorth_distribution_raw_and_log.png'}
```

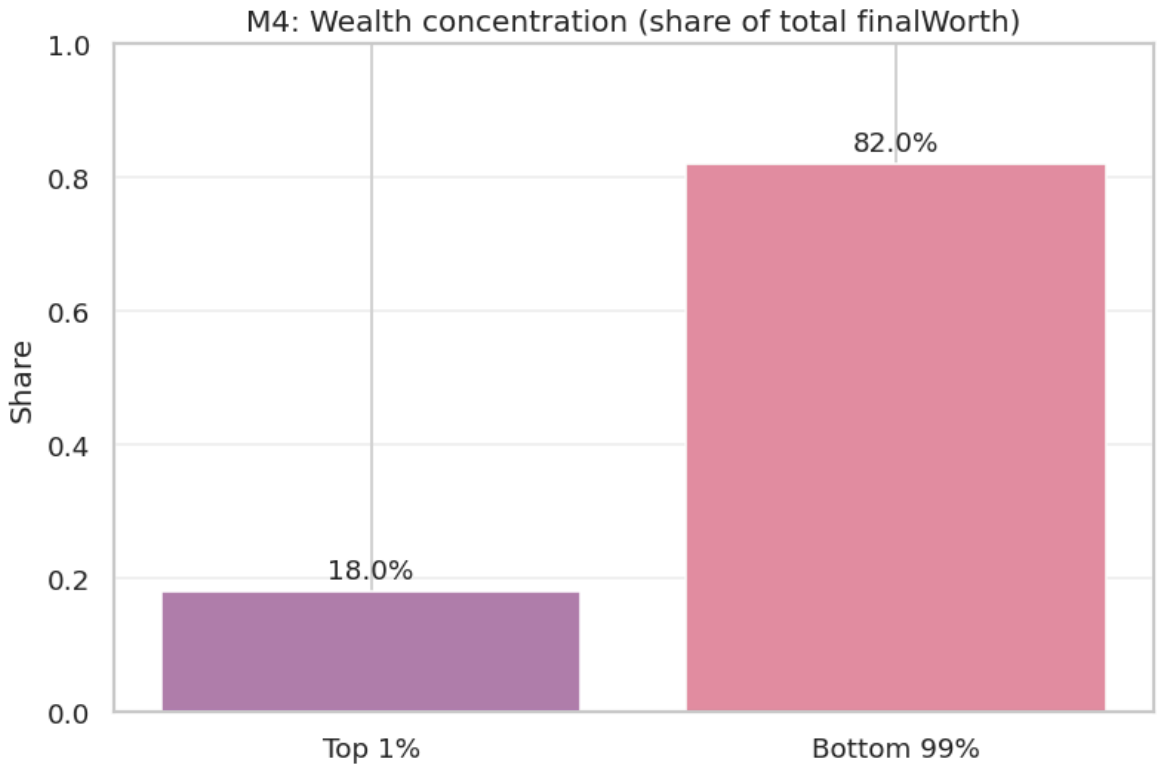
In [27]:
```
# ========================================
# 4.3 CCDF (heavy-tail visualization)
# ========================================
# CCDF on log-log axes is a standard diagnostic for heavy tails.

x = y.sort_values().values
x = x[x > 0]
n = len(x)
ccdf = 1.0 - (np.arange(1, n + 1) / n)

fig, ax = plt.subplots(figsize=(7.6, 5.0))
ax.plot(x, ccdf, marker=".", linestyle="none", color=next_color())
ax.set_xscale("log")
ax.set_yscale("log")
ax.set_title("CCDF of finalWorth (log-log)")
ax.set_xlabel("finalWorth (dataset unit)")
ax.set_ylabel("P(finalWorth ≥ x)")
ax.grid(alpha=0.25, which="both")
save_fig(fig, "fig_M4_finalWorth_ccdf_loglog")
```



Out[27]:
```
{'pdf': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M4_finalWorth_ccd
f_loglog.pdf',
 'png': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M4_finalWorth_ccd
f_loglog.png'}
```

In [28]:
```
# ========================================
# 4.4 Concentration: top 1% wealth share
# ========================================
x_desc = y.sort_values(ascending=False).values
n = len(x_desc)
k = max(1, int(np.ceil(0.01 * n)))

top_share = float(x_desc[:k].sum() / x_desc.sum())
rest_share = 1.0 - top_share

tab_conc = pd.DataFrame([{
    "n_total": n,
    "top_1pct_n": k,
    "top_1pct_wealth_share": top_share,
    "bottom_99pct_wealth_share": rest_share,
}])
save_table(tab_conc, "tab_M4_top1pct_wealth_share", index=False)

fig, ax = plt.subplots(figsize=(6.8, 4.6))
ax.bar(["Top 1%", "Bottom 99%"], [top_share, rest_share], color=[next_col
ax.set_title("M4: Wealth concentration (share of total finalWorth)")
ax.set_ylabel("Share")
ax.set_ylim(0, 1.0)
for i, v in enumerate([top_share, rest_share]):
    ax.text(i, v + 0.02, f"{v:.1%}", ha="center", fontsize=11)
ax.grid(alpha=0.25, axis="y")
save_fig(fig, "fig_M4_top1pct_wealth_share")
```

	n_total	top_1pct_n	top_1pct_wealth_share	bottom_99pct_wealth_share
0	2640	27	0.179801	0.820199

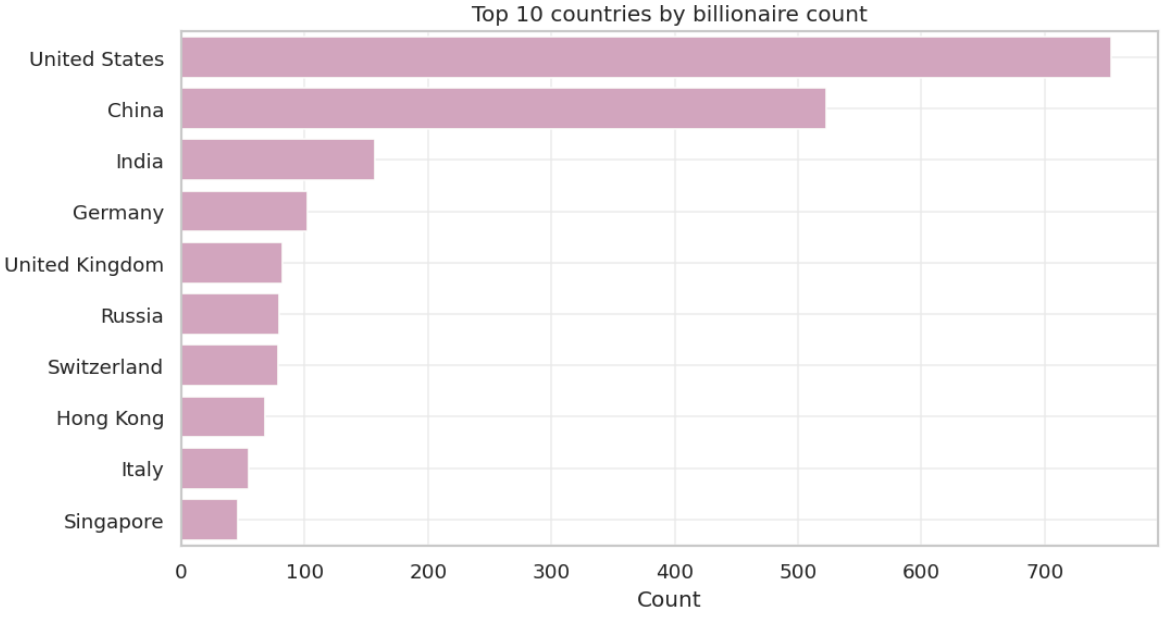


Out[28]:
```
{'pdf': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M4_top1pct_wealth
_share.pdf',
 'png': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M4_top1pct_wealth
_share.png'}
```

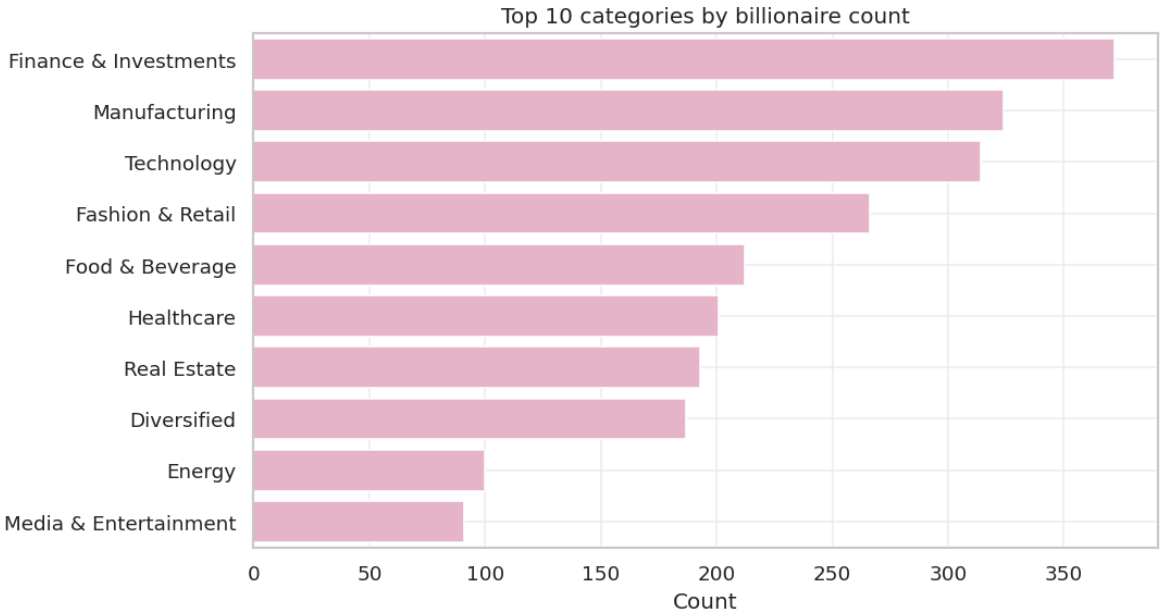
In [29]:
```
# ======================================
# 4.5 Top groups (counts and median wealth)
# ======================================
def topk_table(df_in: pd.DataFrame, group: str, k: int = 10) -> pd.DataFr
    g = df_in.groupby(group, dropna=False).agg(
        n=("finalWorth", "size"),
        median_finalWorth=("finalWorth", "median"),
        mean_finalWorth=("finalWorth", "mean"),
    ).reset_index()
    g = g.sort_values("n", ascending=False).head(k)
    return g

# Top countries by count
if "country" in df_clean.columns:
    top_country = topk_table(df_clean, "country", k=10)
    save_table(top_country, "tab_M4_top10_country_by_count", index=False)

    fig, ax = plt.subplots(figsize=(9.2, 5.0))
    sns.barplot(data=top_country, y="country", x="n", ax=ax, color=next_c
    ax.set_title("Top 10 countries by billionaire count")
    ax.set_xlabel("Count")
    ax.set_ylabel("")
    ax.grid(alpha=0.25)
    save_fig(fig, "fig_M4_top10_country_count")
else:
```

```
    display(pd.DataFrame([{"note": "country column not found"}]))

# Top categories by count (canonical)
if "category_canonical" in df_clean.columns:
    top_cat = topk_table(df_clean, "category_canonical", k=10)
    top_cat = top_cat.rename(columns={"category_canonical": "category"})
    save_table(top_cat, "tab_M4_top10_category_by_count", index=False)

    fig, ax = plt.subplots(figsize=(9.2, 5.0))
    sns.barplot(data=top_cat, y="category", x="n", ax=ax, color=next_colo
    ax.set_title("Top 10 categories by billionaire count")
    ax.set_xlabel("Count")
    ax.set_ylabel("")
    ax.grid(alpha=0.25)
    save_fig(fig, "fig_M4_top10_category_count")
else:
    display(pd.DataFrame([{"note": "category_canonical not found"}]))
```

	country	n	median_finalWorth	mean_finalWorth
74	United States	754	2900.0	6067.771883
16	China	523	1900.0	3452.198853
31	India	157	2100.0	4004.458599
26	Germany	102	2650.0	4530.392157
73	United Kingdom	82	2600.0	4520.731707
58	Russia	79	2500.0	4443.037975
65	Switzerland	78	3100.0	5255.128205
29	Hong Kong	68	2700.0	4727.941176
35	Italy	55	2300.0	2852.727273
59	Singapore	46	2150.0	3000.000000



	category	n	median_finalWorth	mean_finalWorth
5	Finance & Investments	372	2600.0	4314.784946
10	Manufacturing	324	2000.0	3145.061728
16	Technology	314	2200.0	5980.573248
4	Fashion & Retail	266	2500.0	6386.466165
6	Food & Beverage	212	2500.0	4515.094340
8	Healthcare	201	2100.0	3200.000000
13	Real Estate	193	2300.0	3406.217617
2	Diversified	187	2300.0	4840.641711
3	Energy	100	2450.0	4535.000000
11	Media & Entertainment	91	2500.0	4697.802198



RQ1/RQ3 Mini-Summary: Wealth Is Heavy-Tailed and Composition Matters

`finalWorth` is extremely right-skewed: the median is USD 2,300M, the mean is USD 4,624M, the 99th percentile is USD 41,808M, and the maximum is USD 211,000M. The top 1% of records, only 27 individuals, hold 17.98% of total billionaire wealth in this dataset. The dataset-level Gini estimate is 0.549 with a bootstrap 95% interval around [0.518, 0.579].

Composition also matters. The largest residence-country groups are the United States (754), China (523), and India (157). The largest industry categories are Finance & Investments (372), Manufacturing (324), Technology (314), and Fashion & Retail (266). These counts do not directly prove national or sectoral advantage; they define the composition that later inference must respect.

Chapter 5 - Bivariate EDA, Stratification, and Group Comparisons (M5)

This chapter keeps the old notebook's habit of checking relationships visually and by subgroup before formal inference. The goal is to identify where a pooled relationship is stable, weak, or possibly hiding heterogeneity.

In [30]:
```
# ======================================
# M5.1 EDA helpers (plotting + safe subsets)
# ======================================

def _coerce_boolish(series: pd.Series) -> pd.Series:
    """Normalize common boolean encodings (0/1, True/False, yes/no) to {0
    s = series.copy()
    if pd.api.types.is_bool_dtype(s):
        return s.astype("Int64")
    # common numeric encodings
    if pd.api.types.is_numeric_dtype(s):
        return s.round().astype("Int64")
    # string encodings
    s = s.astype(str).str.strip().str.lower().replace({"nan": np.nan})
    mapping = {
        "1": 1, "true": 1, "yes": 1, "y": 1, "t": 1,
        "0": 0, "false": 0, "no": 0, "n": 0, "f": 0,
    }
    return s.map(mapping).astype("Int64")

def scatter(df: pd.DataFrame, x: str, y: str, *, hue: Optional[str] = Non
            logx: bool = False, logy: bool = False, title: str = "",
            note: Optional[str] = None) -> mpl.figure.Figure:
    """Scatter with consistent aesthetics; supports log axes (for heavy t
    d = df[[x, y] + ([hue] if hue else [])].dropna().copy()
    fig, ax = plt.subplots(figsize=(7.2, 5.2))
    base_color = COLOR["primary"] if hue is None else None

    if hue is None:
        ax.scatter(d[x], d[y], s=18, alpha=0.28, c=base_color, edgecolors
    else:
        # For legibility: limit hue levels if too many
        if d[hue].nunique(dropna=True) > 12:
            top_levels = d[hue].value_counts().head(12).index
            d = d[d[hue].isin(top_levels)].copy()
        sns.scatterplot(data=d, x=x, y=y, hue=hue, palette=dict(zip(sorte

    # Add a robust trend line (on the *plotted* scale)
    try:
        sns.regplot(data=d, x=x, y=y, scatter=False, lowess=True,
                    line_kws={"lw": 2.0, "color": COLOR["secondary"]}, ax
    except Exception:
        pass

    if logx:
        ax.set_xscale("log")
    if logy:
        ax.set_yscale("log")

    ax.set_title(title)
```

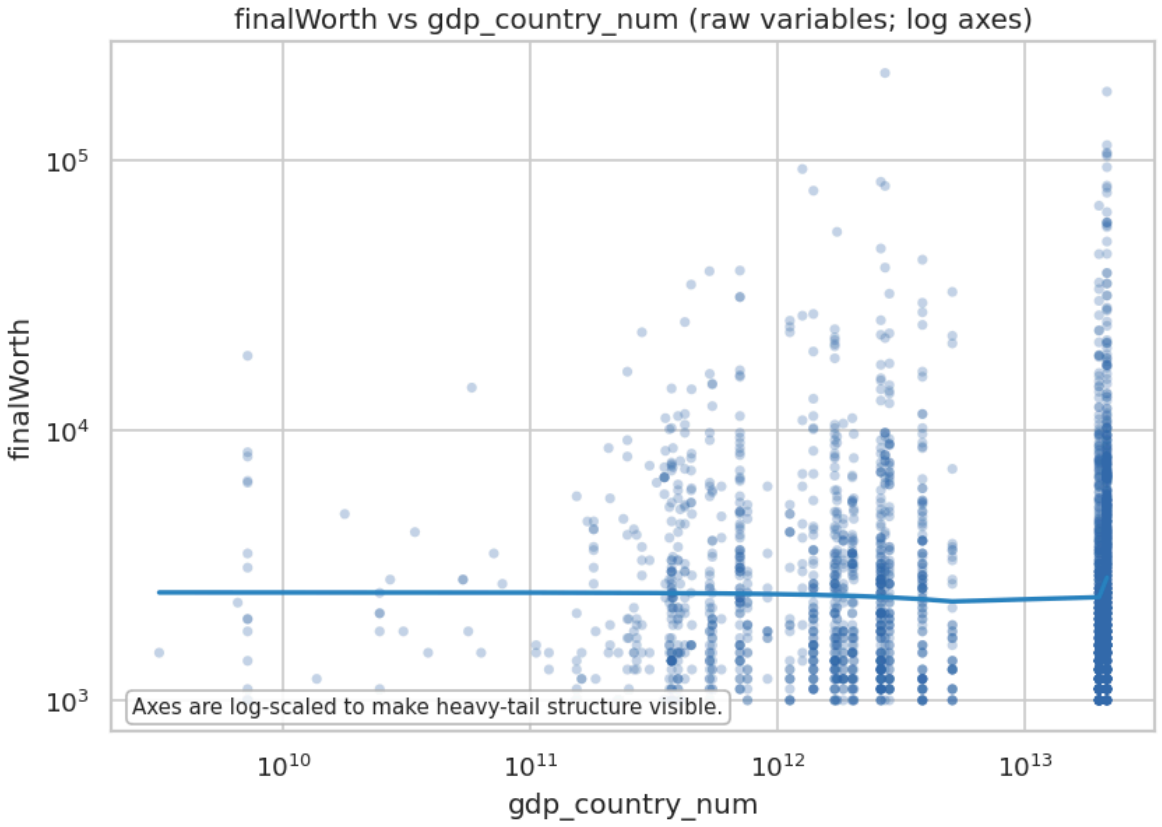
```
    ax.set_xlabel(x)
    ax.set_ylabel(y)
    if note:
        ax.text(0.02, 0.02, note, transform=ax.transAxes, fontsize=9, va=
                bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="0.7",
    return fig

def boxplot(df: pd.DataFrame, y: str, group: str, *, title: str = "", ord
    d = df[[y, group]].dropna().copy()
    if order is None:
        order = d[group].value_counts().index.tolist()
    fig, ax = plt.subplots(figsize=(7.2, 5.2))
    sns.boxplot(data=d, x=group, y=y, hue=group, order=order, ax=ax, pale
    sns.stripplot(data=d, x=group, y=y, order=order, ax=ax, color="0.25",
    ax.set_title(title)
    ax.set_xlabel(group)
    ax.set_ylabel(y)
    return fig

def spearman_heatmap(df: pd.DataFrame, cols: List[str], *, title: str = "
    use = [c for c in cols if c in df.columns]
    d = df[use].copy()
    d = d.apply(pd.to_numeric, errors="coerce")
    corr = d.corr(method="spearman")
    fig, ax = plt.subplots(figsize=(8.2, 6.2))
    sns.heatmap(corr, cmap=CMAP_PRIMARY, center=0, vmin=-1, vmax=1, annot
                linewidths=0.5, cbar_kws={"label": "Spearman correlation"
    ax.set_title(title)
    return fig, corr
```

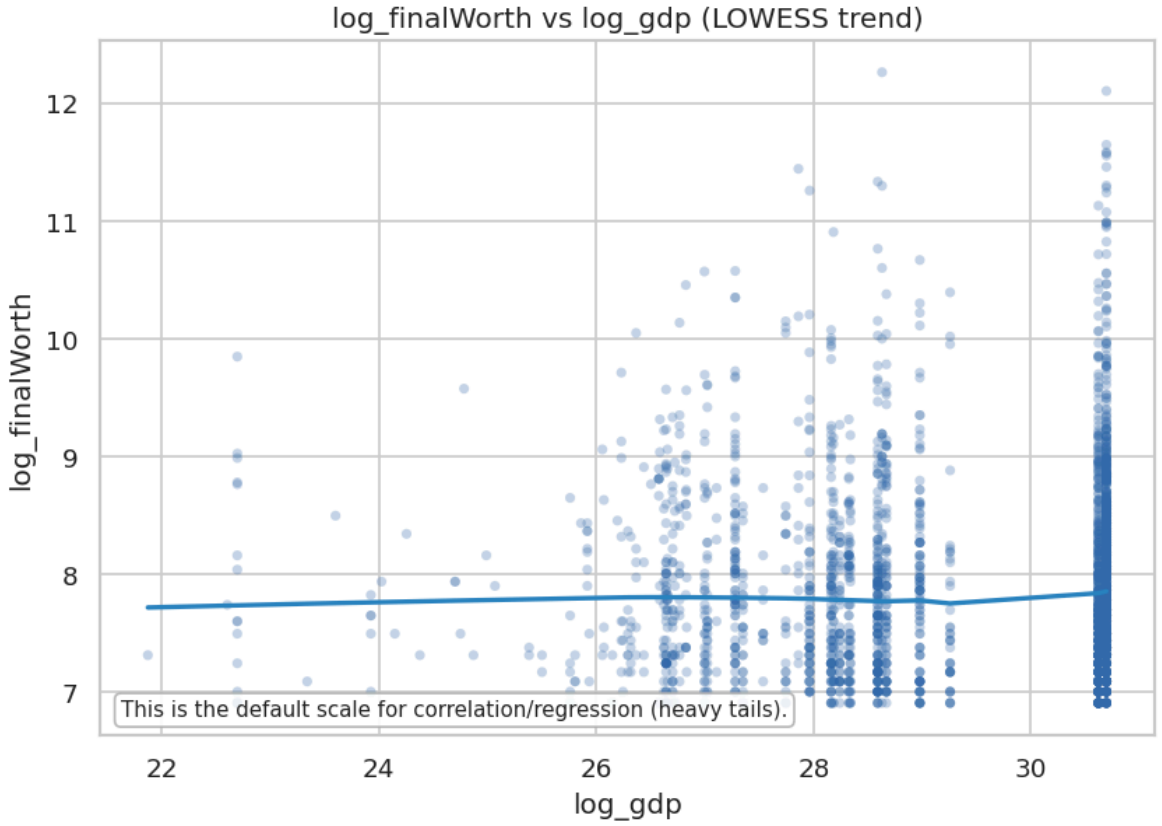
In [31]:

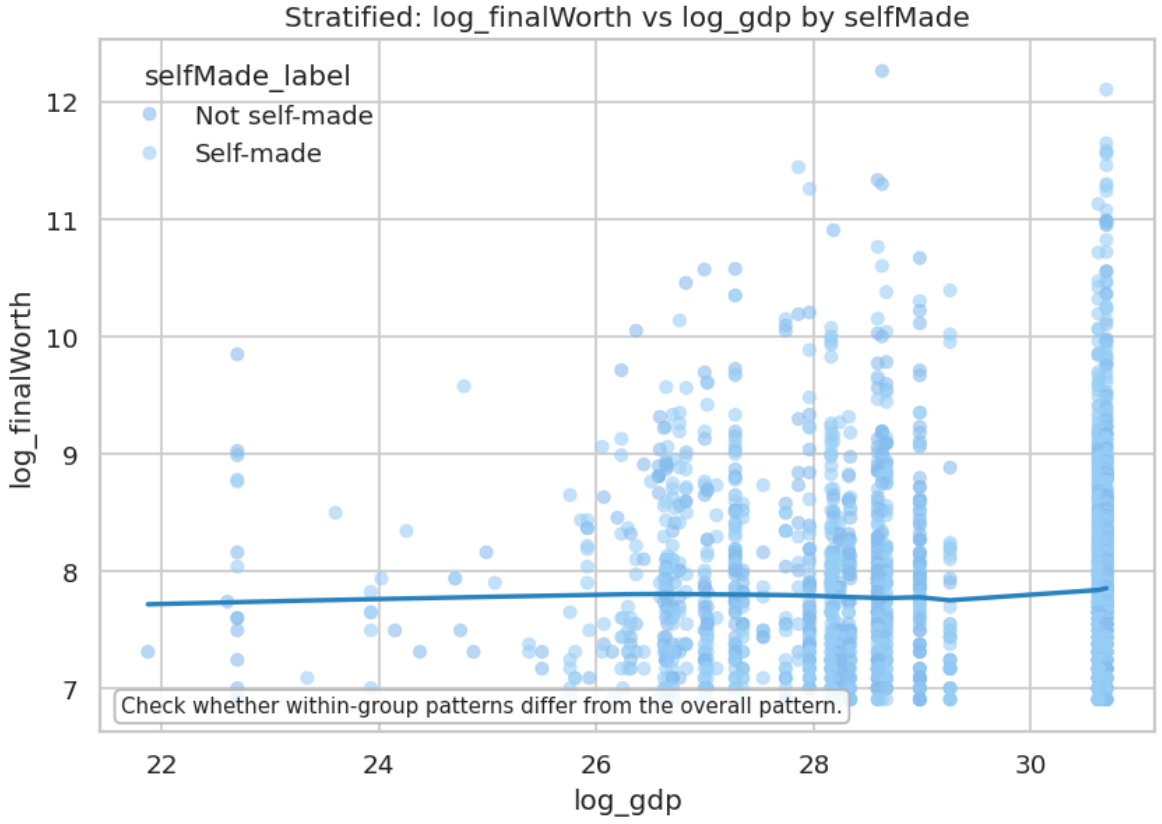
```
# ========================================
# M5.2 Bivariate & stratified EDA
# ========================================

# Boundary check: require df_clean
assert "df_clean" in globals(), "df_clean not found. Run Chapter 3 (M3) f

# Core variables
need_cols = ["finalWorth", "gdp_country_num", "log_finalWorth", "log_gdp"
available = [c for c in need_cols if c in df_clean.columns]
df_eda = df_clean[available].copy()

# Normalize selfMade to a compact label (for plots/tables)
if "selfMade" in df_eda.columns:
    df_eda["selfMade_bin"] = _coerce_boolish(df_eda["selfMade"])
    df_eda["selfMade_label"] = df_eda["selfMade_bin"].map({1: "Self-made"

# ---- 1) Scatter: finalWorth vs gdp (raw variables; log axes for readabi
if {"finalWorth","gdp_country_num"}.issubset(df_eda.columns):
    fig = scatter(
        df_eda, "gdp_country_num", "finalWorth",
        logx=True, logy=True,
        title="finalWorth vs gdp_country_num (raw variables; log axes)",
        note="Axes are log-scaled to make heavy-tail structure visible."
    )
    save_fig(fig, "fig_M5_scatter_finalWorth_vs_gdp_raw")

# ---- 2) Scatter: logWorth vs logGDP (preferred scale for inference/regr
if {"log_finalWorth","log_gdp"}.issubset(df_eda.columns):
    fig = scatter(
```

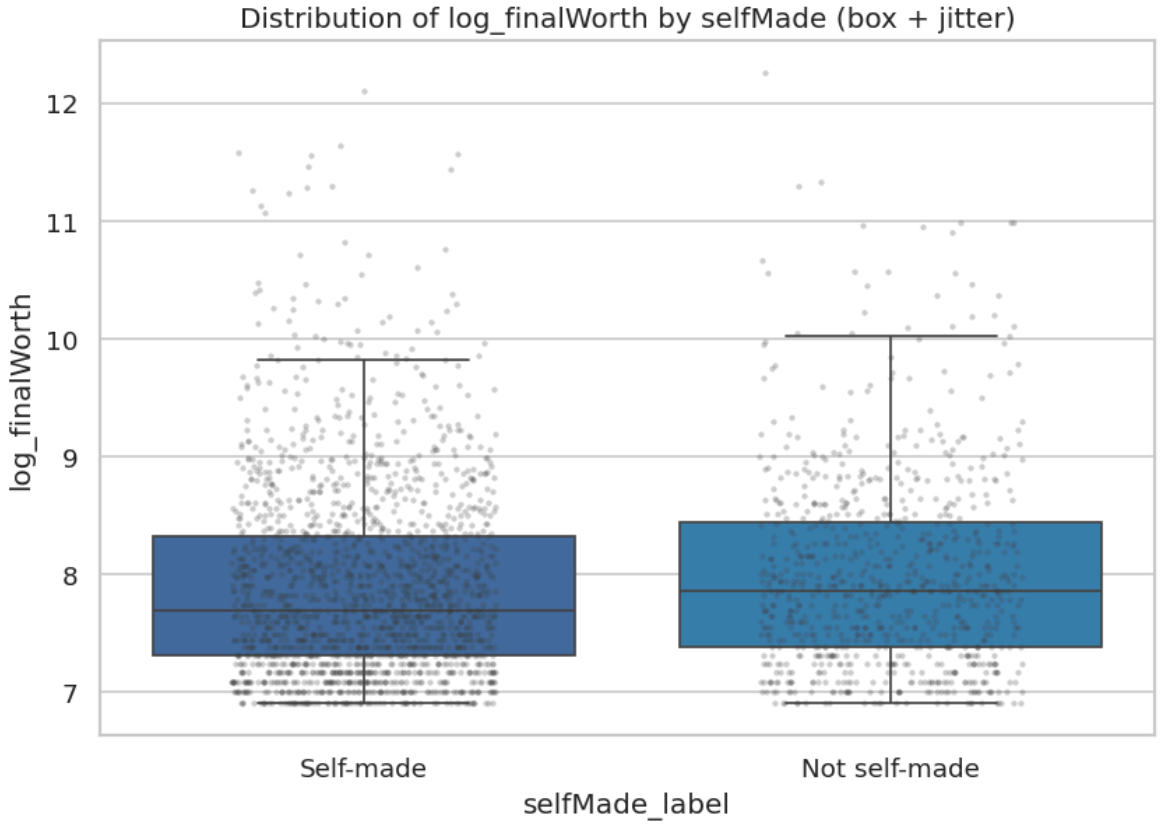
```
        df_eda, "log_gdp", "log_finalWorth",
        title="log_finalWorth vs log_gdp (LOWESS trend)",
        note="This is the default scale for correlation/regression (heavy
    )
    save_fig(fig, "fig_M5_scatter_logWorth_vs_logGDP")

# ---- 3) Stratified scatter (Simpson risk check): hue by selfMade
if {"log_finalWorth","log_gdp","selfMade_label"}.issubset(df_eda.columns)
    fig = scatter(
        df_eda, "log_gdp", "log_finalWorth",
        hue="selfMade_label",
        title="Stratified: log_finalWorth vs log_gdp by selfMade",
        note="Check whether within-group patterns differ from the overall
    )
    save_fig(fig, "fig_M5_scatter_logWorth_vs_logGDP_by_selfMade")

# ---- 4) Group contrasts (visual): boxplots
if {"log_finalWorth","selfMade_label"}.issubset(df_eda.columns):
    fig = boxplot(df_eda, "log_finalWorth", "selfMade_label",
                  title="Distribution of log_finalWorth by selfMade (box
    save_fig(fig, "fig_M5_box_logWorth_by_selfMade")

if {"log_finalWorth","gender"}.issubset(df_eda.columns):
    # Keep top levels only (typically 'M'/'F')
    top_g = df_eda["gender"].value_counts().head(4).index.tolist()
    dtmp = df_eda[df_eda["gender"].isin(top_g)].copy()
    fig = boxplot(dtmp, "log_finalWorth", "gender",
                  title="Distribution of log_finalWorth by gender (top le
                  order=top_g)
    save_fig(fig, "fig_M5_box_logWorth_by_gender")

# ---- 5) Spearman correlation heatmap (numeric features; auto-filter)
candidate_numeric = [
    "log_finalWorth", "log_gdp", "age",
    "cpi_country", "gini_country", "tax_revenue_country",
    "life_expectancy_country", "education_enrolment_tertiary",
    "gross_tertiary_education_enrollment"
]
fig_corr, corr = spearman_heatmap(df_clean, candidate_numeric, title="Spe
save_fig(fig_corr, "fig_M5_corr_heatmap_spearman")
save_table(corr.reset_index().rename(columns={"index":"variable"}), "tab_

# ---- 6) Group-wise correlations (overall vs stratified by selfMade/cate
rows = []
if {"log_finalWorth","log_gdp"}.issubset(df_clean.columns):
    d = df_clean[["log_finalWorth","log_gdp"]].dropna()
    rows.append({"slice": "Overall (complete cases)", "n": len(d),
                 "pearson_r": d["log_finalWorth"].corr(d["log_gdp"], meth
                 "spearman_r": d["log_finalWorth"].corr(d["log_gdp"], met

if {"log_finalWorth","log_gdp","selfMade"}.issubset(df_clean.columns):
    tmp = df_clean[["log_finalWorth","log_gdp","selfMade"]].dropna().copy
    tmp["selfMade_bin"] = _coerce_boolish(tmp["selfMade"])
    for v, lab in [(1,"Self-made"), (0,"Not self-made")]:
        dv = tmp[tmp["selfMade_bin"]==v][["log_finalWorth","log_gdp"]]
        if len(dv) >= 30:
            rows.append({"slice": f"selfMade={lab}", "n": len(dv),
                         "pearson_r": dv["log_finalWorth"].corr(dv["log_g
                         "spearman_r": dv["log_finalWorth"].corr(dv["log_
```

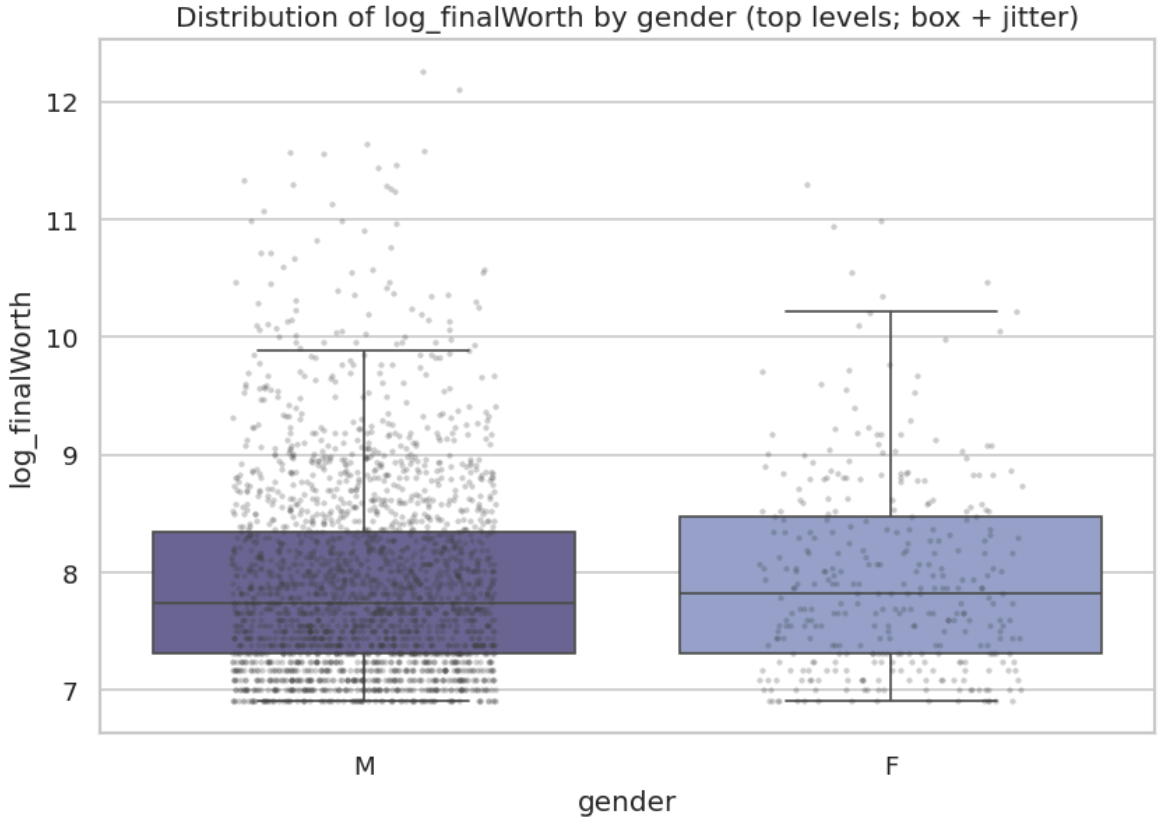
```
if {"log_finalWorth","log_gdp","category"}.issubset(df_clean.columns):
    tmp = df_clean[["log_finalWorth","log_gdp","category"]].dropna()
    top_cat = tmp["category"].value_counts().head(8).index
    for c in top_cat:
        dv = tmp[tmp["category"]==c][["log_finalWorth","log_gdp"]]
        if len(dv) >= 30:
            rows.append({"slice": f"category={c}", "n": len(dv),
                         "pearson_r": dv["log_finalWorth"].corr(dv["log_g
                         "spearman_r": dv["log_finalWorth"].corr(dv["log_

tab = pd.DataFrame(rows).sort_values(["slice"])
save_table(tab, "tab_M5_groupwise_correlations", index=False)
```

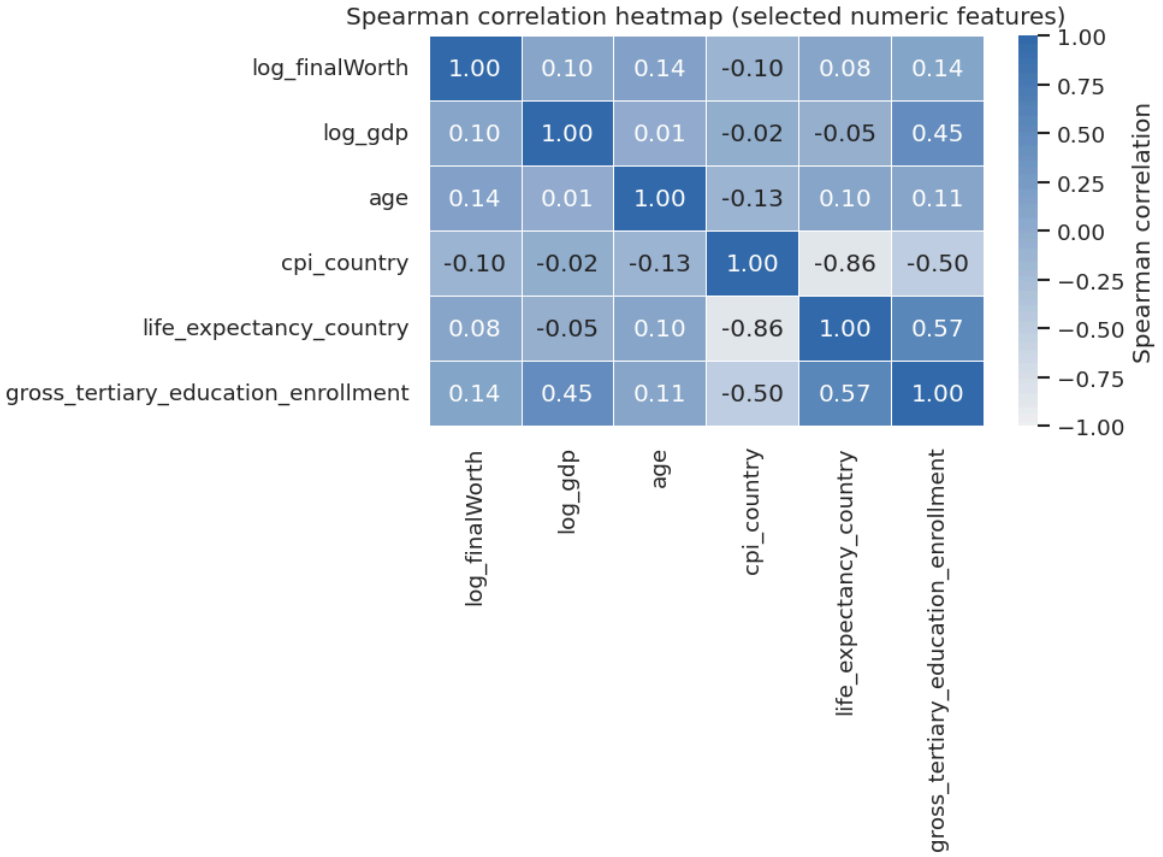



log_finalWorth vs log_gdp (LOWESS trend)
log_finalWorth
log_gdp
12
11
10
9
8
7
22
24
26
28
30
This is the default scale for correlation/regression (heavy tails).


Stratified: log_finalWorth vs log_gdp by selfMade
selfMade_label
Not self-made
Self-made
log_finalWorth
log_gdp
12
11
10
9
8
7
22
24
26
28
30
Check whether within-group patterns differ from the overall pattern.


Distribution of log_finalWorth by selfMade (box + jitter)
log_finalWorth
12
11
10
9
8
7
Self-made
Not self-made
selfMade_label


Distribution of log_finalWorth by gender (top levels; box + jitter)
log_finalWorth
12
11
10
9
8
7
M
F
gender



	variable	log_finalWorth	log_gdp	age	cpi_c
0	log_finalWorth	1.000000	0.096573	0.144039	-0
1	log_gdp	0.096573	1.000000	0.005440	-0
2	age	0.144039	0.005440	1.000000	-0
3	cpi_country	-0.104249	-0.021747	-0.125774	1
4	life_expectancy_country	0.084260	-0.048628	0.100373	-0
5	gross_tertiary_education_enrollment	0.137406	0.452886	0.105191	-0

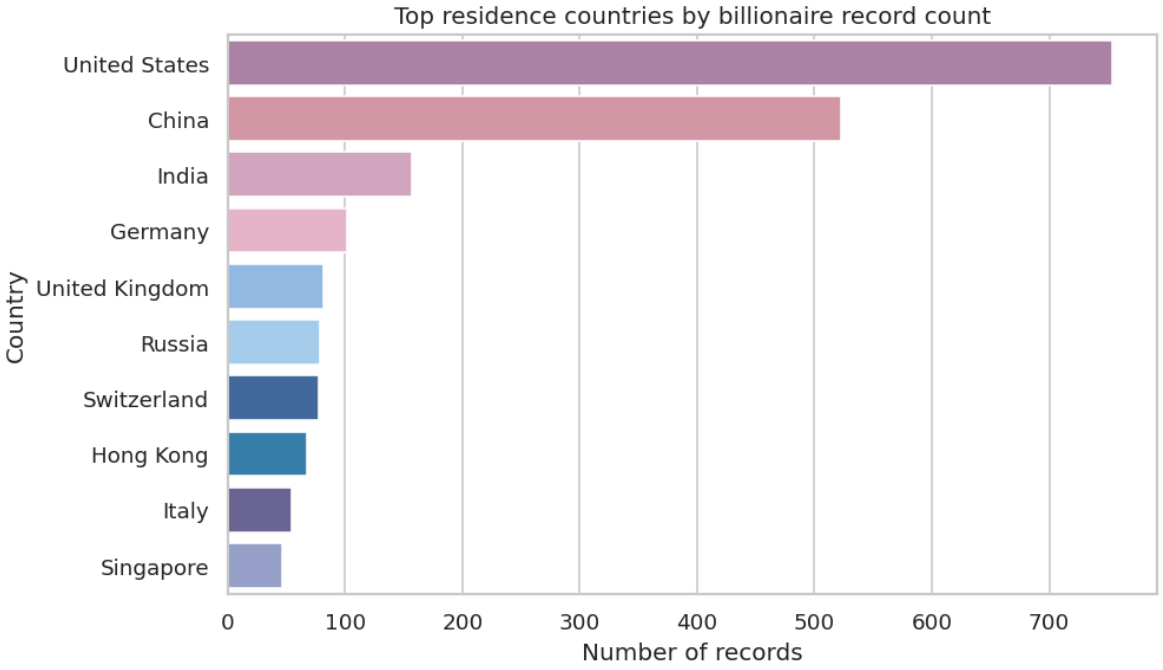
	slice	n	pearson_r	spearman_r
0	Overall (complete cases)	2476	0.045759	0.096573
9	category=Diversified	182	0.071743	0.119415
6	category=Fashion & Retail	252	0.064775	0.088260
3	category=Finance & Investments	345	0.136928	0.178481
7	category=Food & Beverage	201	-0.036885	0.049971
8	category=Healthcare	197	-0.069842	-0.069674
5	category=Manufacturing	298	-0.150761	-0.069781
10	category=Real Estate	162	0.217277	0.290688
4	category=Technology	299	0.090093	0.093944
2	selfMade=Not self-made	755	0.125791	0.150643
1	selfMade=Self-made	1721	0.032309	0.095057

Out[31]:
```
{'csv': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M5_groupwise_corr
elations.csv',
 'tex': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M5_groupwise_corr
elations.tex'}
```

In [32]:
```
# FULL_INTEGRATION_PASS_2026_05_01
# ========================================
# M5.3 Geography drill-down restored from V2: country, US state/region, C
# ========================================
# These are descriptive profiles only. They restore the V2 geography reas

geo = df_clean.copy()
geo['fw'] = pd.to_numeric(geo['finalWorth'], errors='coerce')
geo['logW'] = pd.to_numeric(geo['log_finalWorth'], errors='coerce')

country_profile = (geo.dropna(subset=['country'])
    .groupby('country')
    .agg(n=('country','size'), total_wealth=('fw','sum'), median_wealth=(
    .sort_values('n', ascending=False)
    .reset_index())
total_n = len(geo)
total_fw = geo['fw'].sum()
country_profile['share_records'] = country_profile['n'] / total_n
country_profile['share_wealth'] = country_profile['total_wealth'] / total
country_top10 = country_profile.head(10).copy()
save_table(country_top10, 'tab_M5_geo_country_concentration', index=False

fig, ax = plt.subplots(figsize=(8.5, 5.0))
sns.barplot(data=country_top10, y='country', x='n', hue='country', palett
ax.set_title('Top residence countries by billionaire record count')
ax.set_xlabel('Number of records')
ax.set_ylabel('Country')
save_fig(fig, 'fig_M5_geo_top10_country_records')

if {'country','countryOfCitizenship'}.issubset(geo.columns):
    mismatch = geo[geo['country'].notna() & geo['countryOfCitizenship'].n
    mismatch['country_citizenship_mismatch'] = mismatch['country'].astype
    mismatch_summary = pd.DataFrame([{
        'complete_cases': int(len(mismatch)),
        'mismatch_n': int(mismatch['country_citizenship_mismatch'].sum())
        'mismatch_rate': float(mismatch['country_citizenship_mismatch'].m
    }])
    save_table(mismatch_summary, 'tab_M5_geo_country_citizenship_mismatch

us = geo[geo['country'].eq('United States')].copy()
if not us.empty:
    us_state = (us.dropna(subset=['state'])
        .groupby('state')
        .agg(n=('state','size'), total_wealth=('fw','sum'), median_wealth
        .sort_values('n', ascending=False)
        .reset_index())
    save_table(us_state.head(15), 'tab_M5_geo_us_state_profile', index=Fa

    fig, ax = plt.subplots(figsize=(8.5, 5.2))
    top_state = us_state.head(12)
    sns.barplot(data=top_state, y='state', x='n', hue='state', palette=di
    ax.set_title('US billionaire records by state (top 12)')
    ax.set_xlabel('Number of US records')
```

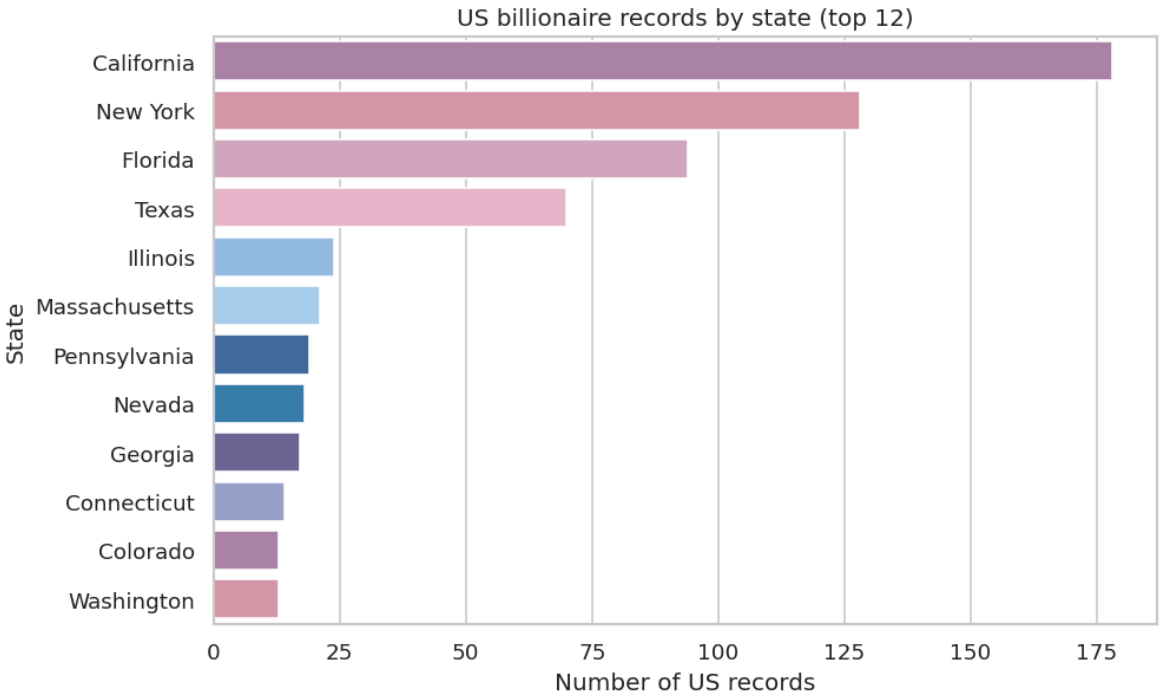
```
    ax.set_ylabel('State')
    save_fig(fig, 'fig_M5_geo_us_state_top12')

    if 'residenceStateRegion' in us.columns:
        us_region = (us.dropna(subset=['residenceStateRegion'])
            .groupby('residenceStateRegion')
            .agg(n=('residenceStateRegion','size'), total_wealth=('fw','s
            .sort_values('n', ascending=False)
            .reset_index())
        save_table(us_region, 'tab_M5_geo_us_region_profile', index=False

china = geo[geo['country'].eq('China')].copy()
if not china.empty:
    china_city = (china.dropna(subset=['city'])
        .groupby('city')
        .agg(n=('city','size'), total_wealth=('fw','sum'), median_wealth=
        .sort_values('n', ascending=False)
        .reset_index())
    save_table(china_city.head(15), 'tab_M5_geo_china_city_profile', inde

    fig, ax = plt.subplots(figsize=(8.5, 5.2))
    top_city = china_city.head(12)
    sns.barplot(data=top_city, y='city', x='n', hue='city', palette=dict(
    ax.set_title('China billionaire records by city (top 12)')
    ax.set_xlabel('Number of China records')
    ax.set_ylabel('City')
    save_fig(fig, 'fig_M5_geo_china_city_top12')
```

	country	n	total_wealth	median_wealth	mean_logW	share_records
0	United States	754	4575100	2900.0	8.122154	0.285606
1	China	523	1805500	1900.0	7.749033	0.198106
2	India	157	628700	2100.0	7.815361	0.059470
3	Germany	102	462100	2650.0	7.973694	0.038636
4	United Kingdom	82	370700	2600.0	8.006329	0.031061
5	Russia	79	351000	2500.0	7.946050	0.029924
6	Switzerland	78	409900	3100.0	8.165148	0.029545
7	Hong Kong	68	321500	2700.0	8.057742	0.025758
8	Italy	55	156900	2300.0	7.780268	0.020833
9	Singapore	46	138000	2150.0	7.748249	0.017424



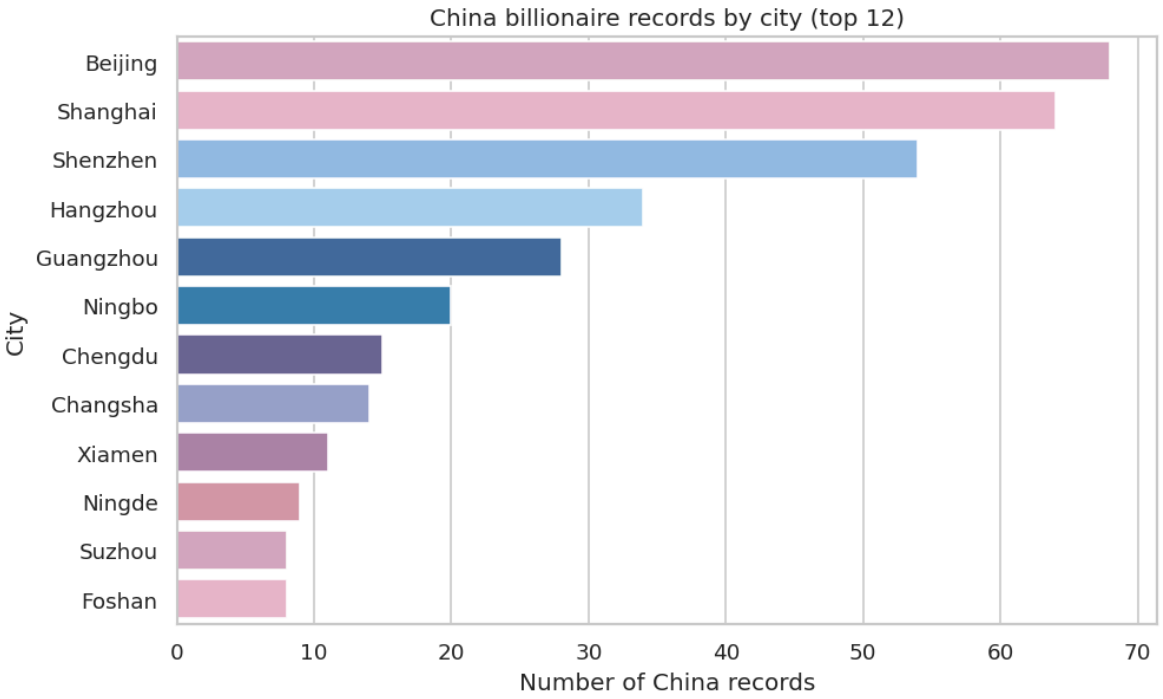
	complete_cases	mismatch_n	mismatch_rate
0	2602	278	0.106841

	state	n	total_wealth	median_wealth	selfmade_rate
0	California	178	882800	2550.0	0.853933
1	New York	128	731800	3050.0	0.718750
2	Florida	94	382200	2550.0	0.723404
3	Texas	70	576500	3800.0	0.671429
4	Illinois	24	102500	3700.0	0.750000
5	Massachusetts	21	86400	2500.0	0.714286
6	Pennsylvania	19	91600	2700.0	0.578947
7	Nevada	18	89000	2500.0	0.722222
8	Georgia	17	74500	4000.0	0.588235
9	Connecticut	14	77900	3650.0	0.785714
10	Colorado	13	41500	1700.0	0.538462
11	Washington	13	350300	3900.0	0.769231
12	Arizona	12	32700	2800.0	0.666667
13	Tennessee	12	48900	2500.0	0.583333
14	Maryland	10	34700	3600.0	0.800000



	residenceStateRegion	n	total_wealth	median_wealth	selfmade_rate
0	West	248	1584200	2500.0	0.810484
1	South	241	1480400	3100.0	0.659751
2	Northeast	190	1024700	2900.0	0.705263
3	Midwest	67	461400	3800.0	0.567164
4	U.S. Territories	1	3000	3000.0	1.000000

	city	n	total_wealth	median_wealth	selfmade_rate
0	Beijing	68	247200	2100.0	0.970588
1	Shanghai	64	179700	1700.0	0.953125
2	Shenzhen	54	246000	1900.0	0.981481
3	Hangzhou	34	213200	2050.0	0.970588
4	Guangzhou	28	76400	1550.0	1.000000
5	Ningbo	20	68600	2100.0	1.000000
6	Chengdu	15	53200	2700.0	1.000000
7	Changsha	14	40200	1600.0	1.000000
8	Xiamen	11	18700	1300.0	1.000000
9	Ningde	9	73200	2200.0	1.000000
10	Suzhou	8	20900	2100.0	1.000000
11	Foshan	8	59800	3750.0	0.750000
12	Nanjing	7	11200	1400.0	0.857143
13	Quanzhou	7	29000	2200.0	0.857143
14	Hefei	7	16600	1500.0	1.000000



RQ5 Mini-Summary: Geography Restored, but Kept Descriptive

The restored geography block keeps V2's broader storytelling while tightening its claims. The country profile shows concentration by residence country; the US drill-down is valid because `state` and `residenceStateRegion` are structurally US-specific; the China drill-down uses city counts because the dataset does not contain reliable province polygons or city coordinates for causal mapping.

Interpretation boundary. These outputs show where billionaire records are concentrated. They do not prove that a state, city, or country policy caused billionaire formation. Geography is treated as composition, clustering, and context.

M5 Mini-Summary: Weak Pooled Association, Stronger Heterogeneity

The pooled wealth-GDP relationship is positive but weak: Pearson r is about 0.046 and Spearman rho is about 0.097 on complete cases. Stratified correlations vary by industry; Real Estate shows a stronger positive Spearman association around 0.291, while Manufacturing is negative on Pearson and near zero on Spearman. This is why the report treats GDP as a contextual association rather than a universal mechanism.

The self-made and gender comparisons are descriptive signals, not social-causal proof. They are useful because they reveal sample composition and potential confounding, but they require cautious language.

Macro-Variable Interpretation Boundary: Scale, Development, and Institution Are Not the Same

The V2 draft framed the macro variables around a useful conceptual warning: GDP is not the same as national development. In the final analysis this becomes a modeling boundary. `log_gdp` captures economic scale; `log_pop` helps separate scale from population size; `log_gdp_pc`, education, and life expectancy are closer to development proxies; CPI and tax variables describe institutional or cost environments. Because these variables are country-level, they should be interpreted as context and association, not individual-level causal mechanisms.

Chapter 6 - Inferential Statistics and Multiple-Comparison Discipline (M6)

This chapter upgrades the old exploratory tests into effect-size-first inference: correlations with confidence intervals, group differences with robust effect sizes, permutation logic, and category comparisons with multiplicity control.

In [33]:
```
# ======================================
# M6.1 Inferential helpers (effect sizes + CI)
# ======================================

RNG = np.random.default_rng(42)

def bootstrap_ci(stat_fn, data, *, n_boot: int = 2000, alpha: float = 0.0
    """Generic bootstrap CI (percentile) for a statistic."""
    stats_ = []
    n = len(data)
    for _ in range(n_boot):
        idx = RNG.integers(0, n, size=n)
        stats_.append(stat_fn(data[idx]))
    lo, hi = np.quantile(stats_, [alpha/2, 1-alpha/2])
    return float(lo), float(hi)

def bootstrap_ci_2groups(stat_fn, x, y, *, n_boot: int = 2000, alpha: flo
    stats_ = []
    x = np.asarray(x); y = np.asarray(y)
    nx, ny = len(x), len(y)
    for _ in range(n_boot):
        xb = x[RNG.integers(0, nx, size=nx)]
        yb = y[RNG.integers(0, ny, size=ny)]
        stats_.append(stat_fn(xb, yb))
    lo, hi = np.quantile(stats_, [alpha/2, 1-alpha/2])
    return float(lo), float(hi)

def cohens_d(x, y) -> float:
    x = np.asarray(x); y = np.asarray(y)
    nx, ny = len(x), len(y)
    vx, vy = x.var(ddof=1), y.var(ddof=1)
    pooled = ((nx-1)*vx + (ny-1)*vy) / (nx + ny - 2)
```

```
    return float((x.mean() - y.mean()) / np.sqrt(pooled))

def cliffs_delta(x, y) -> float:
    """Cliff's delta in [-1,1], robust to non-normality."""
    x = np.asarray(x); y = np.asarray(y)
    # O(nm) exact; acceptable at typical sample sizes; for large, conside
    greater = sum((xi > y).sum() for xi in x)
    less = sum((xi < y).sum() for xi in x)
    return float((greater - less) / (len(x)*len(y)))

def perm_test_diff_means(x, y, *, n_perm: int = 2000) -> float:
    """Two-sided permutation test for difference in means."""
    x = np.asarray(x); y = np.asarray(y)
    obs = x.mean() - y.mean()
    pooled = np.concatenate([x, y])
    nx = len(x)
    more_extreme = 0
    for _ in range(n_perm):
        RNG.shuffle(pooled)
        diff = pooled[:nx].mean() - pooled[nx:].mean()
        if abs(diff) >= abs(obs):
            more_extreme += 1
    return (more_extreme + 1) / (n_perm + 1)

def corr_bootstrap_ci(x, y, method: str = "spearman", *, n_boot: int = 20
    x = np.asarray(x); y = np.asarray(y)
    n = len(x)
    def stat(idx):
        xx = x[idx]; yy = y[idx]
        if method == "pearson":
            return np.corrcoef(xx, yy)[0,1]
        else:
            return stats.spearmanr(xx, yy, nan_policy="omit").correlation
    boots = []
    for _ in range(n_boot):
        idx = RNG.integers(0, n, size=n)
        boots.append(stat(idx))
    lo, hi = np.quantile(boots, [alpha/2, 1-alpha/2])
    return float(lo), float(hi)
```

In [34]:

```
# ========================================
# M6.2 Correlation + hypothesis tests (effect size + CI)
# ========================================

assert "df_clean" in globals(), "df_clean not found. Run M3 first."

results = []

# ---- A) Correlation: log_finalWorth vs log_gdp (Pearson + Spearman + bo
if {"log_finalWorth","log_gdp"}.issubset(df_clean.columns):
    d = df_clean[["log_finalWorth","log_gdp"]].dropna()
    x = d["log_gdp"].to_numpy()
    y = d["log_finalWorth"].to_numpy()

    pear = np.corrcoef(x, y)[0,1]
    spea = stats.spearmanr(x, y, nan_policy="omit").correlation
    pear_ci = corr_bootstrap_ci(x, y, method="pearson")
    spea_ci = corr_bootstrap_ci(x, y, method="spearman")

    results += [
```

```
        {"family": "Correlation", "analysis": "Pearson r (logWorth, logGD
         "effect": pear, "ci_low": pear_ci[0], "ci_high": pear_ci[1], "p_
        {"family": "Correlation", "analysis": "Spearman rho (logWorth, lo
         "effect": spea, "ci_low": spea_ci[0], "ci_high": spea_ci[1], "p_
    ]

# ---- B) Two-group test: selfMade vs not (log_finalWorth)
if {"log_finalWorth","selfMade"}.issubset(df_clean.columns):
    tmp = df_clean[["log_finalWorth","selfMade"]].dropna().copy()
    tmp["selfMade_bin"] = _coerce_boolish(tmp["selfMade"])
    g1 = tmp[tmp["selfMade_bin"]==1]["log_finalWorth"].to_numpy()
    g0 = tmp[tmp["selfMade_bin"]==0]["log_finalWorth"].to_numpy()

    if len(g1) >= 30 and len(g0) >= 30:
        # Welch t-test
        t = stats.ttest_ind(g1, g0, equal_var=False)
        d_eff = cohens_d(g1, g0)
        d_ci = bootstrap_ci_2groups(cohens_d, g1, g0)
        mean_diff = float(g1.mean() - g0.mean())
        mean_ci = bootstrap_ci_2groups(lambda a,b: a.mean()-b.mean(), g1,

        # Mann–Whitney U (nonparam) + Cliff's delta
        u = stats.mannwhitneyu(g1, g0, alternative="two-sided")
        cd = cliffs_delta(g1, g0)
        cd_ci = bootstrap_ci_2groups(cliffs_delta, g1, g0)

        # Permutation test as robustness check
        p_perm = perm_test_diff_means(g1, g0)

        results += [
            {"family": "Two-group", "analysis": "Welch t-test: selfMade v
             "effect": mean_diff, "ci_low": mean_ci[0], "ci_high": mean_c
            {"family": "Two-group", "analysis": "Cohen's d: selfMade vs n
             "effect": d_eff, "ci_low": d_ci[0], "ci_high": d_ci[1], "p_v
            {"family": "Two-group", "analysis": "Cliff's delta: selfMade
             "effect": cd, "ci_low": cd_ci[0], "ci_high": cd_ci[1], "p_va
            {"family": "Robustness", "analysis": "Permutation p-value (me
             "effect": np.nan, "ci_low": np.nan, "ci_high": np.nan, "p_va
        ]

# ---- C) Multi-group test: category differences (log_finalWorth) via Kru
if {"log_finalWorth","category"}.issubset(df_clean.columns):
    tmp = df_clean[["log_finalWorth","category"]].dropna()
    # Use categories with sufficient sample size
    counts = tmp["category"].value_counts()
    use_cats = counts[counts >= 30].index.tolist()
    groups = [tmp[tmp["category"]==c]["log_finalWorth"].to_numpy() for c
    if len(groups) >= 3:
        kw = stats.kruskal(*groups)
        n = sum(len(g) for g in groups); k = len(groups)
        # epsilon-squared for Kruskal-Wallis
        eps2 = (kw.statistic - k + 1) / (n - k)
        results.append({"family":"Multi-group", "analysis":"Kruskal-Walli
                        "effect": float(eps2), "ci_low": np.nan, "ci_high

        # Post-hoc (top categories): pairwise Mann–Whitney with BH-FDR
        top_cats = counts.head(8).index.tolist()
        pairs = []
        pvals = []
        for i in range(len(top_cats)):
```

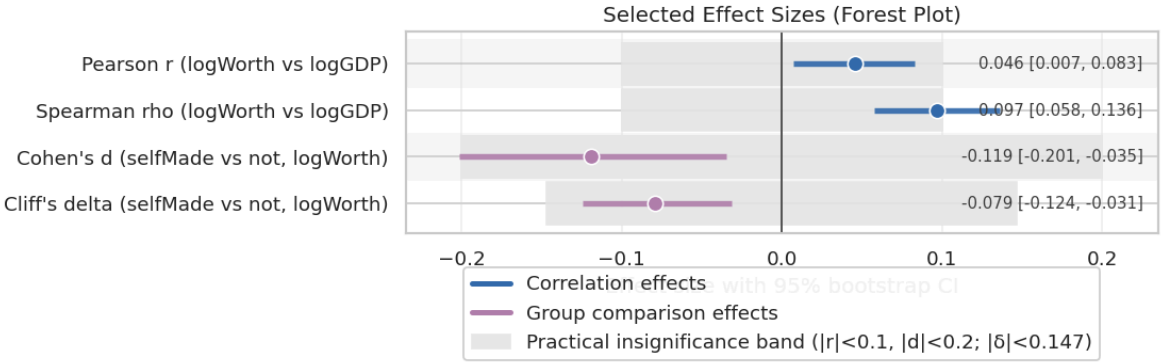
```
            for j in range(i+1, len(top_cats)):
                a = tmp[tmp["category"]==top_cats[i]]["log_finalWorth"].t
                b = tmp[tmp["category"]==top_cats[j]]["log_finalWorth"].t
                if len(a)>=30 and len(b)>=30:
                    u = stats.mannwhitneyu(a,b,alternative="two-sided")
                    cd = cliffs_delta(a,b)
                    cd_ci = bootstrap_ci_2groups(cliffs_delta, a, b, n_bo
                    pairs.append({
                        "group_a": top_cats[i], "group_b": top_cats[j],
                        "n_a": len(a), "n_b": len(b),
                        "cliffs_delta": cd, "ci_low": cd_ci[0], "ci_high"
                        "p_raw": u.pvalue,
                    })
                    pvals.append(u.pvalue)
        if pairs:
            rej, p_adj, _, _ = multipletests(pvals, alpha=0.05, method="f
            for row, padj, rj in zip(pairs, p_adj, rej):
                row["p_fdr_bh"] = padj
                row["reject_fdr05"] = bool(rj)
            tab_pairs = pd.DataFrame(pairs).sort_values("p_fdr_bh")
            save_table(tab_pairs, "tab_M6_posthoc_category_pairwise", ind

# ---- Compile inferential results table
tab_res = pd.DataFrame(results)
# Formatting: keep raw numeric; formatting can be done in LaTeX/report la
save_table(tab_res, "tab_M6_inferential_results", index=False)

# ---- Effect-size forest plot (selected headline effects) [Add practical
forest_rows = []
for _, r in tab_res.iterrows():
    s = str(r["analysis"])
    if s.startswith("Spearman") or s.startswith("Pearson") or s.startswit
        forest_rows.append(r)

forest = pd.DataFrame(forest_rows).dropna(subset=["effect"]).copy()

if len(forest) > 0:
    # Cleaner labels + grouping
    def _short_label(s: str) -> str:
        if s.startswith("Pearson"):
            return "Pearson r (logWorth vs logGDP)"
        if s.startswith("Spearman"):
            return "Spearman rho (logWorth vs logGDP)"
        if s.startswith("Cohen"):
            return "Cohen's d (selfMade vs not, logWorth)"
        if s.startswith("Cliff"):
            return "Cliff's delta (selfMade vs not, logWorth)"
        return s

    forest["label"] = forest["analysis"].astype(str).map(_short_label)
    forest["group"] = np.where(
        forest["analysis"].astype(str).str.startswith(("Pearson", "Spearm
        "Correlation",
        "Group comparison"
    )

    # Practical insignificance thresholds by effect type
    # Requested: |r|<0.1, |d|<0.2. For Cliff's delta, use a common neglig
    def _practical_thr(s: str) -> float:
        if s.startswith(("Pearson", "Spearman")):
```

```
            return 0.10
        if s.startswith("Cohen"):
            return 0.20
        if s.startswith("Cliff"):
            return 0.147  # optional; remove if you don't want a band for
        return np.nan

    forest["thr"] = forest["analysis"].astype(str).map(_practical_thr)

    # Order: Correlation first, then Group comparison
    order = []
    for g in ["Correlation", "Group comparison"]:
        order += forest.loc[forest["group"] == g, "label"].tolist()
    seen = set()
    order = [x for x in order if not (x in seen or seen.add(x))]

    forest["label"] = pd.Categorical(forest["label"], categories=order, o
    forest = forest.sort_values(["group", "label"]).reset_index(drop=True

    # Determine x-limits with padding (symmetric around 0 looks nicer for
    x_min = np.nanmin(forest["ci_low"].to_numpy())
    x_max = np.nanmax(forest["ci_high"].to_numpy())
    pad = 0.10 * (x_max - x_min + 1e-9)
    lim = max(abs(x_min - pad), abs(x_max + pad))
    xlim = (-lim, lim)

    # Colors per group (avoid all-pink)
    group_color = {
        "Correlation": COLOR.get("primary", "#1f77b4"),
        "Group comparison": COLOR.get("accent", "#ff7f0e"),
    }

    n_rows = len(forest)
    fig_h = max(3.2, 0.62 * n_rows)
    fig, ax = plt.subplots(figsize=(9.4, fig_h))
    y = np.arange(n_rows)[::-1]

    # Alternating row shading (subtle)
    for i in range(n_rows):
        if i % 2 == 0:
            ax.axhspan(y[i] - 0.5, y[i] + 0.5, color="0.965", zorder=0)

    # Practical insignificance band per row (row-specific threshold)
    # Draw a light gray band centered at 0 with width depending on effect
    for i, row in forest.iterrows():
        thr = row["thr"]
        if np.isfinite(thr) and thr > 0:
            yi = y[i]
            ax.fill_betweenx([yi - 0.45, yi + 0.45], -thr, thr,
                             color="0.90", alpha=0.9, zorder=1)

    # CI lines + points + numeric annotation
    for i, row in forest.iterrows():
        yi = y[i]
        c = group_color.get(row["group"], COLOR.get("primary", "#1f77b4")

        ax.hlines(yi, row["ci_low"], row["ci_high"], lw=3.2, color=c, alp
        ax.scatter(row["effect"], yi, s=75, color=c, edgecolor="white", l

        txt = f'{row["effect"]:.3f} [{row["ci_low"]:.3f}, {row["ci_high"]
```

```
        ax.text(xlim[1] - 0.02 * (xlim[1] - xlim[0]), yi, txt,
                va="center", ha="right", fontsize=9, color="0.25", zorder

    ax.axvline(0, color="0.35", lw=1.2, zorder=2)
    ax.set_xlim(*xlim)

    ax.set_yticks(y)
    ax.set_yticklabels(forest["label"].astype(str))
    ax.set_xlabel("Effect size with 95% bootstrap CI")
    ax.set_title("Selected Effect Sizes (Forest Plot)")

    # Legend (group colors + practical band note)
    corr_h = plt.Line2D([0], [0], color=group_color["Correlation"], lw=3,
    grp_h = plt.Line2D([0], [0], color=group_color["Group comparison"], l
    band_h = plt.Rectangle((0, 0), 1, 1, color="0.90", alpha=0.9,
                           label="Practical insignificance band (|r|<0.1,
    # Legend: move outside (below) to avoid covering rows
ax.legend(handles=[corr_h, grp_h, band_h],
          loc="upper center",
          bbox_to_anchor=(0.5, -0.12),   # push legend below axes
          ncol=1,                        # set 2 if you want it more comp
          frameon=True,
          framealpha=0.95)

ax.grid(axis="x", alpha=0.25)

# Leave space at bottom for the legend
plt.tight_layout()
fig.subplots_adjust(bottom=0.22)

save_fig(fig, "fig_M6_effectsize_forest")
```

	group_a	group_b	n_a	n_b	cliffs_delta	ci_low	ci_high	
0	Finance & Investments	Manufacturing	372	324	0.176142	0.098139	0.257358	0.
8	Manufacturing	Fashion & Retail	324	266	-0.188689	-0.285856	-0.102522	0.
4	Finance & Investments	Healthcare	372	201	0.159324	0.061792	0.249185	0.
19	Fashion & Retail	Healthcare	266	201	0.170258	0.068434	0.262275	0.
9	Manufacturing	Food & Beverage	324	212	-0.153739	-0.248842	-0.056910	0.
7	Manufacturing	Technology	324	314	-0.132932	-0.218992	-0.044692	0.
12	Manufacturing	Diversified	324	187	-0.133706	-0.220579	-0.032776	0.
22	Food & Beverage	Healthcare	212	201	0.133695	0.012023	0.248895	0.
11	Manufacturing	Real Estate	324	193	-0.119715	-0.213322	-0.025409	0.
15	Technology	Healthcare	314	201	0.112495	0.014607	0.205279	0.
26	Healthcare	Diversified	201	187	-0.113257	-0.220089	0.001578	0.
25	Healthcare	Real Estate	201	193	-0.102286	-0.206742	-0.006671	0.
20	Fashion & Retail	Real Estate	266	193	0.084986	-0.016207	0.206893	0.
5	Finance & Investments	Real Estate	372	193	0.069670	-0.017324	0.175057	0.
21	Fashion & Retail	Diversified	266	187	0.068413	-0.038393	0.172569	0.
13	Technology	Fashion & Retail	314	266	-0.056834	-0.152412	0.034435	0.
6	Finance & Investments	Diversified	372	187	0.052153	-0.044090	0.151306	0.
1	Finance & Investments	Technology	372	314	0.040768	-0.046936	0.134295	0.
23	Food & Beverage	Real Estate	212	193	0.048001	-0.061929	0.166129	0.
18	Fashion & Retail	Food & Beverage	266	212	0.034650	-0.054118	0.143483	0.

	family	analysis	n	effect	ci_low	ci_high	p_value
0	Correlation	Pearson r (logWorth, logGDP)	2476	0.045759	0.007066	0.082935	0.022786
1	Correlation	Spearman rho (logWorth, logGDP)	2476	0.096573	0.057546	0.136319	0.000001
2	Two-group	Welch t-test: selfMade vs not (logWorth)	2640	-0.097502	-0.168643	-0.031732	0.004980
3	Two-group	Cohen's d: selfMade vs not (logWorth)	2640	-0.118971	-0.201232	-0.034527	0.004980
4	Two-group	Cliff's delta: selfMade vs not (logWorth)	2640	-0.079194	-0.124112	-0.031179	0.001072
5	Robustness	Permutation p-value (mean diff, logWorth)	2640	NaN	NaN	NaN	0.004998
6	Multi-group	Kruskal-Wallis epsilon-squared: logWorth acros...	2615	0.008916	NaN	NaN	0.001030



Out[34]:
```
{'pdf': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M6_effectsize_for
est.pdf',
 'png': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M6_effectsize_for
est.png'}
```

M6 Mini-Summary: Statistical Significance Is Not Practical Strength

The GDP-wealth association is statistically detectable but substantively small: Spearman rho is 0.0966 with p near 1e-6. Self-made billionaires differ from non-self-made billionaires on log wealth with Cohen's d about -0.119 and

Cliff's delta about -0.079, which is stable but small. Category-level heterogeneity is statistically present, but pairwise results need multiplicity control; after FDR adjustment, several manufacturing-related contrasts remain significant.

Interpretation boundary. The report therefore avoids overstating p-values. It reports effect sizes, uncertainty, and robustness rather than treating significance as the conclusion.

Chapter 7 - Regression, Prediction, and Model Diagnostics (M7)

This chapter integrates the old scale-vs-development regression idea with the current stronger model checks. The central question is not whether GDP can produce a small p-value, but whether the GDP coefficient survives scale controls, industry composition, outliers, multicollinearity, and out-of-sample validation.

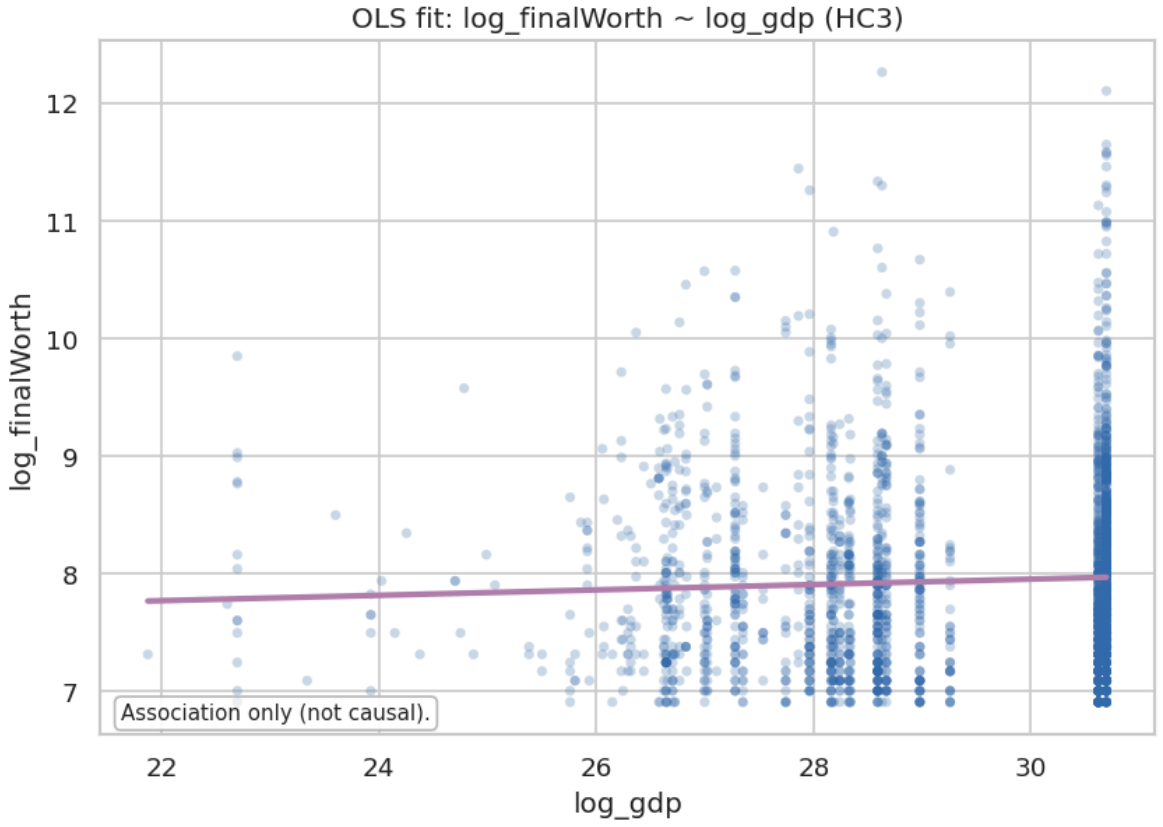
In [35]:
```
# ======================================
# M7.1 Regression models (OLS + robust SE + diagnostics)
# ======================================

# Re-import with safe aliases to avoid namespace collisions (e.g., `sm` o
import statsmodels.api as sm_api
import statsmodels.formula.api as smf_api
from statsmodels.stats.diagnostic import het_breuschpagan

assert "df_clean" in globals(), "df_clean not found. Run M3 first."

reg_rows = []

def _ci_df(res, alpha=0.05):
    """Return confidence intervals as a DataFrame with index=term names."
    ci = res.conf_int(alpha=alpha)
    terms = list(res.model.exog_names)
    if isinstance(ci, np.ndarray):
        return pd.DataFrame(ci, index=terms, columns=["ci_low", "ci_high"
    ci_df = ci.copy()
    if ci_df.shape[1] == 2:
        ci_df.columns = ["ci_low", "ci_high"]
    if len(ci_df.index) != len(terms):
        ci_df.index = terms
    return ci_df

def _result_table(model_name, res_hc3, n, r2):
    terms = list(res_hc3.model.exog_names)
    params = pd.Series(res_hc3.params, index=terms, dtype=float)
    bse = pd.Series(res_hc3.bse, index=terms, dtype=float)
    tvals = pd.Series(res_hc3.tvalues, index=terms, dtype=float)
    pvals = pd.Series(res_hc3.pvalues, index=terms, dtype=float)
    ci = _ci_df(res_hc3, alpha=0.05)
```

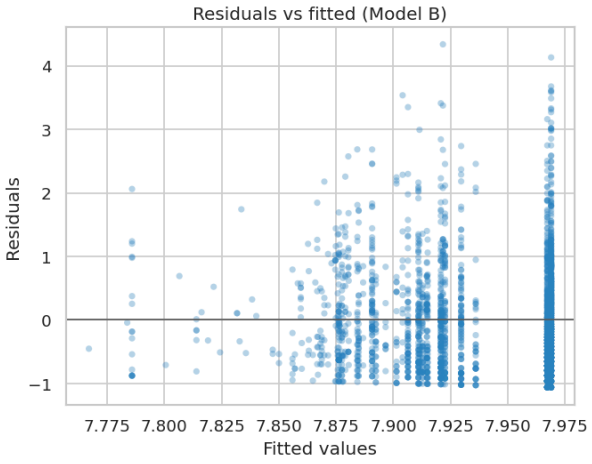
```
    return pd.DataFrame({
        "model": model_name,
        "term": terms,
        "coef": params.values,
        "se_hc3": bse.values,
        "t": tvals.values,
        "p": pvals.values,
        "ci_low": ci["ci_low"].values,
        "ci_high": ci["ci_high"].values,
        "n": int(n),
        "r2": float(r2),
    })

# ---- Model A: finalWorth ~ gdp_country_num (required)
if {"finalWorth", "gdp_country_num"}.issubset(df_clean.columns):
    dA = df_clean[["finalWorth", "gdp_country_num"]].dropna().copy()
    dA = dA[dA["gdp_country_num"].astype(float) > 0].copy()

    X = sm_api.add_constant(dA[["gdp_country_num"]].astype(float))
    y = dA["finalWorth"].astype(float)

    mA = sm_api.OLS(y, X).fit()
    mA_hc3 = mA.get_robustcov_results(cov_type="HC3")

    reg_rows.append(_result_table("A: finalWorth ~ gdp_country_num (HC3)"

# ---- Model B (preferred): log_finalWorth ~ log_gdp
if {"log_finalWorth", "log_gdp"}.issubset(df_clean.columns):
    dB = df_clean[["log_finalWorth", "log_gdp"]].dropna().copy()
    dB = dB[dB["log_gdp"].astype(float) >= 0].copy()

    X = sm_api.add_constant(dB[["log_gdp"]].astype(float))
    y = dB["log_finalWorth"].astype(float)

    mB = sm_api.OLS(y, X).fit()
    mB_hc3 = mB.get_robustcov_results(cov_type="HC3")

    reg_rows.append(_result_table("B: log_finalWorth ~ log_gdp (HC3)", mB

    # Fit plot
    fig, ax = plt.subplots(figsize=(7.2, 5.2))
    ax.scatter(dB["log_gdp"], dB["log_finalWorth"], s=18, alpha=0.25,
               color=COLOR["primary"], edgecolors="none")

    xs = np.linspace(dB["log_gdp"].min(), dB["log_gdp"].max(), 200)
    termsB = list(mB_hc3.model.exog_names)
    b0 = float(mB_hc3.params[termsB.index("const")]) if "const" in termsB
    b1 = float(mB_hc3.params[termsB.index("log_gdp")]) if "log_gdp" in te
    ax.plot(xs, b0 + b1 * xs, lw=2.5, color=COLOR["accent"])

    ax.set_title("OLS fit: log_finalWorth ~ log_gdp (HC3)")
    ax.set_xlabel("log_gdp")
    ax.set_ylabel("log_finalWorth")
    ax.text(0.02, 0.02, "Association only (not causal).", transform=ax.tr
            bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="0.7", alp
    save_fig(fig, "fig_M7_regression_fit_loglog")

    # Diagnostics
    resid = mB_hc3.resid
    fitted = mB_hc3.fittedvalues
```

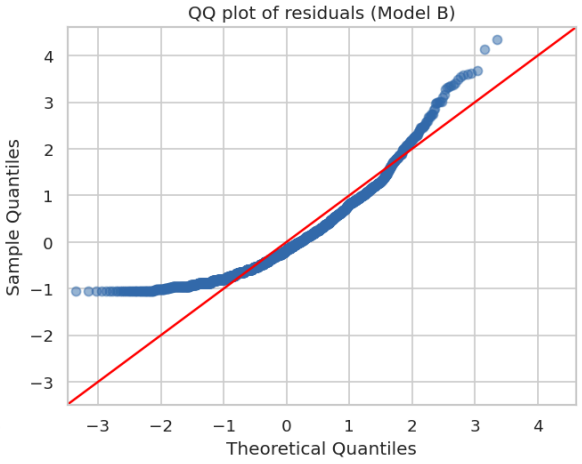
```
    fig, axes = plt.subplots(1, 2, figsize=(11.2, 4.6))
    axes[0].scatter(fitted, resid, s=18, alpha=0.35, color=COLOR["seconda
    axes[0].axhline(0, color="0.35", lw=1)
    axes[0].set_title("Residuals vs fitted (Model B)")
    axes[0].set_xlabel("Fitted values")
    axes[0].set_ylabel("Residuals")

    sm_api.qqplot(resid, line="45", ax=axes[1], markerfacecolor=COLOR["pr
    axes[1].set_title("QQ plot of residuals (Model B)")
    save_fig(fig, "fig_M7_regression_diagnostics")

    # Breusch–Pagan test
    bp = het_breuschpagan(resid, X)
    tab_bp = pd.DataFrame([{
        "lm_stat": float(bp[0]), "lm_pvalue": float(bp[1]),
        "f_stat": float(bp[2]), "f_pvalue": float(bp[3]),
        "note": "Breusch–Pagan test for heteroskedasticity (Model B resid
    }])
    save_table(tab_bp, "tab_M7_bp_test_modelB", index=False)

# ---- Optional Model C: controls + category dummies
cols_C = ["log_finalWorth", "log_gdp", "age", "selfMade", "gender", "cate
if all(c in df_clean.columns for c in cols_C):
    dC = df_clean[cols_C].dropna().copy()
    if len(dC) >= 200:
        dC["selfMade_bin"] = _coerce_boolish(dC["selfMade"])
        dC["gender_F"] = dC["gender"].astype(str).str.upper().eq("F").ast
        top_cat = dC["category"].value_counts().head(8).index
        dC = dC[dC["category"].isin(top_cat)].copy()

        mC = smf_api.ols("log_finalWorth ~ log_gdp + age + selfMade_bin +
        mC_hc3 = mC.get_robustcov_results(cov_type="HC3")
        reg_rows.append(_result_table(
            "C: log_finalWorth ~ log_gdp + controls + C(category) (HC3)",
            mC_hc3, n=len(dC), r2=mC.rsquared
        ))

# ---- Save main regression table
if reg_rows:
    tab_reg = pd.concat(reg_rows, ignore_index=True)
    save_table(tab_reg, "tab_M7_regression_main", index=False)

# ========================================
# M7.2 Robustness: drop top 1% by finalWorth
# ========================================

if {"log_finalWorth", "log_gdp", "finalWorth"}.issubset(df_clean.columns)
    d = df_clean[["log_finalWorth", "log_gdp", "finalWorth"]].dropna().co
    cutoff = d["finalWorth"].astype(float).quantile(0.99)
    d_drop = d[d["finalWorth"].astype(float) < cutoff].copy()

    X1 = sm_api.add_constant(d[["log_gdp"]].astype(float)); y1 = d["log_f
    X2 = sm_api.add_constant(d_drop[["log_gdp"]].astype(float)); y2 = d_d

    m1 = sm_api.OLS(y1, X1).fit().get_robustcov_results(cov_type="HC3")
    m2 = sm_api.OLS(y2, X2).fit().get_robustcov_results(cov_type="HC3")

    def coef_ci_on(res, term):
        terms = list(res.model.exog_names)
```

```
        ci = _ci_df(res, alpha=0.05)
        j = terms.index(term)
        return float(res.params[j]), float(ci.loc[term, "ci_low"]), float

    b1, lo1, hi1 = coef_ci_on(m1, "log_gdp")
    b2, lo2, hi2 = coef_ci_on(m2, "log_gdp")

    tab_rb = pd.DataFrame([
        {"setting": "All complete cases", "n": int(len(d)), "coef_log_gdp
        {"setting": "Drop top 1% finalWorth", "n": int(len(d_drop)), "coe
    ])
    save_table(tab_rb, "tab_M7_robust_drop_top1pct_logmodel", index=False

    # Coefficient comparison plot [Connected dumbbell]
# Assumes tab_rb exists with columns: setting, n, coef_log_gdp, ci_low, c

plot_df = tab_rb.copy()
plot_df["setting"] = plot_df["setting"].astype(str)

# Ensure exactly two rows (expected: All cases vs Drop top 1%)
if len(plot_df) >= 2:
    plot_df = plot_df.iloc[:2].copy()

fig, ax = plt.subplots(figsize=(8.6, 4.8))

# y positions: top row first
y = np.arange(len(plot_df))[::-1]

# Colors: make points clearly different
c1 = COLOR.get("primary", "#1f77b4")
c2 = COLOR.get("accent", "#ff7f0e")
c_line = COLOR.get("secondary", "#2ca02c")

# Draw CIs and points
for i, row in plot_df.iterrows():
    yi = y[i]
    pc = c1 if i == 0 else c2

    # CI segment
    ax.hlines(yi, row["ci_low"], row["ci_high"], color="0.55", lw=5.2, al
    # point estimate
    ax.scatter(row["coef_log_gdp"], yi, s=110, color=pc, edgecolor="white

    # text annotation (coef + CI + n)
    txt = f'{row["coef_log_gdp"]:.3f} [{row["ci_low"]:.3f}, {row["ci_high
    ax.text(row["ci_high"], yi, "  " + txt, va="center", ha="left", fonts

# Connected line (dumbbell connection)
x0 = float(plot_df.iloc[0]["coef_log_gdp"])
x1 = float(plot_df.iloc[1]["coef_log_gdp"])
ax.plot([x0, x1], [y[0], y[1]], color=c_line, lw=2.6, alpha=0.9, zorder=2

# Delta annotation
dx = x1 - x0
mid_x = (x0 + x1) / 2
mid_y = (y[0] + y[1]) / 2
ax.text(mid_x, mid_y + 0.12, f"Δ = {dx:+.3f}", ha="center", va="bottom",
        fontsize=10, color=c_line,
        bbox=dict(boxstyle="round,pad=0.25", fc="white", ec="0.8", alpha=
```

```
ax.axvline(0, color="0.35", lw=1.2, zorder=0)

ax.set_yticks(y)
ax.set_yticklabels(plot_df["setting"])
ax.set_xlabel("Coefficient on log_gdp (95% CI, HC3)")
ax.set_title("Robustness: log_gdp Coefficient With/Without Top-1% Exclusi

# Nice x-limits with padding
xmin = float(np.nanmin(plot_df["ci_low"]))
xmax = float(np.nanmax(plot_df["ci_high"]))
pad = 0.14 * (xmax - xmin + 1e-9)
ax.set_xlim(xmin - pad, xmax + 2.6 * pad)

ax.grid(axis="x", alpha=0.25)
plt.tight_layout()
save_fig(fig, "fig_M7_robust_coef_compare_top1pct")
```





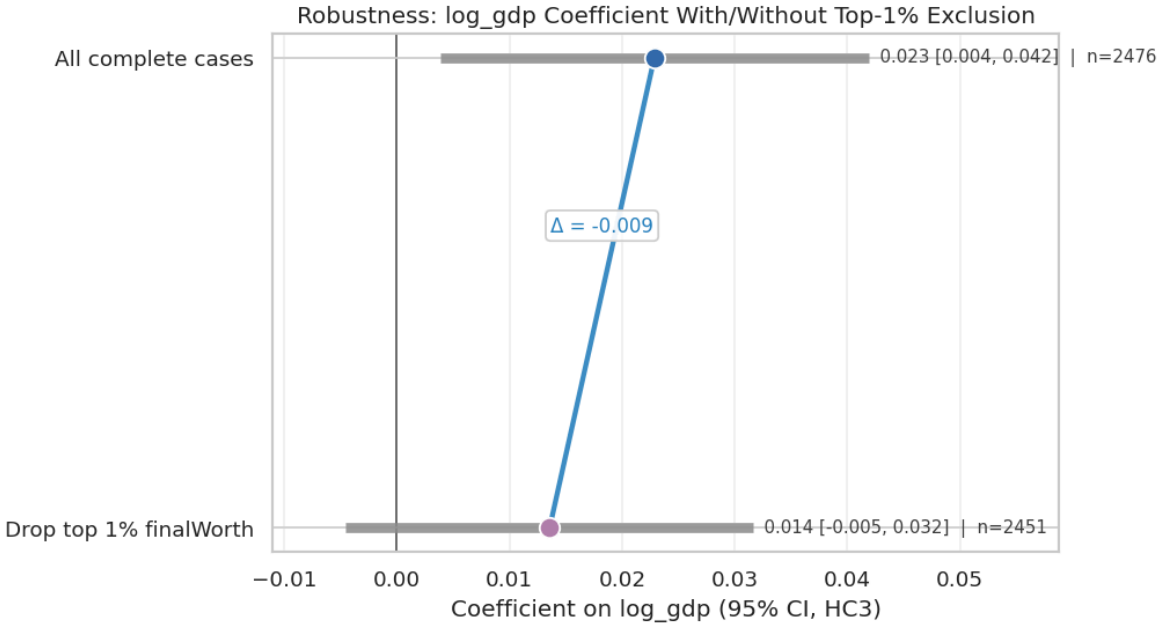


	lm_stat	lm_pvalue	f_stat	f_pvalue	note
0	2.22161	0.136091	2.221809	0.136201	Breusch–Pagan test for heteroskedasticity (Mod...

	model	term	coef	se_hc3	t
0	A: finalWorth ~ gdp_country_num (HC3)	const	4.237584e+03	2.815370e+02	15.051607
1	A: finalWorth ~ gdp_country_num (HC3)	gdp_country_num	3.960163e-11	2.113230e-11	1.873986
2	B: log_finalWorth ~ log_gdp (HC3)	const	7.266857e+00	2.835379e-01	25.629224
3	B: log_finalWorth ~ log_gdp (HC3)	log_gdp	2.287081e-02	9.707155e-03	2.356078
4	C: log_finalWorth ~ log_gdp + controls + C(cat...	Intercept	6.387722e+00	3.501042e-01	18.245205
5	C: log_finalWorth ~ log_gdp + controls + C(cat...	C(category) [T.Fashion & Retail]	1.577470e-01	8.616520e-02	1.830751
6	C: log_finalWorth ~ log_gdp + controls + C(cat...	C(category) [T.Finance & Investments]	1.183357e-01	7.750136e-02	1.526885
7	C: log_finalWorth ~ log_gdp + controls + C(cat...	C(category) [T.Food & Beverage]	7.683187e-02	8.928500e-02	0.860524
8	C: log_finalWorth ~ log_gdp + controls + C(cat...	C(category) [T.Healthcare]	-9.661944e-02	8.159839e-02	-1.184085
9	C: log_finalWorth ~ log_gdp + controls + C(cat...	C(category) [T.Manufacturing]	-9.717724e-02	7.783353e-02	-1.248527
10	C: log_finalWorth ~ log_gdp + controls + C(cat...	C(category) [T.Real Estate]	-8.594995e-02	8.011887e-02	-1.072780
11	C: log_finalWorth ~ log_gdp + controls + C(cat...	C(category) [T.Technology]	2.076338e-01	8.683333e-02	2.391176
12	C: log_finalWorth ~ log_gdp + controls + C(cat...	log_gdp	3.542268e-02	1.178139e-02	3.006664
13	C: log_finalWorth ~ log_gdp + controls + C(cat...	age	9.192149e-03	1.516982e-03	6.059496

	model	term	coef	se_hc3	t
14	C: log_finalWorth ~ log_gdp + controls + C(cat...	selfMade_bin	-1.926782e-01	4.484357e-02	-4.296675
15	C: log_finalWorth ~ log_gdp + controls + C(cat...	gender_F	-4.049913e-02	5.910736e-02	-0.685179

	setting	n	coef_log_gdp	ci_low	ci_high
0	All complete cases	2476	0.022871	0.003836	0.041906
1	Drop top 1% finalWorth	2451	0.013569	-0.004519	0.031658



Out[35]:
```
{'pdf': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M7_robust_coef_co
mpare_top1pct.pdf',
 'png': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M7_robust_coef_co
mpare_top1pct.png'}
```

M7 Mini-Summary: GDP Is a Fragile Contextual Predictor, Not a Strong Individual-Level Explanation

Model A on raw wealth is inappropriate for the heavy-tailed outcome. Model B on log wealth estimates a small positive GDP elasticity, with `log_gdp` around 0.0229 and R-squared around 0.002. Model C adds demographics and top-industry controls; the nested F-test is significant, but the explanatory power remains low (Model C R-squared about 0.048). Ten-fold validation shows an out-of-sample R-squared around 0.024, so predictive strength is modest.

The diagnostic layer matters: VIF is high for `log_gdp` and `age`, indicating collinearity pressure; quantile regression shows the GDP coefficient is visible at the 25th percentile and median but not at the upper tail. The model is

informative as an association map, not as a high-performance prediction engine.

Chapter 8 - Robustness and Sensitivity (M8)

This chapter is the final stress test. It asks whether each headline claim survives plausible changes in sample definition and modeling emphasis.

In [36]:
```
# ======================================
# M8.1 Robustness matrix (key conclusions × settings)
# ======================================

assert "df_clean" in globals(), "df_clean not found."

rows = []

# Claim 1: GDP–wealth association is positive on log scale
if {"log_finalWorth","log_gdp","finalWorth"}.issubset(df_clean.columns):
    d = df_clean[["log_finalWorth","log_gdp","finalWorth"]].dropna().copy
    cutoff = d["finalWorth"].quantile(0.99)
    settings = {
        "Overall (complete cases)": d,
        "Drop top 1% wealth": d[d["finalWorth"] < cutoff],
    }

    # Stratify by selfMade
    if "selfMade" in df_clean.columns:
        tmp = df_clean[["log_finalWorth","log_gdp","finalWorth","selfMade
        tmp["selfMade_bin"] = _coerce_boolish(tmp["selfMade"])
        settings["Self-made only"] = tmp[tmp["selfMade_bin"]==1]
        settings["Not self-made only"] = tmp[tmp["selfMade_bin"]==0]

    for name, dd in settings.items():
        if len(dd) >= 60:
            x = dd["log_gdp"].to_numpy()
            y = dd["log_finalWorth"].to_numpy()
            rho = stats.spearmanr(x, y).correlation
            lo, hi = corr_bootstrap_ci(x, y, method="spearman", n_boot=15
            rows.append({
                "claim": "C1: logWorth--logGDP association",
                "setting": name,
                "metric": "Spearman rho",
                "effect": rho,
                "ci_low": lo,
                "ci_high": hi,
                "n": len(dd),
            })

# Claim 2: selfMade differs in logWorth
if {"log_finalWorth","selfMade","finalWorth"}.issubset(df_clean.columns):
    tmp = df_clean[["log_finalWorth","selfMade","finalWorth"]].dropna().c
    tmp["selfMade_bin"] = _coerce_boolish(tmp["selfMade"])
    cutoff = tmp["finalWorth"].quantile(0.99)

    for name, dd in [("Overall (complete cases)", tmp), ("Drop top 1% wea
```

```
        g1 = dd[dd["selfMade_bin"]==1]["log_finalWorth"].to_numpy()
        g0 = dd[dd["selfMade_bin"]==0]["log_finalWorth"].to_numpy()
        if len(g1) >= 30 and len(g0) >= 30:
            d_eff = cohens_d(g1, g0)
            lo, hi = bootstrap_ci_2groups(cohens_d, g1, g0, n_boot=1500)
            rows.append({
                "claim": "C2: selfMade difference in logWorth",
                "setting": name,
                "metric": "Cohen's d",
                "effect": d_eff,
                "ci_low": lo,
                "ci_high": hi,
                "n": len(dd),
            })

# Claim 3: top 1% concentration (share of total wealth)
if {"finalWorth"}.issubset(df_clean.columns):
    d = df_clean[["finalWorth"]].dropna()
    if len(d) > 0:
        cutoff = d["finalWorth"].quantile(0.99)
        share = d.loc[d["finalWorth"] >= cutoff, "finalWorth"].sum() / d[
        rows.append({
            "claim": "C3: concentration",
            "setting": "Overall (complete cases)",
            "metric": "Top 1% share of total finalWorth",
            "effect": float(share),
            "ci_low": np.nan,
            "ci_high": np.nan,
            "n": len(d),
        })

tab_rb = pd.DataFrame(rows)
save_table(tab_rb, "tab_M8_robustness_matrix", index=False)
```

	claim	setting	metric	effect	ci_low	ci_high	n
0	C1: logWorth--logGDP association	Overall (complete cases)	Spearman rho	0.096573	0.058351	0.134188	2476
1	C1: logWorth--logGDP association	Drop top 1% wealth	Spearman rho	0.086667	0.047038	0.125528	2451
2	C1: logWorth--logGDP association	Self-made only	Spearman rho	0.095057	0.046942	0.143272	1721
3	C1: logWorth--logGDP association	Not self-made only	Spearman rho	0.150643	0.082846	0.221252	755
4	C2: selfMade difference in logWorth	Overall (complete cases)	Cohen's d	-0.118971	-0.205650	-0.032833	2640
5	C2: selfMade difference in logWorth	Drop top 1% wealth	Cohen's d	-0.121316	-0.203633	-0.035288	2613
6	C3: concentration	Overall (complete cases)	Top 1% share of total finalWorth	0.179801	NaN	NaN	2640

Out[36]:
```
{'csv': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M8_robustness_mat
rix.csv',
 'tex': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M8_robustness_mat
rix.tex'}
```

M8 Mini-Summary: Which Claims Survive Stress Testing?

The most stable claims are distributional: billionaire wealth is heavily concentrated and top-tail behavior dominates raw-scale statistics. The GDP-wealth association survives directionally after dropping the top 1%, but it remains small. The self-made difference is stable in sign but small in practical magnitude. The most fragile claims are mechanistic: country-level macro variables cannot, by themselves, explain individual billionaire wealth without strong caveats.

In [37]:
```
# INTEGRATION_UPGRADE_2026_05_01
# ======================================
# M8.5 V2-to-current integration matrix
# ======================================
# This table records continuity between the earlier exploratory notebook

integration_matrix = pd.DataFrame([
    {'dimension': 'Raw-data freeze', 'earlier exploratory contribution':
    {'dimension': 'Record identity', 'earlier exploratory contribution':
    {'dimension': 'Variable reasoning', 'earlier exploratory contribution
    {'dimension': 'Cleaning contract', 'earlier exploratory contribution'
```

```
    {'dimension': 'Distributional analysis', 'earlier exploratory contrib
    {'dimension': 'Geography and composition', 'earlier exploratory contr
    {'dimension': 'Macro framing', 'earlier exploratory contribution': 'S
    {'dimension': 'Regression and prediction', 'earlier exploratory contr
    {'dimension': 'Scope limits', 'earlier exploratory contribution': 'Ex
])

save_table(integration_matrix, 'tab_M8_v2_integration_matrix', index=Fals
```

	dimension	earlier exploratory contribution	present location	evi
0	Raw-data freeze	Hash, shape, schema, immutable raw file	Programmatic fingerprint and schema freeze	tab_M0_schema_freeze_A_sun tab_M1_ra
1	Record identity	personName not unique; composite snapshot key	Identity checks in notebook and main report	tab_M1_identity
2	Variable reasoning	Broad variable-layer discussion before modeling	Variable reasoning matrix with keep/derive/de-...	tab_M1_variable_reasoning_
3	Cleaning contract	Raw immutable, cleaned derivatives, drop/keep ...	Conservative cleaning log and dataset manifest	tab_M3_cleani tab_M3_clean_dataset_
4	Distributional analysis	Robust percentile and tail emphasis	Heavy-tail, top-1%, Gini, BCa bootstrap, CCDF ...	tab_M4_finalWorth_de tab_M4_gini_
5	Geography and composition	Country, US, China, category composition discu...	Country/category composition and bounded subna...	tab_M4_top10_country_by_ tab_M5_geo_
6	Macro framing	Scale vs development vs institution distinction	Macro-variable interpretation and derived cont...	tab_M1_variable_reasoning_ regress
7	Regression and prediction	Scale/development model set and grouped CV	HC3 OLS, nested F-test, VIF, 10-fold CV, quant...	tab_M7_regression tab_M7_kfold_cv; t
8	Scope limits	Explicit attack points and boundaries	RQ summaries and limitation notes in the execu...	rendered executed no ap

Out[37]:
```
{'csv': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M8_v2_integration
_matrix.csv',
 'tex': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M8_v2_integration
_matrix.tex'}
```

Chapter 9 - Integration Map, Report Map, and Reproducibility (M9)

This chapter documents the final integration architecture: V2 supplied the audit trail and broad reasoning; the current version supplies the stronger statistical stress tests and final report polish.

In [38]:
```
# ======================================
# M9.1 Report map (artifact registry)
# ======================================

# Build report map from ARTIFACTS. Each saved figure/table was logged aut
art = pd.DataFrame(ARTIFACTS)

# Add a suggested report section based on module
module_to_report = {
    "M0": "Introduction",
    "M1": "Introduction (Dataset overview)",
    "M2": "Statistical Analysis (Data Quality)",
    "M3": "Statistical Analysis (Cleaning)",
    "M4": "Statistical Analysis (Descriptive)",
    "M5": "Statistical Analysis (EDA)",
    "M6": "Statistical Analysis (Inferential)",
    "M7": "Statistical Analysis (Regression)",
    "M8": "Key Insights & Limitations (Robustness)",
    "M9": "Appendix / Reproducibility",
}
art["report_section"] = art["module"].map(module_to_report).fillna("TBD")
art = art[["report_section","module","kind","name","paths"]].sort_values(

save_table(art, "tab_M9_report_map", index=False)

# ======================================
# M9.2 Reproducibility runbook (print-friendly)
# ======================================

runbook = pd.DataFrame([
    {"step": 1, "action": "Place the raw CSV at ./data/raw/Billionaires S
    {"step": 2, "action": "Open ./supplemental_code.ipynb"},
    {"step": 3, "action": "Kernel → Restart & Run All"},
    {"step": 4, "action": "Verify outputs in ./output/fig and ./output/ta
    {"step": 5, "action": "Use tab_M9_report_map to reference figures/tab
])
save_table(runbook, "tab_M9_runbook", index=False)
```

	report_section	module	kind	name	
5	Introduction	M0	figure	fig_M0_variable_role_counts	"/
0	Introduction	M0	table	tab_M0_environment_snapshot	"/
2	Introduction	M0	table	tab_M0_research_questions	"/
3	Introduction	M0	table	tab_M0_research_questions_source	"/
1	Introduction	M0	table	tab_M0_schema_freeze_A_summary	"/
4	Introduction	M0	table	tab_M0_variable_roles	"/
7	Introduction (Dataset overview)	M1	figure	fig_M1_dtype_distribution	"/
11	Introduction (Dataset overview)	M1	table	tab_M1_dictionary_verification_checks	"/
8	Introduction (Dataset overview)	M1	table	tab_M1_gdp_parse_summary	"/
6	Introduction (Dataset overview)	M1	table	tab_M1_schema_summary	"/
10	Introduction (Dataset overview)	M1	table	tab_M1_v2_to_current_rq_coverage	"/
9	Introduction (Dataset overview)	M1	table	tab_M1_v2_topic_reasoning_archive	"/
15	Statistical Analysis (Data Quality)	M2	figure	fig_M2_missingness_heatmap_country_top12	"/
13	Statistical Analysis (Data Quality)	M2	figure	fig_M2_missingness_top15	"/
18	Statistical Analysis (Data Quality)	M2	figure	fig_M2_outliers_raw_vs_log_box	"/
17	Statistical Analysis (Data Quality)	M2	figure	fig_M2_us_only_state_coverage	"/
20	Statistical Analysis (Data Quality)	M2	table	tab_M2_artifact_index	"/

	report_section	module	kind	name
19	Statistical Analysis (Data Quality)	M2	table	tab_M2_dq_issues_summary
14	Statistical Analysis (Data Quality)	M2	table	tab_M2_missingness_by_country_top12
12	Statistical Analysis (Data Quality)	M2	table	tab_M2_missingness_top15

	step	action
0	1	Place the raw CSV at ./data/raw/Billionaires S...
1	2	Open ./supplemental_code.ipynb
2	3	Kernel → Restart & Run All
3	4	Verify outputs in ./output/fig and ./output/tab
4	5	Use tab_M9_report_map to reference figures/tab...

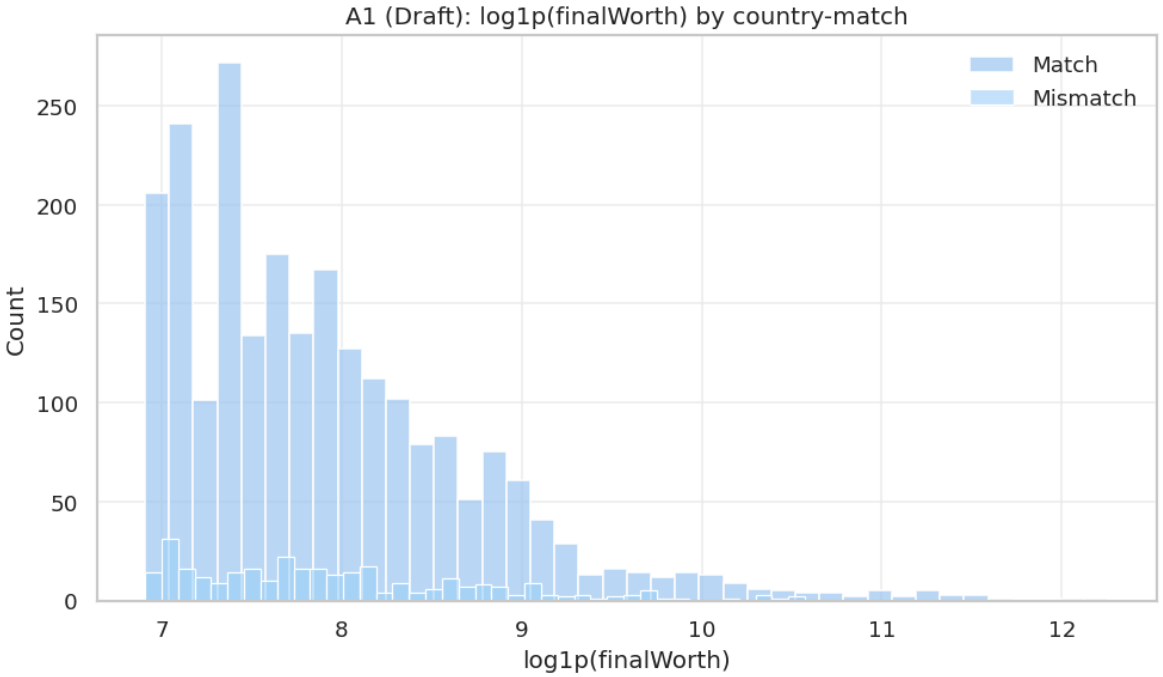
Out[38]:
```
{'csv': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M9_runbook.csv',
 'tex': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/tab/tab_M9_runbook.tex'}
```

In [39]:
```
# A1 — descriptive summary + draft inference (log scale)
from scipy import stats

df_tmp = df_clean.copy()

if ("country" in df_tmp.columns) and ("countryOfCitizenship" in df_tmp.co
    df_tmp["country_match"] = (df_tmp["country"] == df_tmp["countryOfCiti
else:
    df_tmp["country_match"] = np.nan

summary = (
    df_tmp.groupby("country_match")["finalWorth"]
          .agg(n="count", mean="mean", median="median", std="std")
          .reset_index()
)
summary["country_match"] = summary["country_match"].map({True: "Match", F
save_table(summary, "tab_A1_country_match_finalWorth_summary", index=Fals

# Draft inference on log scale (Welch t-test)
a = df_tmp.loc[df_tmp["country_match"] == True, "log_finalWorth"].dropna(
b = df_tmp.loc[df_tmp["country_match"] == False, "log_finalWorth"].dropna

if (len(a) >= 5) and (len(b) >= 5):
    tstat, pval = stats.ttest_ind(a, b, equal_var=False)

    # Effect size (Cohen's d on log scale)
    def cohens_d(x: np.ndarray, y: np.ndarray) -> float:
        nx, ny = len(x), len(y)
        sx, sy = x.std(ddof=1), y.std(ddof=1)
        sp = np.sqrt(((nx-1)*sx*sx + (ny-1)*sy*sy) / (nx+ny-2))
        return float((x.mean() - y.mean()) / sp)
```

```
    d = cohens_d(a.values, b.values)

    # Bootstrap CI for mean difference (log scale)
    rng = np.random.default_rng(42)
    B = 2000
    diffs = []
    a_arr, b_arr = a.values, b.values
    for _ in range(B):
        diffs.append(
            rng.choice(a_arr, size=len(a_arr), replace=True).mean()
            - rng.choice(b_arr, size=len(b_arr), replace=True).mean()
        )
    ci_low, ci_high = np.percentile(diffs, [2.5, 97.5])

    draft = pd.DataFrame([{
        "scale": "log1p(finalWorth)",
        "welch_t_stat": float(tstat),
        "p_value": float(pval),
        "cohens_d": float(d),
        "mean_diff_bootstrap_CI_95%": f"[{ci_low:.4f}, {ci_high:.4f}]"
    }])
    save_table(draft, "tab_A1_country_match_log_inference_draft", index=F

    # Visualization
    fig, ax = plt.subplots(figsize=(8.4, 5.0))
    sns.histplot(a, bins=40, stat="count", alpha=0.55, label="Match", ax=
    sns.histplot(b, bins=40, stat="count", alpha=0.55, label="Mismatch",
    ax.set_title("A1 (Draft): log1p(finalWorth) by country-match")
    ax.set_xlabel("log1p(finalWorth)")
    ax.set_ylabel("Count")
    ax.grid(alpha=0.25)
    ax.legend()
    save_fig(fig, "fig_A1_country_match_log_hist")

else:
    display(pd.DataFrame([{"note": "Not enough records in one or both gro
```

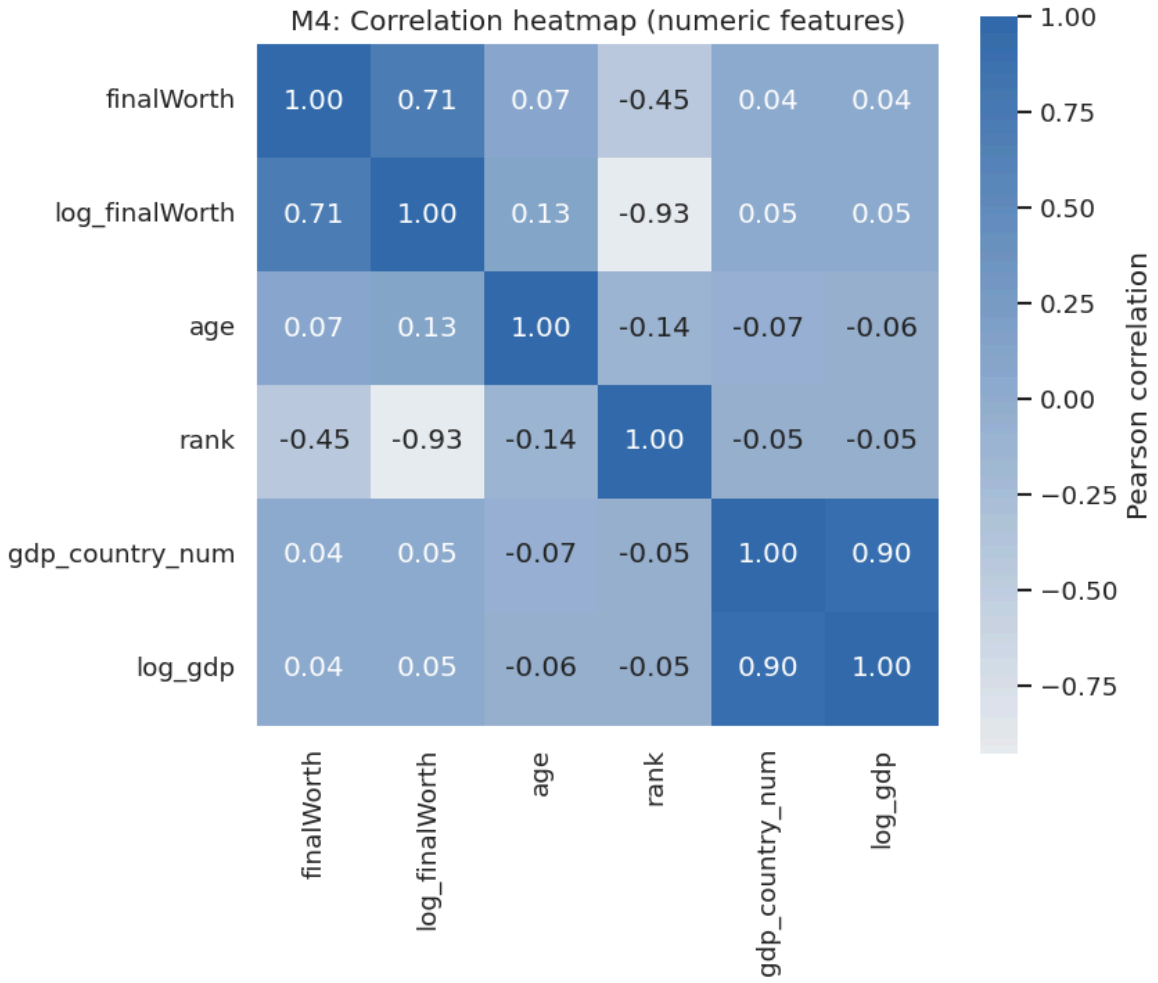
	country_match	n	mean	median	std
0	Mismatch	316	4331.962025	2500.0	5511.968848
1	Match	2324	4663.468158	2300.0	10282.763088

	scale	welch_t_stat	p_value	cohens_d	mean_diff_bootstrap_CI_
0	log1p(finalWorth)	-0.950682	0.342326	-0.05608	[-0.1431, 0.0



In [40]:
```
# Correlation heatmap among numeric columns
num_cols = [c for c in ["finalWorth", "log_finalWorth", "age", "rank", "g
corr = df[num_cols].corr(numeric_only=True)
corr_long = corr.reset_index().rename(columns={"index":"variable"})
save_table(corr_long, "tab_M4_correlation_matrix", index=False)

fig, ax = plt.subplots(figsize=(7.2, 6.2))
sns.heatmap(corr, annot=True, fmt=".2f", cmap=CMAP_PRIMARY, center=0, squ
            cbar_kws={"label": "Pearson correlation"}, ax=ax)
ax.set_title("M4: Correlation heatmap (numeric features)")
save_fig(fig, "fig_M4_correlation_heatmap")
```

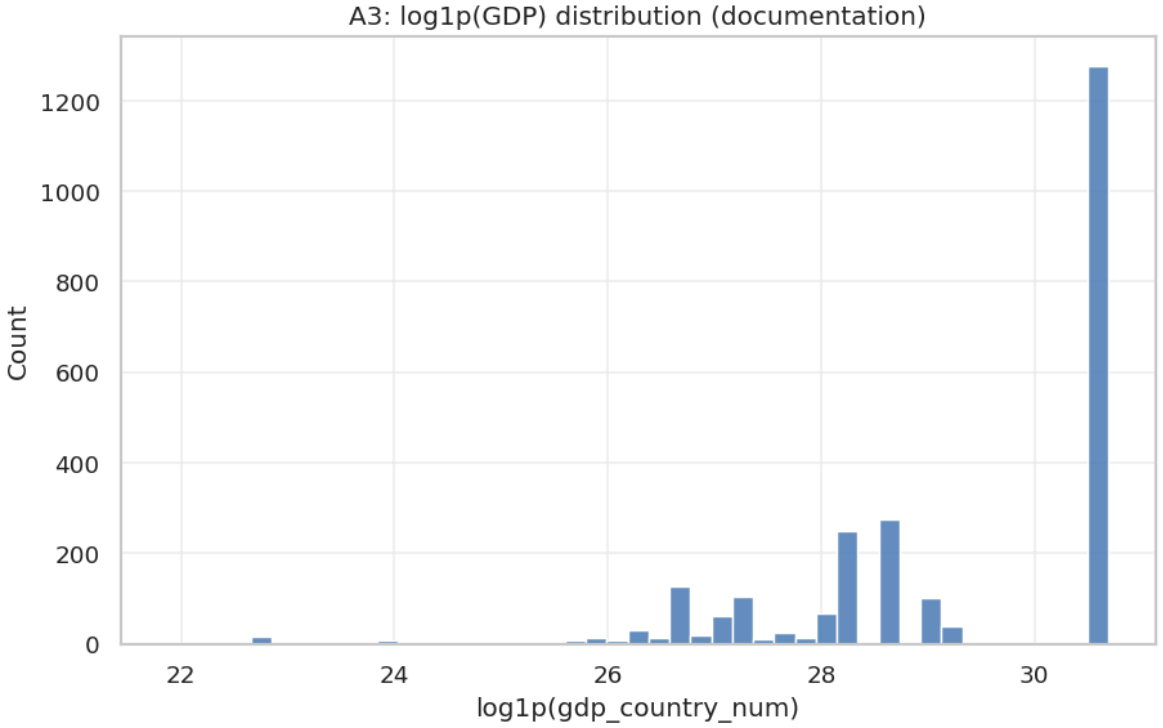
	variable	finalWorth	log_finalWorth	age	rank	gdp_cou
0	finalWorth	1.000000	0.706432	0.067053	-0.448930	
1	log_finalWorth	0.706432	1.000000	0.132698	-0.925961	
2	age	0.067053	0.132698	1.000000	-0.142686	
3	rank	-0.448930	-0.925961	-0.142686	1.000000	
4	gdp_country_num	0.037589	0.050142	-0.068638	-0.052705	
5	log_gdp	0.040908	0.045759	-0.057937	-0.045006	



Out[40]:
```
{'pdf': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M4_correlation_he
atmap.pdf',
 'png': '/mnt/e/canfiles/Common courses study/大学通识/25-26 Spring/UFUG 2
104 Applied statistics/final_project_v3/output/fig/fig_M4_correlation_he
atmap.png'}
```

In [41]:
```python
# A3 — schema slice for macro columns
macro_cols = [
    "gdp_country",
    "cpi_change_country",
    "gross_tertiary_education_enrollment",
    "gross_primary_education_enrollment_country",
    "total_tax_rate_country",
    "life_expectancy_country",
]
present = [c for c in macro_cols if c in df_raw.columns]

macro_schema = tab_schema.loc[tab_schema["column"].isin(present)].copy()
save_table(macro_schema, "tab_A3_macro_columns_schema", index=False)

# GDP parsing is now formalized in M1/M3; this plot remains as a document
if "log_gdp" in df_clean.columns:
    fig, ax = plt.subplots(figsize=(7.8, 5.0))
    sns.histplot(df_clean["log_gdp"].dropna(), bins=45, ax=ax)
    ax.set_title("A3: log1p(GDP) distribution (documentation)")
    ax.set_xlabel("log1p(gdp_country_num)")
    ax.set_ylabel("Count")
    ax.grid(alpha=0.25)
    save_fig(fig, "fig_A3_log_gdp_hist")
```

	column	dtype	missing_n	missing_pct
4	cpi_change_country	float64	184	0.069697
7	life_expectancy_country	float64	182	0.068939
8	gross_tertiary_education_enrollment	float64	182	0.068939
9	total_tax_rate_country	float64	182	0.068939
10	gross_primary_education_enrollment_country	float64	181	0.068561
11	gdp_country	str	164	0.062121



Supplemental Analyses — Meta-Assessment Remedies

Additional diagnostics and sensitivity analyses (Gini, VIF, F-test, balance check, gender test, LOWESS, K-fold CV, quantile regression, BCa bootstrap).

In [42]:
```
# Run supplemental analyses
import subprocess, sys
from pathlib import Path
paths = [
    ROOT / "code" / "remedy_run.py",
    ROOT / "code" / "remedy_run_part2.py",
]
for p in paths:
    if not p.exists():
        print(f"{p.name} -> skipped (missing)")
        continue
    r = subprocess.run([sys.executable, str(p)], capture_output=True, tex
    if r.returncode != 0:
        print("STDERR:", r.stderr[-1000:])
```

```
        raise RuntimeError(f"Supplemental script failed: {p.name}")
    print(f"{p.name}: completed")
print("Supplemental diagnostics completed.")
```

```
remedy_run.py: completed
remedy_run_part2.py: completed
Supplemental diagnostics completed.
```

Final Notebook Synthesis

After this integration pass, the appendix has two jobs. First, it is executable: all figures and tables used in the report are regenerated from the raw CSV. Second, it is argumentative: it explains why variables were selected, why some fields were excluded or de-emphasized, why GDP is treated cautiously, and why the final conclusions are bounded. This is the main lesson from comparing the true V2 draft with the current formal report: the final package should combine V2's broad audit trail with the current version's stricter statistical evidence.